Oracle Database 19c Active Data Guard 과정

Data Guard 개요와 아키텍처

무엇을 지키고, 무엇으로 이루어지며, 어디까지가 기본 기능인가

제1장

학습 목표

- Data Guard가 대응하는 다운타임의 종류를 구분한다
- Physical, Logical, Snapshot Standby와 Far Sync를 구분한다
- 전송·적용·역할 관리 서비스와 주요 프로세스를 짝짓는다
- Switchover, Failover, Fast-Start Failover를 구분한다
- Broker와 관리 도구, ADG 옵션의 라이선스 경계를 판단한다
- RAC, GoldenGate, 백업과 비교해 보호 범위를 설명한다

Agenda

- 1.2 Data Guard가 해결하는 문제 / 1.3 구성 요소
- 1.4 리두 전송 서비스와 적용 서비스
- 1.5 역할 전환 / 1.6 Broker와 관리 도구
- 1.7 Active Data Guard 옵션과 라이선스 경계
- 1.8 구성 요건과 사전 점검
- 1.9 다른 고가용성 기술과의 비교 / 1.10 운영 시 주의점

1.2

Data Guard가 해결하는 문제

- 다운타임의 종류와 대응
- RPO와 RTO로 보는 보호 수준
- 전형적인 배치 형태

다운타임의 종류

원인마다 대책이 다르다

- 계획 정지: 패치, 하드웨어 교체 → Switchover
- 서버·인스턴스 장애 → Failover
- 사이트 재해: 화재, 정전, 네트워크 단절 → 원격 Standby
- 데이터 손상: lost write, 블록 손상 → 리두 검증
- 사용자 오류는 그대로 전달됨 → 지연 적용, Flashback

RPO와 RTO

얼마나 잃어도 되고, 얼마나 멈춰도 되는가

- RPO: 잃어도 되는 데이터의 양(시간)
- RTO: 서비스가 돌아오기까지 허용 시간
- 백업 복원: RPO는 작게, RTO는 수 시간
- Data Guard: RTO 분 단위, RPO는 보호 모드가 결정
- 숫자를 먼저 합의해야 구성을 고를 수 있다

Data Guard의 정의와 도입 효과

이미 있는 리두로 사본을 유지한다

- Primary와 Standby 구성을 생성·관리·감시하는 기반
- 별도 복제 엔진 없이 Primary의 리두를 그대로 사용
- 서비스 연속성과 손실 없는 데이터 보호
- 대기 서버를 조회·백업에 활용
- 속성별 유연한 구성, Broker·EM으로 중앙 관리

전형적인 배치 형태

어떤 장애에 대비하느냐에 따라 위치가 달라진다

- 로컬 Standby: 서버·스토리지 장애, SYNC 전송이 쉬움
- 원격 Standby: 사이트 재해, 대개 ASYNC
- 로컬 + 원격: 두 종류 장애를 모두 대비
- Far Sync + 원격: 원거리에서도 손실 없는 전환
- 위치는 RPO, RTO, 왕복 지연을 함께 놓고 결정

Data Guard가 막지 못하는 것

정상 커밋된 실수는 그대로 복제된다

- 잘못된 UPDATE, TRUNCATE도 정상 리두
- Standby에 수 초 안에 그대로 적용
- 대비책 ①: Standby 하나에 지연 적용(DELAY)
- 대비책 ②: 양쪽에 Flashback Database
- 백업 정책은 Data Guard 도입 뒤에도 유지

1.3

Data Guard 구성 요소

- Primary, Standby, 구성
- Physical, Logical, Snapshot Standby
- Far Sync와 캐스케이딩

Primary와 Standby, 그리고 구성

역할은 바뀌고 이름은 남는다

- Primary는 구성 안에 하나, Standby는 최대 30개
- 30 = LOG_ARCHIVE_DEST_n 31개 중 로컬 1개 제외
- 같은 DB_NAME과 DBID, 다른 DB_UNIQUE_NAME
- 목적지·Broker·서비스 이름은 DB_UNIQUE_NAME 기준
- 대상은 이름보다 현재 역할로 부른다

Physical Standby

블록 단위로 같은 사본

- Redo Apply: 미디어 복구로 리두 적용
- 데이터 유형·DDL 제약 없음
- MOUNT 또는 READ ONLY, 적용 중 조회는 옵션
- 컨트롤 파일 유형 STANDBY, DDL은 거부
- Physical Standby의 백업으로 Primary 복원 가능

Logical Standby

논리적으로 같고, 물리 구조는 달라도 된다

- SQL Apply: LogMiner로 리두를 SQL로 바꿔 실행
- READ WRITE로 열려 보고용 인덱스·뷰 추가 가능
- DATABASE GUARD로 관리 대상 테이블 변경 차단
- 지원하지 않는 데이터 유형이 있음
- 롤링 업그레이드, 스키마 단위 skip 필터

Snapshot Standby

잠시 쓰고, 버리고, 다시 따라잡는다

- Physical Standby를 명령 하나로 전환
- 보장 복원 지점 생성, FRA 필요
- 전환 중에도 리두는 받아서 보관
- 되돌리면 변경은 버리고 밀린 리두 적용
- ADG 옵션 없이 쓰는 기본 기능

Far Sync와 캐스케이딩

데이터 없이 리두만 중계한다

- Far Sync: 컨트롤 파일, SRL, 아카이브 로그만 보유
- Primary → Far Sync는 SYNC, Far Sync → 원격은 ASYNC
- Primary 역할을 넘겨받을 수 없음
- Far Sync와 실시간 캐스케이딩은 ADG 옵션
- 단일 장애점 대비: 2개 배치 또는 ALTERNATE 경로

Standby 유형 비교

재해 복구의 기본 선택지는 Physical

- Physical: 블록 동일, 제약 없음, Failover에 가장 적합
- Logical: 논리 동일, READ WRITE, 데이터 유형 제약
- Snapshot: 일시적 READ WRITE, 리두는 보관만
- DATABASE_ROLE로 유형, OPEN_MODE로 열린 모드 확인
- 한 구성 안에 유형을 섞어 쓸 수 있다

1.4

리두 전송 서비스와 적용 서비스

- 세 가지 서비스
- Primary·Standby 프로세스와 SRL
- 갭 해소와 보호 모드 미리 보기

세 가지 서비스

전송, 적용, 역할 관리

- 리두 전송 서비스: Primary의 리두를 목적지로 (2·3장)
- 적용 서비스: Redo Apply 또는 SQL Apply (7·12장)
- 역할 관리 서비스: Switchover, Failover (16장)
- 전송과 적용은 서로 기다리지 않는다
- 손실은 전송이, 전환 속도는 적용이 결정

Primary 쪽 프로세스

LGWR는 그대로, 전송은 NSS와 TTnn

- LGWR: 온라인 리두 로그 기록
- NSSn: SYNC 목적지 전송, LGWR가 확인을 기다림
- TTnn: ASYNC 목적지 전송, LGWR는 관여 안 함
- ARCn: 로컬 아카이브와 갭 해소, 기본 4개
- 11g까지의 LNS는 19c의 NSS 또는 TTnn

Standby 쪽 프로세스

받는 RFS, 적용하는 MRP0

- RFSn: Primary 연결마다 하나, SRL에 기록
- ARCn: 다 찬 SRL을 아카이브
- MRP0와 PRnn: Physical Standby의 Redo Apply
- LSP: Logical Standby의 SQL Apply
- 12.1부터 SRL이 있으면 실시간 적용이 기본

Standby Redo Log

받은 리두를 담는 Standby 전용 로그

- RFS가 받은 리두를 즉시 기록하는 곳
- 없으면 아카이브 로그로만 기록, 전환을 기다림
- SYNC, 실시간 적용, Far Sync의 전제
- 역할 전환 대비로 Primary에도 생성
- 크기는 온라인 로그와 같게, 개수는 그룹 수 + 1

V$DATAGUARD_PROCESS

프로세스를 역할 이름으로 보는 뷰

- 12.2부터 제공, Primary·Standby 양쪽에서 조회
- ROLE: log writer, async ORL single, managed recovery 등
- ACTION: 지금 하고 있는 일
- V$MANAGED_STANDBY는 19c에서 디프리케이트
- 새 점검 스크립트는 V$DATAGUARD_PROCESS 기준

아카이브 갭의 자동 해소

끊긴 동안의 리두는 나중에 채운다

- 연결이 끊긴 동안 생긴 로그 = 아카이브 갭
- Primary ARCn이 확인(ping) 후 빠진 로그 전송
- Standby는 FAL_SERVER에 요청해 받아 옴
- Primary에 로그가 남아 있어야 자동 해소
- 삭제 정책은 Standby 적용 이후로

보호 모드 미리 보기

커밋 조건과 장애 시 동작

- Maximum Protection: 쓰지 못하면 Primary 종료
- Maximum Availability: 쓰지 못하면 잠시 비동기로
- Maximum Performance: 로컬 기록만으로 커밋 (기본)
- Availability는 두 번째 장애가 겹치지 않는 한 무손실
- 상세와 전환 방법은 2장

1.5

역할 전환

- Switchover와 Failover
- Fast-Start Failover
- 전환과 클라이언트 연결

Switchover와 Failover

계획된 교대와 비상 전환

- Switchover: Primary 정상, 데이터 손실 없음
- 원래 Primary는 자동으로 Standby가 됨
- Failover: Primary 장애, 손실은 보호 모드에 따름
- 원래 Primary는 구성에서 빠짐 → reinstate 또는 재생성
- reinstate는 Flashback Database가 전제

Fast-Start Failover

사람 대신 옵저버가 판단한다

- Primary 장애 시 지정 Standby로 자동 Failover
- Broker 구성에서만 사용
- 옵저버는 제3의 서버에 두는 것이 원칙
- 옵저버와 Standby가 함께 연결을 잃어야 실행
- ADG 옵션 불필요, 원래 Primary는 자동 reinstate 시도

역할 전환과 클라이언트 연결

애플리케이션은 새 Primary를 어떻게 찾는가

- Primary 주소만 적은 접속 문자열은 전환 뒤 실패
- 역할 기반 서비스: Primary 역할일 때만 시작
- 접속 기술자에 두 서버 주소를 모두 기재
- FAN 이벤트로 끊긴 세션을 빨리 정리
- 연결 설계는 구성과 별개로 미리 준비

사례: 스토리지 장애 앞에서의 선택

복구 네 시간을 기다릴 것인가

- 새벽, Primary 스토리지 컨트롤러 고장
- Switchover는 불가: Primary가 정상이 아님
- transport lag로 잃을 양, apply lag로 걸릴 시간 판단
- Maximum Availability였다면 무손실 Failover
- 수리 후 reinstate 또는 Standby 재생성

1.6

Data Guard Broker와 관리 도구

- Broker의 구조
- 관리 인터페이스 선택
- Broker 구성에서의 운영 규칙

Broker의 구조

DMON과 구성 파일

- 생성·유지·감시를 자동화하는 분산 관리 프레임워크
- EE에 포함, 별도 라이선스 없음
- 인스턴스마다 DMON 프로세스가 상태 교환
- 구성 정보는 DG_BROKER_CONFIG_FILE1·2에 저장
- DG_BROKER_START=TRUE로 기동, FSFO·FAN의 기반

관리 인터페이스 선택

DGMGRL, EM, SQL

- Broker 구성: DGMGRL 명령줄
- Broker 구성: Enterprise Manager Cloud Control
- Broker 구성에서도 조회는 SQL로 가능
- Broker 미사용: SQL 명령과 파라미터만 사용
- 이 과정: SQL로 원리 → 13장부터 Broker

Broker가 맡는 일과 SQL로 하는 일

같은 작업, 다른 경로

- 전송 목적지: 속성 지정 vs LOG_ARCHIVE_DEST_n
- 적용 기동: SET STATE vs RECOVER MANAGED STANDBY
- 점검: VALIDATE DATABASE vs 여러 V$ 뷰
- Standby 생성 자체는 Broker 밖에서
- Broker 구성에서는 SQL로 설정을 바꾸지 않는다

1.7

Active Data Guard 옵션과 라이선스 경계

- EE 기본 기능과 옵션 기능
- 옵션 기능의 쓰임새
- 사용 여부 확인과 문서화

기본 Data Guard와 옵션 기능

기준은 적용하면서 여는가

- EE 기본: Physical·Logical·Snapshot, Broker, FSFO
- EE 기본: 적용을 멈춘 READ ONLY 조회
- 옵션: Real-Time Query (READ ONLY WITH APPLY)
- 옵션: DML 리다이렉션, Far Sync, 실시간 캐스케이딩
- 옵션: 자동 블록 복구, Standby 블록 변경 추적

옵션 기능의 쓰임새

무엇을 위해 사는 옵션인가

- Real-Time Query: 보고·조회 업무를 Standby로
- DML 리다이렉션: 조회 위주 앱의 가끔 있는 쓰기
- GTT·PTT 갱신: 임시 결과를 쓰는 보고서
- Far Sync: 원거리 무손실 보호
- 블록 변경 추적: Standby 증분 백업 가속

옵션 기능은 잠겨 있지 않다

명령은 성공하고, 책임은 남는다

- 라이선스 없이도 READ ONLY WITH APPLY 명령은 성공
- 적용 중지 → OPEN → 적용 재기동 순서면 바로 옵션 사용
- 장애 대응·점검 중 무심코 실행되기 쉬움
- 옵션 미보유 사이트는 MOUNT 기동을 절차서에 명시
- 보유 여부를 운영 문서 첫 장에 기록

사용 여부 확인

OPEN_MODE가 가장 확실한 근거

- Standby V$DATABASE.OPEN_MODE = READ ONLY WITH APPLY
- DBA_FEATURE_USAGE_STATISTICS의 사용 이력
- CDB에서는 CDB_FEATURE_USAGE_STATISTICS 조회
- 읽기 전용 Standby는 이력을 기록하지 못함
- 판정은 계약서 기준, 뷰는 보조 근거

1.8

구성 요건과 사전 점검

- 소프트웨어 요건
- 하드웨어와 운영체제
- 데이터베이스 수준 점검 항목

소프트웨어 요건

같은 릴리스의 Enterprise Edition

- Data Guard는 EE 전용, SE에는 없음
- 모든 데이터베이스가 같은 릴리스
- 예외: Logical Standby 롤링 업그레이드
- 실무에서는 RU 수준까지 일치
- ASM·OMF는 양쪽을 같은 조합으로

하드웨어와 운영체제

달라도 되지만, 작으면 곤란하다

- CPU 수, 메모리, 스토리지 구성은 달라도 동작
- Failover 뒤 업무를 감당할 용량은 필요
- 혼합 플랫폼은 일부 조합만 허용
- 확인: MOS 413484.1(Physical), 1085687.1(Logical)
- 같은 서버 배치는 학습·시험용

데이터베이스 수준 점검 항목

같아야 하는 것과 달라야 하는 것

- 같아야 함: 릴리스·RU, DB_NAME, DBID, 문자셋
- 같게 둠: DB_BLOCK_SIZE, COMPATIBLE
- 달라야 함: DB_UNIQUE_NAME
- Primary 전제: ARCHIVELOG, FORCE LOGGING
- 양쪽 이름 해석: 리스너, tnsnames.ora

1.9

다른 고가용성 기술과의 비교

- RAC, GoldenGate와의 비교
- 백업·Flashback·스토리지 미러링
- 기술을 겹쳐 쓰는 방식

RAC와의 비교

인스턴스를 지키는가, 데이터를 지키는가

- RAC: 데이터베이스 하나를 여러 인스턴스가 공유
- 서버 장애에는 수 초 단위로 대응
- 데이터 파일은 한 벌, 스토리지·사이트 장애는 대응 불가
- Data Guard: 별도 사본, 스토리지·사이트 장애 대응
- RAC Primary + RAC Standby 조합이 흔함

GoldenGate와의 비교

논리 복제와 물리 복제

- GoldenGate: 리두에서 변경을 뽑아 논리 복제
- 양방향, 이기종, 버전이 다른 대상 가능
- 무중단 이전, active-active 구성에 강함
- 비동기·데이터 유형 제약 → 무손실 재해 복구는 Physical
- 별도 라이선스 제품

백업, Flashback, 스토리지 미러링

Data Guard가 대신하지 않는 것

- RMAN 백업: 모든 전략의 바닥, RTO가 김
- Flashback Database: 논리 오류 되감기, reinstate 기반
- 스토리지 미러링: 손상된 블록도 그대로 복제
- Data Guard는 리두 적용 중 검증, 손상이 번지지 않음
- 백업은 Physical Standby에서 받아 Primary 부하 경감

기술을 겹쳐 쓰는 방식

MAA의 계층 구조

- 기본: 단일 인스턴스 + RMAN 백업 + Flashback
- 서버 장애 대비: + RAC
- 사이트 재해 대비: + Data Guard (필요 시 ADG)
- 무중단 이전·다중 사이트 쓰기: + GoldenGate
- 시스템별 RPO·RTO에 맞는 계층을 고른다

사례: 어떤 기술부터 도입할까

단일 서버 병원 정보 시스템

- 요구: 30분 내 복구, 데이터 손실 없음, 패치 중단 최소
- RAC: 서버 장애는 해결, 스토리지 위험은 남음
- Physical Standby + Maximum Availability로 세 요구 충족
- 패치는 Standby 선적용 후 Switchover
- 조회 분산이 필요해지면 그때 ADG 옵션 검토

1.10

운영 시 주의점

- 접속 대상과 상태 확인
- 기본값과 라이선스
- 감시 기준과 삭제 정책

운영 시 주의점

경보 없이 생기는 문제들

- 접속한 노드부터 확인: DB_UNIQUE_NAME, DATABASE_ROLE
- 상태는 역할·열린 모드·적용 여부의 조합으로
- 기본 보호 모드(Performance)를 그대로 쓰는지 점검
- 정기 Switchover 훈련, 아카이브 삭제 정책 확인
- 감시 기준은 살아 있는가가 아니라 따라오는가

요약

- 구성 = Primary 1 + Standby 최대 30, 리두로 동기화
- Physical(블록 동일)·Logical(SQL Apply)·Snapshot(시험용), Far Sync는 중계
- 전송(LGWR·NSS·TTnn·ARCn)과 적용(RFS·MRP0·LSP)은 분리
- 갭은 ARCn·FAL로 자동 해소, 기본 보호 모드는 Performance
- Switchover는 계획 교대, Failover는 비상 전환, FSFO는 자동
- 적용 중 조회·Far Sync·DML 리다이렉션은 ADG 옵션

퀴즈 (1/2)

- Q1. Standby 유형이 아닌 것은? → Primary
- Q2. Standby 최대 개수와 근거는? → 30개, LOG_ARCHIVE_DEST_n
- Q3. Snapshot Standby는 리두를 받지 않는다? → 틀림, 받아서 보관

퀴즈 (2/2)

- Q4. ASYNC 목적지로 보내는 프로세스는? → TTnn
- Q5. 스토리지 고장 시 전환과 원래 Primary 처리는? → Failover, reinstate
- Q6. 옵션 없이 허용되는 운영은? → 적용 중지 후 READ ONLY 조회

실습 개요

- 01 현재 구성 파악: 이름·역할·열린 모드 (1.3)
- 02 Data Guard 프로세스 식별: V$DATAGUARD_PROCESS (1.4)
- 03 라이선스 확인: 에디션, 사용 이력, OPEN_MODE (1.7)
- 04 Standby 유형별 상태 확인: Snapshot 전환과 복귀 (1.3)
- 05 구성 전 점검표: 릴리스·플랫폼·문자셋·파라미터 (1.8)

Oracle Database 19c Active Data Guard 과정

데이터 보호 모드와 리두 전송 서비스

커밋은 어디까지 기다리는가, 무엇을 잃을 수 있는가

제2장

학습 목표

- 전송 모드와 보호 모드의 관계를 설명한다
- NSS, TTnn, ARCn, RFS의 리두 경로를 그린다
- SYNC/ASYNC, AFFIRM/NOAFFIRM 조합을 비교한다
- LOG_ARCHIVE_DEST_n 주요 속성을 읽고 작성한다
- 세 보호 모드의 손실 보장과 Primary 동작을 구분한다
- 모드 전환 절차와 선택 기준을 적용한다

Agenda

- 2.2 리두 전송 서비스의 구조
- 2.3 전송 방식과 확인 응답
- 2.4 LOG_ARCHIVE_DEST_n 주요 속성
- 2.5 세 가지 데이터 보호 모드
- 2.6 보호 모드 설정과 전환
- 2.7 보호 모드 선택 기준 / 2.8 운영 시 주의점

2.2

리두 전송 서비스의 구조

- 리두 소스와 리두 목적지
- 전송 프로세스와 SRL
- 전송 모드와 보호 모드의 관계

리두 소스와 리두 목적지

LOG_ARCHIVE_DEST_n 하나가 목적지 하나

- 소스: Primary, Far Sync, 캐스케이딩 Standby
- 목적지: Standby, Far Sync, 로컬 아카이브
- LOG_ARCHIVE_DEST_1~31, 원격은 최대 30개
- 전송은 SRL·아카이브 로그 기록으로 끝난다

Primary 쪽 전송 프로세스

방식에 따라 일하는 프로세스가 다르다

- LGWR: 로컬 기록, SYNC면 NSS 확인을 기다림
- NSSn: SYNC 목적지마다 하나, RFS로 즉시 전송
- TTnn: ASYNC 목적지, 버퍼나 온라인 로그에서 읽음
- ARCn: 로컬 아카이브와 갭 해소

Standby 수신과 SRL

RFS가 받아 SRL에 쓴다

- Primary 전송 프로세스마다 RFS 하나
- SRL: 받은 리두 전용, 온라인 로그와 같은 크기
- SRL이 없으면 아카이브 로그에 직접 기록
- 세 보호 모드 모두 SRL을 요건으로 둠

전송 모드와 보호 모드

둘은 따로 설정한다

- 전송 모드: 목적지별 SYNC/ASYNC, AFFIRM
- 보호 모드: 구성 전체 규칙, ALTER DATABASE로 설정
- 보호 모드는 특정 전송 모드를 요구한다
- 전송 모드만 바꿔서는 보호 모드가 정해지지 않음
- 올릴 때 속성 먼저, 내릴 때 모드 먼저

커밋 하나가 지나가는 경로

Maximum Availability + SYNC 기준

- LGWR가 로컬 쓰기와 NSS 전달을 동시에 시작
- NSS가 RFS로 리두 전송
- NOAFFIRM은 수신 즉시, AFFIRM은 SRL 기록 후 확인
- 로컬 쓰기와 원격 확인이 모두 끝나야 커밋 완료
- ASYNC면 로컬 쓰기에서 커밋이 끝난다

2.3

전송 방식과 확인 응답

- SYNC와 ASYNC
- AFFIRM과 NOAFFIRM, FASTSYNC
- 조합별 비용과 손실 가능성

SYNC

커밋이 Standby 확인을 기다린다

- 목적지 도착 확인 후에 커밋 완료
- 활성 SYNC 목적지 모두를 기다림
- 로컬 쓰기와 원격 전송은 병렬 진행
- 비용은 log file sync 대기 증가로 드러남
- 거리와 커밋 빈도가 클수록 영향 큼

ASYNC

커밋은 기다리지 않는다

- 기본값, 로컬 기록 즉시 커밋
- TTnn이 뒤따라 전송
- 손실 범위 = 그 시점의 transport lag
- 네트워크 지연이 커밋 시간에 붙지 않음
- 밀리면 온라인 로그 읽기 부하 발생

AFFIRM과 NOAFFIRM

확인 응답을 언제 보내는가

- AFFIRM: SRL 쓰기 완료 후 확인
- NOAFFIRM: 수신 즉시, 쓰기 완료 전 확인
- 기본값: SYNC는 AFFIRM, ASYNC는 NOAFFIRM
- ASYNC AFFIRM은 디프리케이트
- NOAFFIRM 위험: 확인 직후 Standby도 장애

FASTSYNC

SYNC NOAFFIRM에 붙인 이름

- 파라미터 키워드는 없고 Broker LogXptMode 값
- 커밋 경로에서 Standby 디스크 I/O 제외
- 같은 지연 예산으로 조금 더 먼 거리 가능
- Maximum Availability 전용

네 가지 조합 비교

손실 가능성과 비용의 맞바꿈

- SYNC AFFIRM: SRL 기록 확인, 단일 장애 무손실
- SYNC NOAFFIRM: 수신 확인, 동시 장애 때만 손실
- ASYNC NOAFFIRM: 기다림 없음, lag만큼 손실
- ASYNC AFFIRM: 디프리케이트

SYNC 목적지가 여럿일 때

가장 느린 목적지가 커밋 시간을 정한다

- 활성 SYNC 목적지 모두의 확인을 기다림
- 하나가 NET_TIMEOUT을 넘기면 그것만 실패 처리
- 보호 판단은 동기화된 목적지 하나로 충분
- 먼 목적지는 ASYNC로 두는 편이 낫다

전환 전에 잴 것

숫자로 비용을 예상한다

- Primary와 Standby 사이 왕복 지연
- 업무 피크 시간의 초당 커밋 수
- 전환 전후 log file sync 평균 대기
- 허용 범위 초과 시 FASTSYNC나 Far Sync 검토

2.4

LOG_ARCHIVE_DEST_n 주요 속성

- 목적지 정의와 식별
- VALID_FOR와 역할 전환
- 실패 처리와 확인 뷰

목적지 정의의 기본 형태

LOCATION 또는 SERVICE는 필수

- SERVICE=Net 서비스 이름, LOCATION=로컬 경로
- 목적지마다 LOG_ARCHIVE_DEST_STATE_n (기본 ENABLE)
- DEFER: 정의는 두고 전송만 멈춤
- 속성 하나만 바꾸는 문법 없음, 문자열 전체 재지정
- 철자 오류는 ORA-16024로 거부

DB_UNIQUE_NAME과 LOG_ARCHIVE_CONFIG

목적지가 누구인지 밝힌다

- 받는 쪽 DB_UNIQUE_NAME 값과 일치해야 함
- DB_NAME이 같은 Primary·Standby를 구분
- DG_CONFIG 목록에 없으면 ORA-16053
- 등록 목록은 V$DATAGUARD_CONFIG로 확인

VALID_FOR

역할에 따라 목적지를 켜고 끈다

- (로그 유형, 역할) 한 쌍으로 지정
- 로그: ONLINE / STANDBY / ALL_LOGFILES
- 역할: PRIMARY / STANDBY / ALL_ROLES
- 맞지 않는 목적지는 오류 없이 무시
- 역할 전환 뒤 자동으로 쓰이기 시작

VALID_FOR 조합

유일하게 잘못된 조합은 하나

- 원격 목적지 일반형: ONLINE_LOGFILES, PRIMARY_ROLE
- 로컬 아카이브: ALL_LOGFILES, ALL_ROLES
- STANDBY_LOGFILE, PRIMARY_ROLE은 ORA-16026
- ONLINE_LOGFILE, STANDBY_ROLE은 논리 Standby 전용
- 생략 시 ALL_LOGFILES, ALL_ROLES

NET_TIMEOUT과 REOPEN

얼마나 기다리고 언제 다시 시도하나

- NET_TIMEOUT: SYNC 응답 대기 상한, 기본 30초
- 범위 1~1200초, TCP의 긴 타임아웃 회피
- Maximum Availability의 커밋 정지 시간 상한
- REOPEN: 재연결 최소 간격, 기본 300초
- 재연결 후 밀린 로그는 갭 해소로 전송

MAX_FAILURE, ALTERNATE, MANDATORY

포기와 대체, 강제

- MAX_FAILURE: 연속 실패 허용 횟수, 기본 0(무제한)
- 한도 도달 시 재시도 중단, ALTERNATE로 전환
- ALTERNATE: Far Sync 장애 대비 경로에 주로 사용
- MANDATORY 원격 목적지는 Primary 정지 위험

그 밖의 속성

라이선스와 다른 장 몫을 구분

- COMPRESSION=ENABLE: Advanced Compression 옵션 필요
- DELAY: 적용만 늦춤, 7장
- NOREGISTER: 일반 물리 Standby에서는 쓰지 않음
- Broker 속성: NetTimeout, ReopenSecs, RedoCompression

설정 결과 확인

V$ARCHIVE_DEST로 해석 결과를 본다

- TRANSMIT_MODE, AFFIRM, NET_TIMEOUT, REOPEN_SECS
- VALID_NOW: YES 또는 WRONG VALID_ROLE 등
- STATUS와 ERROR로 연결 실패 확인
- 동기화 여부는 V$ARCHIVE_DEST_STATUS

역할 전환을 대비한 대칭 구성

거울처럼 같은 모양의 목적지

- 양쪽 dest_1: 로컬, ALL_LOGFILES, ALL_ROLES
- 양쪽 dest_2: 상대 방향, ONLINE_LOGFILES, PRIMARY_ROLE
- 전송 방식까지 대칭으로 맞춤
- 비대칭이면 Switchover 후 보호 요건이 깨짐

2.5

세 가지 데이터 보호 모드

- Performance, Availability, Protection
- 장애 유형별 결과
- PROTECTION_MODE와 PROTECTION_LEVEL

Maximum Performance

기본 모드, Primary 성능 우선

- 로컬 기록 즉시 커밋
- Standby로는 비동기 전송
- 손실 가능 범위 = transport lag
- 요건: SRL, ASYNC 목적지 하나 이상

Maximum Availability

가용성을 해치지 않는 최고 보호

- 로컬 기록 + Standby 확인 후 커밋
- 응답 없으면 NET_TIMEOUT 후 계속 진행
- 동기화 Standby가 없으면 비동기화 상태로 운영
- 갭 해소 후 자동으로 원래 수준 복귀
- 요건: SRL, SYNC(또는 FASTSYNC) 목적지

Maximum Protection

무손실을 위해 서비스를 포기한다

- 로컬 기록 + Standby SRL 기록 후 커밋
- 동기화 Standby가 없으면 Primary 인스턴스 종료
- 마지막 동기 Standby의 정상 종료는 거부(ORA-01154)
- Standby 2개 이상 권장, Far Sync 미지원
- 요건: SRL, SYNC AFFIRM 목적지

세 모드 비교

위로 갈수록 Primary가 Standby에 묶인다

- 손실: 없음(이중 장애) / 없음(단일 장애) / lag만큼
- 전송: SYNC AFFIRM / SYNC / ASYNC
- 확인 없을 때: 종료 / NET_TIMEOUT 후 진행 / 무관
- 성능 영향: 가장 큼 / 중간 / 거의 없음

장애 유형별 결과

Standby 하나인 구성 기준

- Primary만 장애: Protection·Availability 무손실
- Standby만 장애: Protection은 Primary 종료
- 네트워크 단절: Availability는 보호 수준 하락
- 단절 후 Primary 장애: 단절 이후 리두 손실

PROTECTION_MODE와 PROTECTION_LEVEL

설정값과 실제 달성 수준

- MODE: 설정한 모드, LEVEL: 지금 달성한 수준
- 평소에는 두 값이 같다
- 동기화 Standby가 없으면 LEVEL이 RESYNCHRONIZATION
- MODE만 감시하면 보호 붕괴를 놓친다

RESYNCHRONIZATION 원인 구분

같은 값, 세 가지 원인

- 목적지 STATUS=ERROR와 ORA 번호: 전송 장애
- STATUS=VALID인데 SYNC 목적지 없음: 설정 요건 미충족
- 잠깐 나타났다 사라짐: 정상적인 따라잡기
- 모드 변경 직후 로그 스위치로 빨리 해소

2.6

보호 모드 설정과 전환

- 설정 전 요건
- SQL과 Broker로 설정
- Maximum Protection으로 가는 경로

설정 전 갖출 요건

요건 없이 올리면 거부되거나 이름만 바뀐다

- 대상 Standby의 SRL (V$STANDBY_LOG)
- Primary에도 SRL
- 목적지 전송 속성을 모드에 맞게 변경
- Standby 쪽에도 상대 방향 목적지
- Redo Apply 동작, 갭 없음

SQL로 모드 변경

Primary에서 한 문장

- 전송 속성 먼저: SYNC AFFIRM 등
- ALTER DATABASE SET STANDBY DATABASE TO MAXIMIZE ...
- 키워드: PROTECTION, AVAILABILITY, PERFORMANCE
- 제어 파일에 기록, 재기동 후에도 유지
- 로그 스위치 후 PROTECTION_LEVEL 확인

Maximum Protection으로 가는 경로

출발점은 동기화된 Maximum Availability

- Performance에서 직접: ORA-01126
- Availability라도 LEVEL이 RESYNC면: ORA-01126
- 순서: SYNC AFFIRM → Availability → 동기화 확인 → Protection
- 마운트 상태 우회는 권하지 않음
- 내릴 때는 열린 상태에서 가능

Broker로 설정할 때

LogXptMode와 PROTECTION MODE

- 순서는 같다: SRL, 전송 방식, 보호 모드
- LogXptMode: SYNC, FASTSYNC, ASYNC
- EDIT CONFIGURATION SET PROTECTION MODE AS ...
- Primary의 LogXptMode도 함께 설정
- RedoRoutes가 있으면 그 값이 우선

2.7

보호 모드 선택 기준

- 허용 손실과 커밋 지연
- 거리, Standby 수, 자동 Failover
- 업무 유형별 예

허용 손실과 커밋 지연

기술 설정이기 전에 업무 결정

- 몇 초 손실을 허용하는가 → Performance
- 한 건도 못 잃는가 → Availability + SYNC
- Primary 정지도 감수하는가 → Protection
- 손실 비용과 중단 비용을 비교

거리와 네트워크

SYNC는 커밋마다 왕복 지연을 낸다

- 같은 도시 안: 대개 감당 가능
- 수백 km 이상: OLTP 응답 시간 증가
- FASTSYNC는 디스크 시간만 줄임
- 원거리 무손실: Far Sync(ADG 옵션, 10장)
- 대역폭 부족 시 ASYNC도 lag 누적

Standby 수와 구성

보호 모드는 구성 전체에 하나

- Protection은 SYNC AFFIRM Standby 2개 이상
- 흔한 구성: 근거리 SYNC + 원거리 ASYNC
- 모드는 하나, 전송 방식은 목적지별
- ASYNC 쪽이 끊겨도 보호 수준 유지

자동 Failover와의 관계

Fast-Start Failover와 보호 모드

- 교재: Availability나 Performance에서 사용
- 12.2부터 Protection도 가능, Standby 2개 이상
- Availability: 무손실 자동 Failover
- Performance: FastStartFailoverLagLimit 안에서만

업무 유형별 선택 예

출발점일 뿐, 측정으로 확정한다

- 그룹웨어·리포팅: Performance + ASYNC
- 온라인 주문·회원: Availability + SYNC/FASTSYNC
- 결제 원장: Availability + SYNC AFFIRM 이상
- 원거리 DR 전용: Performance + ASYNC(+압축)

2.8

운영 시 주의점

- 모드와 속성은 짝으로 변경
- MODE와 LEVEL 감시
- 대칭 구성과 Broker 일원화

운영 시 주의점

조용히 무너지는 보호

- 올릴 때 속성 먼저, 내릴 때 모드 먼저
- PROTECTION_MODE와 LEVEL 일치 여부 감시
- Protection 구성의 Standby 작업은 모드 하향부터
- Broker 구성에서는 ALTER SYSTEM으로 목적지 수정 금지
- 압축·Far Sync는 라이선스 확인 후 설정

요약

- 목적지마다 LOG_ARCHIVE_DEST_n, SYNC는 NSS, ASYNC는 TTnn
- AFFIRM은 SRL 기록 후 확인, SYNC NOAFFIRM이 FASTSYNC
- 원격 목적지: SERVICE, DB_UNIQUE_NAME, VALID_FOR 필수
- Performance 기본, Availability 단일 장애 무손실, Protection 이중 장애 무손실
- Protection은 동기화된 Availability에서만 올림(ORA-01126)
- MODE는 설정값, LEVEL은 실제 수준

퀴즈 (1/2)

- Q1. SYNC AFFIRM + Performance에서 Standby 장애 시? → NET_TIMEOUT 후 계속 진행
- Q2. FASTSYNC의 실체는? → SYNC NOAFFIRM
- Q3. Standby의 PRIMARY_ROLE 목적지는? → 오류 없이 무시

퀴즈 (2/2)

- Q4. MODE는 AVAILABILITY, LEVEL은 RESYNC면? → 무손실 보장 상실
- Q5. Performance에서 Protection 직행 시 ORA-01126 대응? → Availability 경유
- Q6. Protection 위험 완화와 거리가 먼 것? → 원격 목적지 MANDATORY

실습 개요

- 01 보호 모드 확인 — MODE와 LEVEL, 모드별 요건 (2.5, 2.6)
- 02 LOG_ARCHIVE_DEST 속성 해부 — 속성과 뷰 열 대응 (2.4)
- 03 보호 모드 전환 — Protection 경로, ORA-01126 (2.6)
- 04 모드별 Primary 동작 — Standby 상실 시 대비 (2.5)
- 05 MODE와 LEVEL 어긋남 — RESYNCHRONIZATION 판별 (2.5, 2.8)

Oracle Database 19c Active Data Guard 과정

Data Guard를 위한 Oracle Net 구성

이름, 리스너, 인증, 그리고 양방향 연결

제3장

학습 목표

- 데이터베이스가 클라이언트가 되는 구조를 설명한다
- 별칭, 서비스 이름, DB_UNIQUE_NAME을 구분한다
- 동적 등록과 정적 등록, _DGMGRL 서비스를 구성한다
- 패스워드 파일과 REDO_TRANSPORT_USER로 인증을 맞춘다
- DG_CONFIG의 역할과 네트워크 성능 조정값을 정한다
- 계층별 순서로 연결을 검증하고 오류를 해석한다

Agenda

- 3.2 Data Guard에서 Oracle Net이 쓰이는 곳
- 3.3 이름 해석과 tnsnames.ora
- 3.4 리스너와 서비스 등록
- 3.5 리두 전송 인증 / 3.6 LOG_ARCHIVE_CONFIG와 DG_CONFIG
- 3.7 네트워크 성능 조정
- 3.8 연결 검증과 오류 진단 / 3.9 운영 시 주의점

3.2

Data Guard에서 Oracle Net이 쓰이는 곳

- 데이터베이스가 클라이언트가 되는 구조
- Oracle Net을 쓰는 여섯 경로
- 양방향 대칭 구성

데이터베이스가 클라이언트가 된다

전송 프로세스는 Standby 리스너의 손님

- 리스너가 요청을 받아 전용 서버 프로세스 생성
- 로컬 sqlplus / as sysdba는 리스너를 거치지 않음
- Primary의 NSS, TT, ARCn이 클라이언트 역할
- Standby에서는 그 접속을 RFS가 받는다

Oracle Net을 쓰는 경로

모두 같은 설정 파일을 공유한다

- 리두 전송: Primary에서 Standby로
- FAL 갭 해소: Standby에서 FAL 서버로
- RMAN DUPLICATE: NOMOUNT auxiliary 접속
- 역할 전환 뒤 반대 방향 전송, Broker DMON 통신
- 애플리케이션의 현재 Primary 접속

양방향 연결과 대칭 구성

평상시에는 보이지 않는 반대 방향

- FAL은 Standby가 Primary에 접속하는 연결
- Switchover 뒤에는 boston이 chicago로 전송
- Broker DMON은 구성원끼리 서로 접속
- 별칭, 정적 등록, 방화벽 허용을 양쪽에 똑같이

구성 파일과 위치

인스턴스가 실제로 읽는 파일은 어디인가

- tnsnames.ora: 별칭과 접속 기술자
- sqlnet.ora: 이름 해석 순서, 타임아웃, TCP 옵션
- listener.ora: 수신 주소와 정적 등록
- TNS_ADMIN을 먼저, 없으면 ORACLE_HOME/network/admin
- 백그라운드 프로세스는 기동 시점 환경을 물려받음

3.3

이름 해석과 tnsnames.ora

- 접속 기술자의 두 요소
- 별칭, 서비스 이름, DB_UNIQUE_NAME
- 역할 전환에 대비한 클라이언트 접속

접속 기술자의 두 요소

어디로 가서, 누구를 찾는가

- ADDRESS: 프로토콜, 호스트, 포트
- CONNECT_DATA: SERVICE_NAME 또는 SID
- 주소가 틀리면 리스너에 닿지 못함
- 서비스가 틀리면 리스너가 거절함

이름 해석 방법

이 과정은 로컬 이름 지정을 쓴다

- 로컬(tnsnames.ora), 디렉터리(LDAP)
- Easy Connect, 외부 이름 서비스
- 순서는 sqlnet.ora의 NAMES.DIRECTORY_PATH
- Broker는 별칭 대신 완전한 기술자를 쓰기도 함
- 호스트에 IP를 쓰면 DNS 조회를 건너뜀

Data Guard용 tnsnames.ora

모든 구성원 별칭을 모든 서버에

- CHICAGO, BOSTON 별칭을 양쪽에 같은 내용으로
- 자기 자신을 가리키는 별칭도 둔다
- SERVICE_NAME은 DB_UNIQUE_NAME 기반으로
- 호스트 이름 표기는 한 가지로 통일

세 가지 이름의 층

같은 boston이라도 층이 다르다

- 넷 서비스 이름: tnsnames.ora의 별칭
- 서비스 이름: 인스턴스가 리스너에 등록
- DB_UNIQUE_NAME: 구성원 고유 이름
- DEST의 SERVICE는 별칭, DB_UNIQUE_NAME은 실제 이름

역할 전환에 대비한 접속

접속 시 페일오버와 대기 상한

- ADDRESS_LIST에 두 호스트를 함께
- 앞 리스너가 응답 없으면 다음 주소로
- FAILOVER=on은 기본값이라 생략 가능
- SQLNET.OUTBOUND_CONNECT_TIMEOUT으로 빨리 포기
- Primary에서만 뜨는 서비스 이름을 쓴다

3.4

리스너와 서비스 등록

- 동적 등록과 그 한계
- 정적 등록 SID_LIST와 GLOBAL_DBNAME
- Broker의 _DGMGRL 서비스

listener.ora의 구조

기본값만으로도 리스너는 뜬다

- 수신 주소, 서비스 정보, 동작 파라미터
- 기본 이름 LISTENER는 명령에서 생략 가능
- 리스너를 나누면 포트도 달라야 함
- DG 때문에 바꾸는 이유: 정적 등록과 성능 조정

동적 등록

LREG가 리스너에게 알려 준다

- 서비스 이름, 인스턴스 이름, 상태를 등록
- 기본 서비스 이름은 DB_UNIQUE_NAME.DB_DOMAIN
- 1521이 아니면 LOCAL_LISTENER 지정
- 즉시 반영은 ALTER SYSTEM REGISTER

동적 등록만으로 부족한 순간

인스턴스가 없거나 NOMOUNT일 때

- 인스턴스가 내려가면 동적 서비스 정보 소멸
- 그 서비스로 접속하면 ORA-12514
- NOMOUNT, MOUNT는 BLOCKED로 등록
- BLOCKED 서비스 접속은 ORA-12528
- DUPLICATE와 Broker 재기동이 이 순간을 만남

정적 등록 SID_LIST

파일에 적힌 선언을 리스너가 믿는다

- SID_LIST_ 뒤에 리스너 이름을 붙인다
- SID_DESC 하나가 정적 서비스 하나
- GLOBAL_DBNAME은 SERVICE_NAME과 글자 그대로 일치
- SID_NAME, ORACLE_HOME은 실제 인스턴스 환경
- 변경 후 lsnrctl reload

Broker의 _DGMGRL 서비스

인스턴스를 다시 올릴 때 쓰는 문

- 이름 규칙: db_unique_name_DGMGRL.db_domain
- StaticConnectIdentifier 속성이 이 서비스를 가리킴
- 없으면 Switchover 마지막 재기동에서 실패
- _DGB 서비스는 DMON용 동적 등록
- Clusterware 관리 DB에는 불필요

lsnrctl status로 상태 읽기

READY, BLOCKED, UNKNOWN

- READY: 동적 등록, 접속 가능
- BLOCKED: 동적 등록, NOMOUNT·MOUNT라 거부
- UNKNOWN: 정적 등록, 리스너는 상태를 모름
- 같은 서비스에 정적·동적 항목이 함께 보일 수 있음
- Listener Parameter File 줄로 실제 파일 경로 확인

정적 등록은 양쪽에 같은 모양으로

역할이 바뀌면 chicago도 NOMOUNT가 된다

- 두 서버에 db_unique_name 서비스와 _DGMGRL 서비스
- GLOBAL_DBNAME, SERVICE_NAME, DB_DOMAIN을 한 표로
- 셋 중 하나를 바꾸면 나머지 둘도 확인

3.5

리두 전송 인증

- 패스워드 파일 기반 인증
- REDO_TRANSPORT_USER
- 인증 실패의 증상과 회복

전송 세션도 로그온한다

패스워드 파일로 서로를 확인

- 인증 방식: SSL 또는 원격 로그인 패스워드 파일
- 대부분 패스워드 파일 방식을 사용
- 인스턴스가 자기 파일에서 자격 정보를 읽음
- FAL, DMON, DUPLICATE도 같은 파일에 기댐

패스워드 파일 요건

이름, 내용, 파라미터

- REMOTE_LOGIN_PASSWORDFILE=EXCLUSIVE
- 파일 이름은 orapw + 각 서버의 ORACLE_SID
- Primary 파일을 복사해 Standby 이름으로
- orapwd 재생성은 리두로 전달되지 않음
- 로그온 버전 설정도 양쪽 sqlnet.ora에서 일치

REDO_TRANSPORT_USER

SYS 대신 전송 전용 사용자

- 기본값은 SYS로 인증
- 지정한 사용자는 SYSOPER 권한 필요
- 모든 구성원에 같은 값과 같은 사용자
- 패스워드 파일 일치 조건은 그대로

인증 실패의 증상

ORA-01017에서 ORA-16191로

- 원격 SYSDBA 접속: ORA-01017
- 리두 전송 목적지: ORA-16191
- ORA-16191은 로그온 실패라는 결과만 알려 줌
- 원인 제거 후에도 목적지는 실패 상태로 남음
- DEFER 후 ENABLE로 즉시 재시도

실무 시나리오: 비밀번호 변경 다음 날

transport lag가 계속 늘어난다

- Primary에서 orapwd로 SYS 비밀번호 변경
- 업무 영향은 없고 다음 날 lag 증가 발견
- 목적지 오류 ORA-16191, 원격 접속 ORA-01017
- 파일 복사, DEFER·ENABLE로 전송 재개

3.6

LOG_ARCHIVE_CONFIG와 DG_CONFIG

- 구성원 목록의 역할
- SEND, RECEIVE 옵션
- FAL 파라미터와의 관계

DG_CONFIG 구성원 목록

이 구성에 누가 속하는가

- 모든 구성원의 DB_UNIQUE_NAME을 나열
- 목적지 DB_UNIQUE_NAME 검사, 없으면 ORA-16053
- 목록 밖에서 온 리두는 수신 거부
- 모든 구성원에 같은 목록

SEND, RECEIVE 옵션

기본값을 거의 바꾸지 않는다

- 기본값은 SEND, RECEIVE
- NOSEND: 리두를 보내지 않음
- NORECEIVE: 리두를 받지 않음
- 잘못 남기면 역할 전환 뒤 전송이 시작되지 않음

FAL_SERVER와 FAL_CLIENT

갭 해소도 Oracle Net 접속이다

- FAL_SERVER: 빠진 로그를 요청할 대상 별칭
- Standby에는 Primary, Primary에는 Standby 별칭
- FAL_CLIENT는 11.2부터 필수 아님
- Standby 쪽 tnsnames.ora와 Primary 리스너가 필요

3.7

네트워크 성능 조정

- SDU
- TCP 소켓 버퍼와 BDP
- 커널 한도와 TCP.NODELAY

SDU

리두를 몇 조각으로 나눠 보내는가

- Oracle Net 버퍼 단위, 기본 8192바이트
- 리두는 훨씬 큰 단위로 흐름
- 권고값 64KB(65535), 12.1부터 상한 2MB
- 양쪽 협상값 중 작은 값이 적용됨

TCP 소켓 버퍼와 BDP

선로를 채울 만큼의 버퍼

- BDP = 대역폭(바이트/초) × 왕복 지연(초)
- 권고값: BDP×3과 10MB 중 큰 값
- SEND_BUF_SIZE, RECV_BUF_SIZE
- 양쪽 tnsnames.ora와 listener.ora에 함께

운영체제 커널 한도

OS가 허용하지 않으면 조용히 줄어든다

- Oracle Net 지정값은 OS 최대값을 넘지 못함
- 줄어들어도 오류는 나지 않음
- rmem_max, wmem_max, tcp_rmem, tcp_wmem
- preinstall 패키지 값도 권고보다 작음

TCP.NODELAY

작은 패킷을 기다리지 않고 보낸다

- Nagle 알고리즘은 작은 패킷을 모아서 송신
- sqlnet.ora에 TCP.NODELAY=YES
- Primary와 Standby 양쪽에 설정
- SYNC 전송에서는 커밋 시간에 직접 영향

조정 전후를 같은 조건에서

숫자로 효과를 확인한다

- 대역폭과 왕복 지연을 먼저 측정해 BDP 계산
- 같은 부하에서 transport lag와 전송 속도 기록
- SDU, 버퍼, 커널, NODELAY를 양쪽에 한 번에
- 교재 listener.ora 예의 SEND_SDU는 SEND_BUF_SIZE로 읽음

3.8

연결 검증과 오류 진단

- 계층별 검증 순서
- 오류 코드 지도
- Data Guard 쪽 확인

계층별 검증 순서

아래층부터 올라간다

- 1 tnsping: 이름 해석과 리스너 응답
- 2 lsnrctl status: 서비스 이름과 상태
- 3 sqlplus sys@별칭 as sysdba: 인스턴스와 인증
- 4 DG_CONFIG, 5 V$ARCHIVE_DEST_STATUS
- 모든 단계를 양방향으로

오류 코드 지도 (1)

이름, 네트워크, 리스너

- ORA-12154, TNS-03505: 별칭을 풀지 못함
- ORA-12170, TNS-12535: 응답 없이 시간 초과
- ORA-12541: 리스너 없음, 즉시 거부
- ORA-12514: 서비스를 모름
- ORA-12528: 인스턴스가 막음(BLOCKED)

오류 코드 지도 (2)

인스턴스, 인증, Data Guard

- ORA-01034: 인스턴스가 떠 있지 않음
- ORA-01017: 인증 실패, 패스워드 파일 확인
- ORA-16191: 전송 클라이언트 로그온 실패
- ORA-16053: DB_UNIQUE_NAME이 DG_CONFIG에 없음

Data Guard 쪽에서 하는 확인

목적지 상태와 Broker 검증 명령

- V$ARCHIVE_DEST_STATUS: STATUS, ERROR
- ERROR는 마지막 실패 기록, 고친 뒤 재조회
- VALIDATE NETWORK CONFIGURATION FOR ALL
- VALIDATE STATIC CONNECT IDENTIFIER FOR ALL

구축 전 네트워크 점검표

DUPLICATE 전에 모두 통과

- 양쪽 tnsnames.ora, 모든 별칭 tnsping 성공
- 양쪽 정적 서비스 2개, UNKNOWN 표시
- 양방향 원격 SYSDBA 접속, NOMOUNT에서도
- 리스너 포트 양방향 허용

3.9

운영 시 주의점

- 양쪽을 함께 바꾼다
- 비밀번호와 IP 변경은 Data Guard 변경
- 원인 제거 뒤 상태 회복까지 확인

운영 시 주의점

평소에는 드러나지 않는 설정들

- 네트워크 설정은 양쪽을 함께 수정
- IP, 호스트 이름, DNS 변경은 DG 변경으로 취급
- 비밀번호 변경 절차에 파일 동기화 포함
- 방화벽은 끄지 말고 포트를 양방향 허용
- 조정값과 BDP 근거를 구축 문서에

요약

- 전송 프로세스는 Standby 리스너의 클라이언트, 받는 쪽은 RFS
- 별칭, 서비스 이름, DB_UNIQUE_NAME은 서로 다른 층
- NOMOUNT 접속과 Broker 재기동에는 정적 등록과 _DGMGRL
- 패스워드 파일 일치, REDO_TRANSPORT_USER는 SYSOPER
- DG_CONFIG는 구성원 목록, 오기는 ORA-16053
- SDU 64KB, 버퍼 max(BDP×3, 10MB), NODELAY, 양방향 검증

퀴즈 (1/2)

- Q1. ADDRESS_LIST의 접속 시 페일오버는? → 기본으로 켜짐
- Q2. NOMOUNT auxiliary 접속에서 ORA-12528이면? → 정적 등록 추가
- Q3. Broker 재기동용 정적 서비스 이름은? → db_unique_name_DGMGRL

퀴즈 (2/2)

- Q4. REDO_TRANSPORT_USER는 Primary에만? → 아니다, 모든 구성원
- Q5. 목적지 DB_UNIQUE_NAME 오기를 막는 설정은? → DG_CONFIG
- Q6. 1Gbit/s, 30ms 구간 소켓 버퍼는? → 약 11.25MB(BDP×3)

실습 개요

- 01 tnsnames 구성과 검증: tnsping의 범위 (3.3, 3.8)
- 02 listener 정적 등록: READY·BLOCKED·UNKNOWN (3.4)
- 03 정적 등록 제거와 복원: ORA-12514, 12528 (3.4)
- 04 DG_CONFIG 설정과 누락: ORA-16053 (3.6)
- 05 패스워드 파일 불일치: 01017, 16191 (3.5)
- 06 포트 차단과 해소: 오류 계층 지도 (3.8)

Oracle Database 19c Active Data Guard 과정

Primary 준비

ARCHIVELOG, FORCE LOGGING, Standby Redo Log, 파라미터

제4장

학습 목표

- Primary 준비 항목과 빠졌을 때의 증상을 연결한다
- ARCHIVELOG 전환의 선행 조건과 확인 방법을 안다
- NOLOGGING의 위험과 FORCE LOGGING 우선순위를 구분한다
- 19c STANDBY NOLOGGING 두 모드와 라이선스를 설명한다
- SRL 개수·크기를 산정하고 Primary에 미리 만든다
- FRA와 Data Guard 파라미터의 역할을 설명한다

Agenda

- 4.2 Primary 준비의 전체 그림
- 4.3 ARCHIVELOG 모드
- 4.4 FORCE LOGGING과 NOLOGGING 작업
- 4.5 19c의 STANDBY NOLOGGING 모드
- 4.6 Standby Redo Log / 4.7 FRA와 아카이브 보관
- 4.8 Primary의 Data Guard 파라미터 / 4.9 점검과 주의점

4.2

Primary 준비의 전체 그림

- Standby 생성 여섯 단계 중 첫 단계
- 준비 항목과 빠졌을 때의 증상
- 역할 전환까지 내다본 준비

Standby 생성 절차와 Primary 준비

DUPLICATE는 그 시점의 Primary를 복제한다

- 1 Primary 준비 → 2 Standby 파라미터 → 3 Net
- 4 NOMOUNT 기동 → 5 DUPLICATE → 6 전송·적용
- Primary에 SRL이 있으면 Standby에도 자동 생성
- FORCE LOGGING은 복제 시작 전에 켠다
- 재기동이 필요한 작업은 ARCHIVELOG 전환뿐

준비 항목과 빠졌을 때의 증상

무엇이 빠지면 어디서 드러나는가

- ARCHIVELOG 없음: 전송 서비스가 성립하지 않음
- FORCE LOGGING 없음: Standby에 무효 블록
- SRL 없음: 실시간 적용, SYNC, 캐스케이딩 불가
- FRA 부족: 아카이브 중단, Primary 정지
- DG 파라미터 오류: 리두 송수신 거부

역할 전환까지 내다본 준비

Primary도 언젠가 Standby가 된다

- SRL, FAL_SERVER, STANDBY_FILE_MANAGEMENT
- 셋 다 Primary 역할에서는 쓰이지 않음
- VALID_FOR로 역할별 목적지를 미리 구분
- Broker를 쓰려면 spfile이 필요

4.3

ARCHIVELOG 모드

- 리두는 Standby의 유일한 재료
- MOUNT 상태에서의 전환
- 상태 확인과 아카이브 로그 이름

리두가 Standby의 유일한 재료

덮어쓰인 리두는 다시 구할 수 없다

- 최초 복제 이후 변경은 리두로만 전달
- NOARCHIVELOG면 한 바퀴 돈 리두는 사라짐
- 갭 해소(FAL)도 아카이브 로그에서 읽음
- ACTIVE DUPLICATE도 아카이브 로그로 따라잡음
- 로그 모드는 CDB 단위로 하나

전환 절차와 선행 조건

MOUNT에서만, 깨끗한 종료 뒤에만

- SHUTDOWN IMMEDIATE → STARTUP MOUNT
- ALTER DATABASE ARCHIVELOG → OPEN
- OPEN 상태에서 실행하면 ORA-01126
- ABORT 직후면 ORA-00265, 정상 종료 후 재시도
- 같은 정지 시간에 FORCE LOGGING도 처리

상태 확인과 아카이브 로그 이름

모드와 실제 아카이브는 따로 본다

- ARCHIVE LOG LIST: 모드, 대상, 현재 순번
- 스크립트 점검은 V$DATABASE.LOG_MODE
- 로그 전환 후 V$ARCHIVED_LOG로 실제 생성 확인
- LOG_ARCHIVE_FORMAT에는 %t, %s, %r 필수
- FRA에 쓰면 OMF 이름이 붙어 FORMAT 무시

4.4

FORCE LOGGING과 NOLOGGING 작업

- NOLOGGING 작업이 Standby에 남기는 구멍
- FORCE LOGGING의 우선순위
- 켜는 시점과 이미 생긴 손상 처리

NOLOGGING 작업의 범위

버퍼 캐시를 거치지 않는 대량 작업

- APPEND 힌트 INSERT, SQL*Loader direct
- CTAS, 인덱스 생성·재구성, MOVE
- 일반 INSERT/UPDATE/DELETE는 해당 없음
- 딕셔너리 변경 리두는 남고 데이터 블록만 빠짐

Standby에 남는 흔적

적용할 때는 조용하고, 읽을 때 터진다

- 리두 없는 블록은 복구 불가 상태로 표시
- 읽는 순간 ORA-01578 + ORA-26040
- Primary: V$DATAFILE.UNRECOVERABLE_TIME
- Standby: V$NONLOGGED_BLOCK (12.2 이후)

실무 시나리오: 야간 적재 다음 날

Primary는 멀쩡했다

- SQL*Loader direct + UNRECOVERABLE 옵션
- Primary는 FORCE LOGGING 없이 운영
- 다음 날 ADG 보고서 조회가 ORA-01578로 실패
- 복구: NONLOGGED BLOCK 복구 또는 파일 재수신
- 재발 방지: FORCE LOGGING (스크립트 옵션은 무시됨)

FORCE LOGGING의 동작과 우선순위

데이터베이스 설정이 모든 하위 설정을 이긴다

- 객체·테이블스페이스 NOLOGGING, 문장 지정 모두 무시
- 임시 테이블스페이스·임시 세그먼트는 예외
- 수준: 데이터베이스, PDB, 테이블스페이스
- 기본값은 NOFORCE LOGGING
- 물리·논리 Standby 모두에 필요

켜는 시점과 확인

복제 전에 켜고, 구성이 있는 동안 유지

- OPEN에서도 가능, 진행 중 NOLOGGING 작업 종료까지 대기
- 공식 교재 권장은 MOUNT 상태
- V$DATABASE.FORCE_LOGGING = YES
- DBA_TABLESPACES.FORCE_LOGGING도 함께 확인

이미 생긴 무효 블록의 처리

FORCE LOGGING은 과거를 고치지 않는다

- 켜기 전 작업의 무효 블록은 그대로 남음
- 범위가 작으면 RECOVER DATABASE NONLOGGED BLOCK
- 범위가 넓으면 RECOVER DATAFILE ... FROM SERVICE
- Redo Apply를 멈춘 상태에서 수행

4.5

19c의 STANDBY NOLOGGING 모드

- FORCE LOGGING의 비용
- DATA AVAILABILITY와 LOAD PERFORMANCE
- 라이선스와 점검 스크립트

FORCE LOGGING의 비용

적재 데이터만큼 리두가 늘어난다

- direct-path 적재 블록 전체가 리두로
- LGWR 쓰기, 아카이브, 전송량 모두 증가
- SYNC 구성이면 커밋 대기도 길어짐
- 비용 때문에 끄면 다시 구멍이 생김

FOR DATA AVAILABILITY

커밋을 Standby 적용까지 미룬다

- 리두 대신 별도 연결로 적재 블록을 직접 전송
- 모든 Standby가 적용해야 커밋 완료
- 리두는 적고, Standby 데이터는 완전
- Standby·네트워크가 느리면 적재도 느려짐

FOR LOAD PERFORMANCE

적재 속도를 지키고 나중에 채운다

- 기본 동작은 DATA AVAILABILITY와 같음
- 네트워크가 못 따라오면 전송을 멈추고 적재 계속
- 빠진 블록은 Standby가 managed recovery 중 자동 수신
- 보충 전 Failover 시 해당 블록은 무효

네 가지 로깅 상태 비교

리두 양, Standby 데이터, 적재 속도

- NOFORCE: 리두 적음, 무효 블록, 빠름
- FORCE LOGGING: 리두 많음, 완전, 느림
- DATA AVAILABILITY: 리두 적음, 커밋 시 완전, 중간
- LOAD PERFORMANCE: 리두 적음, 일시 누락, 빠름
- 서로 배타적, 대상은 direct-path 적재

FORCE_LOGGING 열은 YES/NO만이 아니다

점검 스크립트 조건을 다시 본다

- 모드 설정 시 열에 모드 이름이 그대로 표시
- force_logging = 'NO' 조건은 이 모드를 놓침
- 정상 기준 'YES', 그 밖의 값을 모두 보고
- FOR PERFORMANCE처럼 LOAD를 빼면 ORA-02231

라이선스와 적용 조건

Active Data Guard 환경이 전제

- 두 모드는 Active Data Guard 환경 기준으로 설명됨
- ADG는 EE 위의 별도 구매 옵션
- Standby를 마운트로만 운영하면 FORCE LOGGING만
- 적재 후 원복 여부를 절차서에 명시

4.6

Standby Redo Log

- SRL을 거치는 리두의 흐름
- 개수와 크기 규칙
- Primary에 먼저 만드는 이유

SRL을 거치는 리두의 흐름

RFS가 쓰고, ARCn이 아카이브하고, MRP가 읽는다

- 받은 리두는 RFS가 SRL에 기록
- 온라인 로그와 비슷한 주기로 차고 전환
- 다 찬 SRL은 Standby ARCn이 아카이브
- 실시간 적용이면 MRP가 쓰는 중인 SRL을 읽음
- STANDBY_ARCHIVE_DEST는 폐기됨

SRL이 없거나 맞지 않을 때

오류 없이 아카이브 로그로 직접 쓴다

- SRL이 하나도 없음
- 들어오는 로그보다 크거나 같은 SRL이 없음
- 크기 맞는 SRL이 모두 아카이브 대기 중
- 전송은 계속되지만 로그 단위 적용으로 후퇴

SRL이 필요한 기능

없으면 동작하지 않는 것들

- SYNC 전송 모드
- 실시간 적용 (19c 기본 적용 방식)
- 캐스케이딩 목적지
- Far Sync 인스턴스의 리두 중계
- 결과적으로 Max Availability·Protection 모드

개수 산정

스레드마다 온라인 그룹 수 + 1

- 단일 인스턴스: 온라인 3그룹 → SRL 4그룹
- 2노드 RAC, 스레드당 3그룹 → (3+1)×2 = 8
- 여분 한 그룹은 아카이브 대기를 흡수
- 부족하면 피크 시간대에 실시간 적용이 끊김

크기 규칙

가장 큰 온라인 로그 이상, 실무에서는 전부 같은 크기

- 리두 소스의 최대 온라인 로그보다 작으면 안 됨
- 관리 편의를 위해 온라인 로그와 SRL을 동일 크기로
- 더 크게 잡아도 이점 없음
- 온라인 로그 변경 시 SRL도 함께 변경
- 멤버 다중화는 필수 아님

Primary에도 SRL을 만드는 이유

쓰이지 않는 파일을 미리 두는 까닭

- 역할 전환 직후 곧 리두를 받을 수 있음
- DUPLICATE가 Standby에 같은 SRL을 자동 생성
- Far Sync 생성 때도 동일하게 복제
- 구성 전체가 같은 SRL 규격을 가짐

생성과 확인

ADD STANDBY LOGFILE THREAD n

- 그룹 번호는 온라인 로그와 같은 번호 공간
- 단일 인스턴스라도 THREAD 절 명시
- V$LOGFILE TYPE='STANDBY', V$STANDBY_LOG
- Primary에서는 모두 UNASSIGNED가 정상
- 파일 충돌은 ORA-00301 아래 ORA-27038

4.7

FRA와 아카이브 보관

- FRA가 Data Guard에서 맡는 일
- 크기와 위치 지정
- 삭제 정책과 공간 부족

FRA가 Data Guard에서 맡는 일

아카이브 로그와 플래시백 로그의 집

- 아카이브 로그, 플래시백 로그, 백업 보관
- reinstate와 Snapshot Standby가 플래시백에 기댐
- 공간이 부족하면 삭제 정책에 따라 자동 정리
- Standby에도 자체 FRA 필요

크기와 위치 지정

크기 먼저, 위치 다음

- DB_RECOVERY_FILE_DEST_SIZE 먼저 지정
- 위치부터 지정하면 ORA-19802
- 둘 다 동적 파라미터, 크기는 상한일 뿐
- 하루 리두량 × 보관 일수 + 플래시백·백업 몫
- V$RECOVERY_FILE_DEST, V$RECOVERY_AREA_USAGE

삭제 정책과 공간 부족

너무 빨리 지우면 갭, 너무 늦게 지우면 정지

- FRA가 차면 아카이브 중단, Primary DML 정지
- 받지 못한 로그를 지우면 FAL로도 못 메움
- SHIPPED TO ALL STANDBY / APPLIED ON ALL STANDBY
- 사례: Standby 사흘 정지 중 Primary FRA 포화

4.8

Primary의 Data Guard 파라미터

- 이름: DB_UNIQUE_NAME, LOG_ARCHIVE_CONFIG
- 목적지: LOG_ARCHIVE_DEST_n, VALID_FOR
- 역할 전환 대비와 인증

DB_NAME과 DB_UNIQUE_NAME

이름은 같고, 고유 이름은 다르다

- DB_NAME과 DBID는 구성 전체가 같음
- 구성원 구분은 DB_UNIQUE_NAME
- Primary는 기본값(DB_NAME과 동일) 사용 가능
- 예: DB_NAME chicago, 고유 이름 chicago·boston
- 정적 파라미터, 처음 정하면 바꾸지 않음

LOG_ARCHIVE_CONFIG

구성원 명단

- DG_CONFIG=(chicago,boston)
- 목록에 없는 이름과는 송수신하지 않음
- 미래의 Standby·Far Sync도 미리 등록 가능
- V$DATAGUARD_CONFIG로 확인
- Broker 구성에서는 Broker가 관리

LOG_ARCHIVE_DEST_n과 STATE_n

어디로 보내는가

- n은 1~31, 보통 1은 로컬, 2는 Standby
- LOCATION 또는 SERVICE 중 하나는 필수
- 원격 목적지에는 DB_UNIQUE_NAME 속성
- STATE_n: ENABLE(기본) 또는 DEFER
- 로컬은 USE_DB_RECOVERY_FILE_DEST로 명시

VALID_FOR 조합

로그 종류 × 데이터베이스 역할

- 예: (ONLINE_LOGFILES, PRIMARY_ROLE)
- 로컬은 (ALL_LOGFILES, ALL_ROLES)가 일반적
- 역할에 맞지 않으면 오류 없이 무시
- (STANDBY_LOGFILE, PRIMARY_ROLE)만 ORA-16026
- 전송 속성 상세는 2장

역할 전환에 대비하는 파라미터

Standby가 되었을 때 쓰일 값

- FAL_SERVER: 갭을 메울 로그를 요청할 대상
- FAL_CLIENT: 생략 가능, 명시하는 구성이 많음
- STANDBY_FILE_MANAGEMENT=AUTO (상세는 8장)
- DB_/LOG_FILE_NAME_CONVERT: 경로가 다를 때 (5장)
- ARCHIVE_LAG_TARGET, LOG_ARCHIVE_MAX_PROCESSES

비밀번호 파일과 spfile

전송 연결의 인증

- 모든 구성원에 비밀번호 파일, SYS 비밀번호 동일
- REMOTE_LOGIN_PASSWORDFILE=EXCLUSIVE
- Standby 파일은 Primary 파일을 복사해 이름 변경
- REDO_TRANSPORT_USER로 전송 전용 사용자 가능
- spfile은 Broker의 전제

Primary 파라미터 묶음

chicago 기준, Standby는 거울상

- DB_UNIQUE_NAME=chicago, DG_CONFIG=(chicago,boston)
- DEST_1 로컬 FRA, DEST_2 SERVICE=boston ASYNC
- FAL_SERVER=boston, STANDBY_FILE_MANAGEMENT=AUTO
- 바꾼 뒤에는 V$ARCHIVE_DEST의 STATUS·ERROR 확인

4.9

준비 상태 점검과 운영 시 주의점

- 생성 전 점검 조회
- 운영 중 지켜야 할 것

생성 전 점검 조회

DUPLICATE 직전에 한 번 더

- LOG_MODE = ARCHIVELOG, FORCE_LOGGING = YES
- SRL 개수 ≥ 스레드별 온라인 그룹 + 1
- SRL 최소 크기 ≥ 온라인 로그 최대 크기
- FRA 지정과 여유 공간
- dest_1 VALID, dest_2는 생성 후 VALID

운영 시 주의점

조용히 무너지는 준비 항목

- FORCE LOGGING은 구성이 있는 동안 끄지 않음
- 온라인 로그 변경과 SRL 변경은 한 묶음
- FRA 경보는 Primary와 Standby 모두에
- SYS 비밀번호 변경은 계획된 작업으로
- 고유 이름·서비스 이름 표기를 한 문서로 관리

요약

- Primary 준비는 DUPLICATE 전에 끝낸다 (SRL·FORCE LOGGING)
- ARCHIVELOG는 MOUNT에서 전환, 리두 보존의 전제
- FORCE LOGGING이 모든 NOLOGGING 설정보다 우선
- STANDBY NOLOGGING 두 모드는 ADG 환경 전제
- SRL: 스레드별 온라인 그룹 + 1, 같은 크기, Primary에 먼저
- FRA·삭제 정책, DB_UNIQUE_NAME·DG_CONFIG·DEST_n·FAL

퀴즈 (1/2)

- Q1. OPEN 상태에서 ARCHIVELOG 전환? → ORA-01126
- Q2. FORCE LOGGING + NOLOGGING 테이블 APPEND 적재? → 리두 생성
- Q3. LOAD PERFORMANCE 모드의 특징? → 전송 생략 후 Standby가 보충

퀴즈 (2/2)

- Q4. 2노드 RAC, 스레드당 4그룹의 SRL 수? → 10
- Q5. 온라인 로그만 키우고 SRL을 두면? → 아카이브 직접 기록, 실시간 적용 중단
- Q6. Primary에 FAL_SERVER를 두는 이유? → 역할 전환 대비

실습 개요

- 01 DBCA로 CDB 생성 — 옵션의 Data Guard 의미 (4.2, 4.7)
- 02 ARCHIVELOG 확인과 전환 — ORA-01126, FRA (4.3)
- 03 FORCE LOGGING 실증 — 리두 양 비교 (4.4)
- 04 SRL 산정과 생성 / 05 SRL 없이 실시간 적용 (4.6)
- 06 19c STANDBY NOLOGGING — 모드와 열 값 (4.5)

Oracle Database 19c Active Data Guard 과정

Physical Standby 생성 ① RMAN DUPLICATE

경로 지정 방식으로 Standby를 만드는 전체 과정

제5장

학습 목표

- Standby가 Primary와 공유하는 것과 달라야 하는 것을 구분한다
- Standby 생성 방법들을 비교하고 상황에 맞게 고른다
- 패스워드 파일, 최소 pfile, NOMOUNT 기동의 이유를 설명한다
- DUPLICATE FOR STANDBY의 각 절과 옵션을 해석한다
- convert 파라미터로 Standby 경로를 설계한다
- 실패 원인을 분류하고 생성 후 확인 순서를 따른다

Agenda

- 5.2 Standby 생성이라는 작업의 성격 / 5.3 Standby를 만드는 방법들
- 5.4 생성 전체 흐름 / 5.5 Standby 인스턴스 준비
- 5.6 DUPLICATE ... FOR STANDBY 명령 구성
- 5.7 경로 지정: DB_FILE_NAME_CONVERT와 LOG_FILE_NAME_CONVERT
- 5.8 내부 단계 / 5.9 흔한 실패 원인 분류
- 5.10 생성 후 확인과 Redo Apply 연결 / 5.11 운영 시 주의점

5.2

Standby 생성이라는 작업의 성격

- Primary의 블록 사본으로서의 Standby
- BR 과정의 DUPLICATE와 다른 점
- 만들어야 하는 재료

물리 Standby는 Primary의 블록 사본

같아야 하는 것과 달라야 하는 것

- DBID, DB_NAME: Primary와 같음
- DB_UNIQUE_NAME: 반드시 다름 (chicago / boston)
- 컨트롤 파일: STANDBY 종류
- ORACLE_SID: Standby 고유 (orapw, spfile 이름과 연결)

BR 과정의 DUPLICATE와 무엇이 다른가

같은 명령의 변형, 다른 목적

- 일반 DUPLICATE: 새 DBID, RESETLOGS로 OPEN
- FOR STANDBY: 같은 DBID, MOUNT 상태로 멈춤
- 일반 복제는 원본과 관계가 끊김
- Standby는 복제 이후 리두로 계속 연결

만들어야 하는 재료

방법이 달라도 결과물은 같다

- 데이터 파일 사본 (퍼지 상태, 리두로 맞춤)
- Standby 컨트롤 파일
- Standby용 SPFILE
- Primary와 같은 패스워드 파일
- Standby Redo Log

5.3

Standby를 만드는 방법들

- Active DUPLICATE와 백업 기반 DUPLICATE
- 수동 복원 방식
- DBCA, Enterprise Manager
- 선택 기준

RMAN DUPLICATE FROM ACTIVE DATABASE

이 과정의 기준 방법

- 운영 중인 Primary 파일을 네트워크로 직접 복사
- 미리 받아 둔 백업이 필요 없음
- target과 auxiliary에 동시 접속
- 복사 동안 Primary I/O와 네트워크 사용

백업 기반 DUPLICATE

Primary 부하를 생성 작업에서 떼어 냄

- FROM ACTIVE DATABASE 없이 기존 백업에서 복원
- 복원은 Standby 호스트에서 일어남
- 백업 파일에 Standby가 접근할 수 있어야 함
- 대용량, 느린 WAN, 정기 백업 재활용

수동 복원 방식

DUPLICATE가 자동화한 일을 손으로

- CREATE STANDBY CONTROLFILE로 컨트롤 파일 생성
- CREATE PFILE FROM SPFILE 후 값 수정
- 백업을 Standby로 옮겨 복원
- ASM이면 데이터 파일 추출에 RMAN 필요

DBCA와 Enterprise Manager

더 높은 자동화, 더 좁은 적용 범위

- dbca -createDuplicateDB -createAsStandby
- 교재 기준 비 CDB, 단일 인스턴스 Primary 대상
- DBCA는 Standby만 만들고 Primary 설정은 안 함
- EM 마법사는 SRL, Broker 구성까지 처리

방법 선택 기준

데이터 출처, Primary 부하, 자동화 정도

- 일반적인 신규 구축: Active DUPLICATE
- 대용량, 좁은 대역: 백업 기반 DUPLICATE
- 비 CDB를 빠르게: DBCA
- EM 운영 조직: Add Standby Database 마법사

5.4

생성 전체 흐름

- 시작 전에 갖춰져 있어야 하는 것
- 위치별로 할 일

시작 전 확인 목록

3장과 4장에서 만든 것

- ARCHIVELOG, FORCE LOGGING, Primary SRL
- LOG_ARCHIVE_CONFIG, LOG_ARCHIVE_DEST_2
- REMOTE_LOGIN_PASSWORDFILE=EXCLUSIVE
- 양쪽 tnsnames.ora, Standby 리스너 정적 등록

위치별로 할 일

Primary, Standby, RMAN 실행 위치

- Primary: 패스워드 파일 복사, 전송 확인
- Standby: 디렉터리, 최소 pfile, NOMOUNT 기동
- RMAN: 양쪽 접속 확인 후 DUPLICATE 실행
- RMAN은 어느 호스트에서 실행해도 됨

5.5

Standby 인스턴스 준비

- 패스워드 파일 복사
- 최소 초기 파라미터 파일과 디렉터리
- NOMOUNT 기동과 원격 접속

패스워드 파일 복사

orapwd로 새로 만들지 않는다

- Primary의 orapw 파일을 orapwboston으로 복사
- 같은 비밀번호로 새로 만든 파일은 피함
- auxiliary 접속을 위해 DUPLICATE 전에 필요
- DUPLICATE가 실행 중 Primary 파일로 다시 덮어씀

최소 초기 파라미터 파일

DB_NAME 한 줄이면 충분

- NOMOUNT 기동에 필요한 최소 값만 기록
- 메모리 값이 없으면 기본 SGA로 기동
- SPFILE 절이 Primary 값으로 모두 대체
- CDB 복제 시 enable_pluggable_database=true

디렉터리 준비

convert는 이름만 바꾸고 디렉터리는 만들지 않음

- 데이터 파일: 루트, PDB 시드, 각 PDB 하위까지
- 컨트롤 파일, 온라인 로그 위치
- AUDIT_FILE_DEST, FRA
- 없으면 파일 생성 단계에서 실패

NOMOUNT 기동과 원격 접속

정적 등록이 필요한 이유

- ORACLE_SID=boston 지정 후 STARTUP NOMOUNT
- NOMOUNT 인스턴스는 리스너에서 BLOCKED
- 서비스 접속 시 ORA-12528
- 정적 등록은 DUPLICATE 중 재기동에도 필요

5.6

DUPLICATE … FOR STANDBY 명령 구성

- 접속과 채널, 파일 전송 방식
- SPFILE 절, PARAMETER_VALUE_CONVERT, SET
- DORECOVER, NOFILENAMECHECK
- 병렬 채널, SECTION SIZE, 압축

명령의 전체 형태

한 명령에 Standby의 모양이 담김

- RUN 블록: target 채널과 auxiliary 채널
- FOR STANDBY FROM ACTIVE DATABASE
- DORECOVER, SPFILE 절과 SET 절
- NOFILENAMECHECK로 마무리

접속과 채널

target은 Primary, auxiliary는 Standby

- 양쪽 모두 Oracle Net 서비스 이름으로 접속
- 리커버리 카탈로그는 필수가 아님
- ALLOCATE CHANNEL은 target 쪽 프로세스
- auxiliary 채널은 최소 하나 필요

파일 전송 방식: push와 pull

12c 이후 Active DUPLICATE의 두 형태

- push: target 채널이 이미지 사본을 밀어 보냄
- pull: auxiliary 채널이 백업 셋을 끌어옴
- USING BACKUPSET, SECTION SIZE 지정 시 pull
- auxiliary 채널 수가 target 이상이어도 pull

SPFILE 절과 PARAMETER_VALUE_CONVERT

일괄 치환과 그 부작용

- Primary SPFILE을 가져와 Standby용으로 생성
- 문자열 값의 chicago를 boston으로 일괄 치환
- FAL_SERVER='boston'은 그대로 남음
- 대소문자 구분, convert 파라미터는 SET으로 명시

SET 절에 넣는 파라미터

PARAMETER_VALUE_CONVERT 다음에 적용

- DB_UNIQUE_NAME, CONTROL_FILES
- DB_FILE_NAME_CONVERT, LOG_FILE_NAME_CONVERT
- FAL_SERVER, LOG_ARCHIVE_DEST_2
- STANDBY_FILE_MANAGEMENT=AUTO

DORECOVER와 NOFILENAMECHECK

복구 시점과 이름 검사

- 온라인 복사본은 파일마다 시점이 다름
- DORECOVER: 복사 중 생긴 리두까지 적용
- 생략하면 Redo Apply가 이후에 맞춤
- NOFILENAMECHECK: 이름 중복 검사 생략

병렬 채널, SECTION SIZE, 압축

생성 시간과 부하의 조절

- 채널 수: 동시에 복사하는 파일 수
- SECTION SIZE: 큰 파일을 구간으로 나눠 병렬
- USING COMPRESSED BACKUPSET: 대역 절약, CPU 사용
- BASIC 외 압축 알고리즘은 ACO 라이선스

5.7

경로 지정: DB_FILE_NAME_CONVERT와 LOG_FILE_NAME_CONVERT

- 문자열 치환 규칙
- 두 파라미터가 쓰이는 두 시점
- 컨트롤·임시·로그 파일의 처리, ASM과 OMF

문자열 치환 규칙

Primary 문자열, Standby 문자열의 쌍

- 파일 이름의 부분 문자열을 찾아 바꿈
- 쌍은 여러 개, 먼저 일치하는 쌍이 적용
- 대소문자 구분, 끝의 / 까지 맞추기
- 교재는 값 길이 제한(255자)을 언급

두 파라미터가 쓰이는 두 시점

생성 때와 운영 중

- 생성: DUPLICATE가 Standby 파일 이름 결정
- 운영: STANDBY_FILE_MANAGEMENT=AUTO와 함께
- Primary에 추가된 파일의 Standby 이름 결정
- 역할 전환 대비 Primary에도 반대 방향 값

컨트롤·임시·로그 파일의 처리

convert가 닿는 곳과 닿지 않는 곳

- 컨트롤 파일: CONTROL_FILES로 직접 지정
- 임시 파일: DB_FILE_NAME_CONVERT를 따름
- 온라인 로그: Redo Apply 시작 시 새로 생성
- SRL: Primary에 있으면 DUPLICATE가 생성

ASM과 OMF의 경우

6장으로 이어지는 대조

- 디스크 그룹 이름도 치환 가능 (+DATA → +SBDAT)
- OMF Standby는 convert 파라미터를 설정하지 않음
- 파일 이름을 오라클이 생성
- 경로 지정 방식과의 비교는 6장

5.8

DUPLICATE 내부에서 일어나는 단계

- 메모리 스크립트로 나뉜 단계들
- 로그에서 단계를 찾는 단서

내부 단계

메모리 스크립트가 차례로 실행됨

- SPFILE 생성 후 auxiliary 재기동
- 스냅샷에서 Standby 컨트롤 파일 생성, 마운트
- set newname 후 데이터 파일 복사
- 복구, 파일 전환, SRL 생성, 완료

로그 읽기와 오류 스택

아래에서 위로 읽는다

- Memory Script 내용으로 실패 단계 확인
- 채널 여러 개면 input, output 순서가 섞임
- 맨 위 RMAN-03002, 05501은 결과
- 원인은 가장 아래쪽 ORA 오류

5.9

흔한 실패 원인 분류

- 단계별 실패 분류
- 원격 오류의 방향, 파일 이름 충돌
- 실패 후 다시 시작하기

실패 원인 분류

오류 번호보다 단계로 먼저 나눈다

- 접속: ORA-01017, ORA-12528
- 파일 이름: RMAN-05001, RMAN-05538 경고
- 디렉터리: ORA-17628 아래의 ORA-19505
- 공간, 파라미터 오타, 작업 중 인스턴스 종료

원격 오류와 이름 충돌

어디를 볼지 오류가 알려 준다

- ORA-17628: 원격 서버(대개 Standby)의 오류
- ORA-19505: 파일 생성·열기 실패
- RMAN-05001 일부 파일: convert 쌍 누락
- RMAN-05001 전체 파일: 규칙 미적용, 대소문자

실패 후 다시 시작하기

DUPLICATE는 뒷정리를 하지 않는다

- 반쯤 복사된 파일과 SPFILE이 남음
- 인스턴스를 내리고 남은 파일 삭제
- spfileboston.ora 삭제 후 PFILE로 NOMOUNT
- Primary는 읽기만 하므로 재시도에 부담 없음

5.10

생성 후 확인과 Redo Apply 연결

- 식별 정보와 역할
- 파일, 로그, 파라미터 구성
- 전송 시작과 Redo Apply 기동
- DBMS_DBCOMP 블록 비교

식별 정보와 역할

V$DATABASE부터 본다

- DBID, NAME: Primary와 같음
- DB_UNIQUE_NAME=boston, PHYSICAL STANDBY
- CONTROLFILE_TYPE=STANDBY, MOUNTED
- ORA-32002: SPFILE이 이미 사용 중, 조치 불필요

파일, 로그, 파라미터 구성

Primary 기준값과 대조

- V$DATAFILE: 개수, 번호, 바뀐 경로
- V$STANDBY_LOG: SRL 개수와 크기
- V$PDBS: PDB 목록
- FAL_SERVER, LOG_ARCHIVE_DEST_n 값 재확인

전송 시작과 Redo Apply 기동

7장으로 넘어가는 지점

- Primary: LOG_ARCHIVE_DEST_STATE_2=ENABLE
- 로그 전환 후 V$ARCHIVE_DEST_STATUS 확인
- Standby: RECOVER MANAGED STANDBY DATABASE DISCONNECT
- READ ONLY WITH APPLY는 ADG 라이선스 필요

DBMS_DBCOMP로 블록 비교

19c의 블록 단위 검증

- Primary와 Standby의 같은 블록을 직접 비교
- DB_LOST_WRITE_PROTECT 설정과 무관
- 두 번째 인자는 출력 파일 접두어
- Primary에서 실행하면 모든 Standby와 비교

5.11

운영 시 주의점

- 재생성 대비, 경로 변경, 보안

운영 시 주의점

만든 다음에도 남는 일

- DUPLICATE 스크립트와 convert 규칙 보관
- Primary 경로가 늘면 Standby 값과 디렉터리도 갱신
- 생성 중 생긴 아카이브 로그 보존
- 스크립트에 SYS 비밀번호를 적지 않음

요약

- Standby: 같은 DBID·DB_NAME, 다른 DB_UNIQUE_NAME, STANDBY 컨트롤 파일
- 방법: Active, 백업 기반, 수동, DBCA, EM 중 상황에 맞게 선택
- 준비: 패스워드 파일 복사, 최소 pfile, 디렉터리, NOMOUNT, 정적 등록
- SPFILE 절: PARAMETER_VALUE_CONVERT 후 SET으로 보정
- convert는 대소문자 구분 문자열 치환, 디렉터리는 따로 생성
- 실패는 단계로 분류, 확인 후 Redo Apply 기동

퀴즈 (1/2)

- Q1. FOR STANDBY 복제본의 DBID는? → Primary와 같고 MOUNT로 끝남
- Q2. NOMOUNT 인스턴스 접속 시 ORA-12528이면? → 정적 등록 확인
- Q3. SET 없이 FAL_SERVER='boston'은? → 그대로 남아 자기 자신을 가리킴

퀴즈 (2/2)

- Q4. Chicago 경로에 'chicago' 규칙이면? → 미적용, RMAN-05001
- Q5. ORA-17628과 ORA-19505는 어디부터? → Standby alert 로그와 디렉터리
- Q6. 생성 후 CREATE SPFILE에 ORA-32002면? → 이미 SPFILE 사용 중, 조치 불필요

실습 개요

- 01 Primary 파라미터 확인 / 02 Standby 초기 pfile (5.4, 5.5)
- 03 NOMOUNT 기동과 접속 확인 (5.5)
- 04 DUPLICATE 실행과 로그 읽기 (5.6, 5.8)
- 05 실패 재현과 진단: RMAN-05001, ORA-19505 (5.9)
- 06 결과 검증과 ADG 전환 (5.10)
- 07 DBMS_DBCOMP 블록 비교 (5.10)

Oracle Database 19c Active Data Guard 과정

Physical Standby 생성 ② OMF 방식

파일 이름은 데이터베이스가, 위치는 관리자가

제6장

학습 목표

- OMF가 관리하는 파일과 수명 관리 방식을 설명한다
- OMF 파라미터 조합별 파일 배치를 판단한다
- OMF 디렉터리 계층과 이름 형식을 읽는다
- OMF Standby에 convert가 필요 없는 이유를 설명한다
- SPFILE 절의 CONTROL_FILES 상속을 예방·교정한다
- ASM, AUTO와의 관계를 알고 두 방식 중 하나를 고른다

Agenda

- 6.2 OMF가 해결하는 문제 / 6.3 OMF 파라미터와 배치 규칙
- 6.4 OMF 파일 이름 규칙
- 6.5 convert 파라미터가 필요 없는 이유
- 6.6 OMF 방식 DUPLICATE / 6.7 컨트롤 파일과 CONTROL_FILES
- 6.8 ASM / 6.9 STANDBY_FILE_MANAGEMENT=AUTO
- 6.10 두 방식의 비교와 선택 / 6.11 운영 시 주의점

6.2

OMF가 해결하는 문제

- 경로 지정 방식에서 사람이 맡던 일
- OMF의 정의와 대상 파일
- 파일 수명 관리

경로 지정 방식에서 관리자가 맡던 일

Standby 이름은 Primary 이름의 문자열 치환

- convert 규칙이 Primary 디렉터리를 모두 덮어야 함
- 치환 결과 디렉터리는 Standby에 미리 생성
- CONTROL_FILES 별도 지정, 필요하면 NOFILENAMECHECK
- Primary에 새 경로가 생기면 규칙도 다시 손봄

OMF의 정의와 대상 파일

위치만 주면 이름은 데이터베이스가 짓는다

- 데이터 파일, 임시 파일, 온라인 로그, SRL
- 컨트롤 파일은 CONTROL_FILES가 비어 있을 때만
- FRA의 아카이브 로그, 백업, Flashback 로그
- 패스워드 파일, 경보 로그, 트레이스는 대상 아님
- 파라미터가 비어 있으면 이름 생략 시 ORA-02236

OMF 파일의 수명 관리

만든 파일은 지우는 것까지 책임진다

- DROP TABLESPACE 시 OMF 데이터 파일도 삭제
- 일반 파일은 INCLUDING ... DATAFILES 없으면 남음
- DROP LOGFILE GROUP도 OMF 멤버 파일 삭제
- OMF 파일과 일반 파일의 공존은 정상
- MOVE DATAFILE을 TO 없이 쓰면 OMF로 이동

DBCA로 만든 데이터베이스가 OMF인가

최상위 경로 지정과 OMF는 다른 설정

- datafileDestination만으로 OMF가 되지 않음
- 기본 템플릿은 system01.dbf 같은 이름으로 생성
- OMF로 만들려면 useOMF true 또는 생성 후 설정
- 판단은 V$DATAFILE 이름의 o1_mf_ 접두어로

6.3

OMF 파라미터와 파일 배치 규칙

- DB_CREATE_FILE_DEST
- DB_CREATE_ONLINE_LOG_DEST_n
- DB_RECOVERY_FILE_DEST와 조합별 배치

DB_CREATE_FILE_DEST

데이터 파일과 임시 파일의 기본 위치

- 값은 디렉터리 경로 또는 +디스크그룹
- 최상위 디렉터리는 미리 있어야 하고 쓰기 가능해야
- 하위 디렉터리는 데이터베이스가 필요할 때 생성
- 동적 파라미터, ALTER SESSION으로 예외 배치

DB_CREATE_ONLINE_LOG_DEST_n

온라인 로그와 컨트롤 파일 전용 위치

- n은 1부터 5까지, 지정한 수만큼 다중화
- 하나라도 있으면 로그는 이 위치에만 생성
- 로그와 데이터를 다른 디스크에 둘 때 사용
- SRL도 같은 규칙, SIZE만 주면 멤버 생성

DB_RECOVERY_FILE_DEST와 FRA

로그 다중화 위치로도 쓰인다

- DB_RECOVERY_FILE_DEST_SIZE와 짝으로 설정
- FRA의 아카이브 로그와 백업은 항상 OMF 이름
- FILE_DEST와 FRA가 모두 있으면 로그가 두 벌
- 지시하지 않아도 생기는 다중화, FRA 크기 재계산

조합별 배치 결과

어느 파라미터에 값이 있는지가 결과를 정한다

- FILE_DEST만: 로그와 컨트롤 파일 1개씩
- FILE_DEST + FRA: 두 위치에 1개씩
- LOG_DEST_1, _2: 그 두 위치에 1개씩
- LOG_DEST_1 + FRA: LOG_DEST_1에만
- 컨트롤 파일 열은 CONTROL_FILES가 빈 경우만

잘못된 경로가 드러나는 시점

OMF는 기동 단계에서 경로를 검사한다

- 실행 중 변경: ORA-02097 + ORA-01261로 거부
- pfile의 없는 경로: NOMOUNT에서 ORA-01261, 01262
- 경로 지정 방식은 DUPLICATE 도중에야 드러남
- 값이 아예 없으면 기동은 정상, 문제는 나중에

6.4

OMF 파일 이름 규칙

- 디렉터리 계층
- DB_UNIQUE_NAME과 PDB GUID
- 파일 이름 형식

디렉터리 계층

기본위치 / DB_UNIQUE_NAME / 종류 / 파일

- 두 번째 단계는 DB_UNIQUE_NAME의 대문자
- 종류: datafile, onlinelog, controlfile
- FRA: archivelog, backupset 아래 날짜 디렉터리
- PDB 파일은 GUID 디렉터리 아래 datafile

DB_UNIQUE_NAME과 GUID를 쓰는 이유

같은 DB_NAME, 다른 디렉터리

- Primary와 Standby는 DB_NAME이 같음
- DB_UNIQUE_NAME만 달리 주면 디렉터리가 갈림
- Boston과 BOSTON은 리눅스에서 다른 디렉터리
- PDB GUID는 Primary와 Standby가 동일

파일 이름 형식

o1_mf_ 접두어와 8자리 고유 문자열

- 데이터 파일: o1_mf_테이블스페이스_고유_.dbf
- 임시 파일은 .tmp, 로그와 SRL은 그룹 번호 .log
- 컨트롤 파일은 고유 문자열만, .ctl
- 파일 번호는 이름에 없어 V$DATAFILE로 확인
- 로그와 SRL 구분은 V$LOGFILE.TYPE으로

OMF 이름을 직접 다룰 때

OMF 이름은 데이터베이스만 붙인다

- OMF DB에서도 이름을 적으면 일반 파일로 생성
- o1_mf_ 형식 이름을 사람이 쓰면 ORA-01276
- 일반 파일의 OMF 전환: MOVE DATAFILE, TO NEW
- 운영체제에서 OMF 파일 이름을 바꾸지 않음

6.5

convert 파라미터가 필요 없는 이유

- 변환하는 방식과 새로 짓는 방식
- NOFILENAMECHECK
- 함께 주었을 때의 우선순위

이름을 변환하는 방식과 새로 짓는 방식

입력이 다르면 설정도 달라진다

- convert: Primary 이름을 받아 문자열 치환
- OMF: Standby 파라미터로 새 이름을 생성
- Primary 경로가 몇 갈래든 Standby 설정은 한 줄
- 치환 쌍 누락으로 인한 실패가 구조적으로 사라짐

NOFILENAMECHECK가 필요 없는 이유

이름 검사가 그대로 통과한다

- DUPLICATE는 원본과 같은 이름이면 RMAN-05001
- 같은 구조를 쓰는 경로 지정 방식은 검사를 꺼야 함
- OMF 이름은 원본과 같아질 수 없음
- 검사를 켜 둔 상태가 더 안전

Primary가 OMF가 아니어도 된다

Standby 이름은 Standby 설정으로 정해진다

- 일반 파일 Primary에서 OMF Standby 생성 가능
- 반대 조합은 OMF 경로를 바꾸는 convert 규칙 필요
- 새 Standby 구축은 파일 관리를 정리할 기회
- 혼합 구성은 역할 전환 뒤를 함께 고려

OMF와 convert를 함께 주면

RMAN 이름 결정 우선순위

- SET NEWNAME
- CONFIGURE AUXNAME
- DB_FILE_NAME_CONVERT
- DB_CREATE_FILE_DEST (가장 낮음)
- convert가 남아 있으면 OMF 이름이 붙지 않음

같은 서버에 두 번째 Standby 추가

실무 시나리오

- 최상위 경로는 같게, DB_UNIQUE_NAME만 다르게
- 파일은 BOSTON, BOSTON2로 자동 분리
- LOG_ARCHIVE_CONFIG 목록에 두 Standby 모두
- 리스너 정적 등록과 서비스 이름은 각각 필요

6.6

OMF 방식 DUPLICATE의 흐름과 차이

- Standby 서버 준비
- SPFILE 절에서 달라지는 것
- 실패 양상과 복제 후 점검

Standby 서버 준비

미리 만들 디렉터리가 최상위 둘로 줄어든다

- OMF 위치와 FRA 최상위 경로만 준비
- BOSTON, datafile, PDB GUID는 복제 중 생성
- adump처럼 OMF 밖의 디렉터리는 여전히 수동
- 초기 pfile에 db_create_file_dest 한 줄 추가

SPFILE 절에서 달라지는 것

빠지는 줄과 들어가는 줄

- 빠짐: PARAMETER_VALUE_CONVERT, convert 2종
- 빠짐: NOFILENAMECHECK
- 추가: DB_CREATE_FILE_DEST, FRA 위치와 크기
- 유지 권장: SET CONTROL_FILES
- 적지 않은 값은 Primary spfile 값이 그대로

실패 양상의 차이

오류가 나는 경우와 조용히 지나가는 경우

- 없는 OMF 경로: NOMOUNT 실패, RMAN은 ORA-01034
- OMF도 convert도 없음: RMAN-05538 후 05001
- convert 잔존: 오류 없이 일반 이름으로 생성
- CONTROL_FILES 미지정: 오류 없이 Primary 경로

복제 직후 파일 배치 점검

Best Practice

- 데이터, 임시, 로그, 컨트롤 파일 이름을 한 번에 조회
- 모두 기본위치/BOSTON 아래인지 확인
- convert 값이 비어 있는지 확인
- V$LOG.MEMBERS로 다중화 확인
- Standby 서버에 Primary 이름 디렉터리가 생겼는지

6.7

컨트롤 파일 위치와 CONTROL_FILES

- OMF가 컨트롤 파일 이름을 정하는 조건
- SPFILE 절 상속 문제와 예방
- 옮기는 순서

OMF가 컨트롤 파일 이름을 정하는 조건

CONTROL_FILES가 비어 있을 때뿐

- 값이 있으면 DB_CREATE_FILE_DEST보다 우선
- OMF로 만들면 이름이 spfile에 자동 기록
- 정적 파라미터, SCOPE=SPFILE과 재기동 필요

SPFILE 절의 상속이 만드는 문제

데이터 파일은 BOSTON, 컨트롤 파일만 Chicago

- SPFILE 절이 Primary의 CONTROL_FILES를 복사
- SET CONTROL_FILES가 없으면 그 경로에 생성
- 오류 없이 복제 성공으로 끝남
- 잔재로 오인해 지우면 Standby가 마운트 불가

처음부터 피하는 방법

SPFILE 절에 Standby 경로를 직접 적는다

- SET CONTROL_FILES로 BOSTON 계층 경로 지정
- 다중화가 필요하면 FRA 쪽 경로 추가
- 지정한 디렉터리는 미리 생성
- PARAMETER_VALUE_CONVERT는 영향 범위 확인 후

이미 만들어진 뒤 옮기는 순서

파라미터 먼저, 파일은 내린 상태에서

- ALTER SYSTEM SET control_files ... SCOPE=SPFILE
- 적용 중지 후 SHUTDOWN IMMEDIATE
- 운영체제에서 파일 이동(mv), 사본을 남기지 않음
- STARTUP MOUNT 후 V$CONTROLFILE 확인
- 하나라도 없으면 ORA-00205, STARTED에 머묾

복제 후 spfile을 다시 만들지 않는다

Note

- 일부 문서의 CREATE SPFILE FROM PFILE 단계
- spfile로 기동 중이면 ORA-32002로 거부
- 성공한다면 DUPLICATE가 SET한 값이 모두 사라짐
- SPFILE 절이 Standby spfile까지 만들어 둠

6.8

ASM 환경과 OMF

- ASM에서는 OMF가 기본
- ASM Standby를 만드는 방법

ASM에서는 OMF가 기본 동작

+디스크그룹/DB_UNIQUE_NAME/종류/태그.번호.인카네이션

- DB_CREATE_FILE_DEST='+DATA', FRA='+FRA'
- 예: +DATA/BOSTON/DATAFILE/SYSTEM.257.1149876543
- PDB 파일은 GUID 디렉터리 아래, 계층은 동일
- 읽기 쉬운 이름이 필요하면 별칭(alias)

ASM Standby를 만드는 방법

RMAN이 필수, convert보다 OMF 설정

- ASM 파일은 운영체제 명령으로 복사 불가
- Standby의 DB_CREATE_FILE_DEST에 자기 디스크 그룹
- 디스크 그룹 이름이 달라도 convert 불필요
- Primary ASM, Standby 파일 시스템 조합도 가능
- Grid 홈과 DB 홈 환경 변수 구분, oraenv 사용

6.9

STANDBY_FILE_MANAGEMENT=AUTO와 OMF

- Primary 파일 추가의 자동 반영
- OMF Standby의 기본 파라미터 묶음

AUTO와 OMF의 궁합

Primary의 파일 추가가 Standby에서 OMF 이름으로

- AUTO: Primary 파일 추가·삭제를 Standby가 따라 함
- 이름은 Standby 파라미터가 결정
- OMF → convert → 같은 경로 순으로 적용
- 경로 지정 방식은 규칙 밖 경로에서 적용 실패
- 상세 동작과 복구는 8장

OMF Standby의 기본 파라미터 묶음

Best Practice

- DB_CREATE_FILE_DEST, FRA 위치와 크기
- STANDBY_FILE_MANAGEMENT=AUTO
- convert 두 파라미터는 비워 둠
- 같은 설정을 Primary에도
- 양쪽 값을 같은 조회로 나란히 비교

6.10

두 방식의 비교와 선택

- 항목별 비교
- 선택 기준
- 혼합 구성과 역할 전환

항목별 비교

결과물은 같고 이름을 정하는 주체가 다르다

- 이름 결정: convert 치환 대 Standby에서 생성
- 사전 준비: 모든 하위 디렉터리 대 최상위만
- 가독성: system01.dbf 대 o1_mf_ 규칙 이름
- 오류 시점: DUPLICATE 도중 대 NOMOUNT 기동
- 구조 변경 반영: 규칙 의존 대 자동

선택 기준

새 구축은 OMF부터 검토

- 신규 Standby: OMF가 준비와 유지 부담이 작음
- ASM 저장소: 이미 OMF, 디스크 그룹만 맞춤
- 파일별 정밀 배치나 명명 규칙이 있으면 경로 지정
- 이름 패턴에 기댄 스크립트는 전환 전 점검

혼합 구성과 역할 전환

Switchover 뒤에는 방향이 바뀐다

- 평소: 일반 Primary, OMF Standby로 문제없음
- 전환 후: boston이 OMF 파일 생성, chicago가 수신
- chicago에 OMF 위치가 없으면 convert 필요
- chicago에도 DB_CREATE_FILE_DEST를 설정

비OMF Primary 옆에 OMF Standby

실무 시나리오

- Primary: 여섯 마운트 지점에 일반 파일
- 재해 복구 서버: 큰 볼륨 하나
- Standby DB_CREATE_FILE_DEST 한 줄로 해결
- 계획서: CONTROL_FILES, FRA 크기, Primary OMF 설정

6.11

운영 시 주의점

- 권한, 이동, FRA, 정리, 점검

운영 시 주의점

파일 이름을 사람이 정하지 않는 대가

- OMF 경로 소유자와 권한을 양쪽 같은 기준으로
- OMF 파일은 MOVE DATAFILE이나 RMAN으로만 이동
- FRA 경보 기준을 로그 다중화 기준으로 재설정
- Primary 이름 디렉터리는 V$ 조회 후 정리
- 운영 문서는 파일 이름 대신 번호와 테이블스페이스로

요약

- OMF: 위치만 주면 이름·생성·삭제는 데이터베이스가
- FILE_DEST + FRA면 로그와 컨트롤 파일 두 벌
- 경로: 기본위치/DB_UNIQUE_NAME/(GUID)/종류/o1_mf_
- convert는 치환, OMF는 생성, 함께 두면 convert 우선
- SPFILE 절은 CONTROL_FILES를 상속, 직접 지정
- ASM은 OMF가 기본, AUTO와 함께 Primary에도 설정

퀴즈 (1/2)

- Q1. FILE_DEST와 FRA만 설정 시 로그 멤버 수는? → 2개
- Q2. Standby SYSTEM 파일의 디렉터리는? → BOSTON/datafile
- Q3. NOFILENAMECHECK가 필요 없는 이유는? → 이름이 겹칠 수 없음

퀴즈 (2/2)

- Q4. 컨트롤 파일만 Chicago에 생긴 원인은? → CONTROL_FILES 상속
- Q5. OMF와 convert를 함께 주면? → convert가 우선
- Q6. 혼합 구성의 Switchover 대비는? → chicago에도 OMF 위치

실습 개요

- 01 OMF 파라미터와 기준 상태 (6.3)
- 02 없는 디렉터리 / 03 파라미터 누락 실패 재현 (6.3.5, 6.6.4)
- 04 OMF 방식 DUPLICATE 실행 (6.6)
- 05 OMF 구조 해부: 계층, GUID, 이름, 다중화 (6.3.4, 6.4)
- 06 컨트롤 파일 교정 (6.7)
- 07 두 방식 비교와 ADG 검증 (6.7.4, 6.10)

Oracle Database 19c Active Data Guard 과정

Redo Apply 기동과 동기화 검증

무엇이 적용하는가, 언제 적용하는가, 얼마나 따라가는가

제7장

학습 목표

- 전송 서비스와 Redo Apply를 구분해 설명한다
- RFS, MRP0, 복구 슬레이브의 역할을 구분한다
- Redo Apply를 기동·중지하고 상태 오류를 해석한다
- 실시간 적용의 조건과 19c 기본 동작을 판별한다
- 갭 해소 경로(ARCn ping, FAL)와 DELAY를 설명한다
- transport lag, apply lag로 지연 구간을 좁힌다

Agenda

- 7.2 Redo Apply의 위치와 동작 원리
- 7.3 Redo Apply 기동과 중지
- 7.4 실시간 적용 (Real-Time Apply)
- 7.5 적용을 떠받치는 Standby 설정
- 7.6 아카이브 갭과 FAL / 7.7 지연 적용 (DELAY)
- 7.8 동기화 상태를 판단하는 지표 / 7.9 운영 시 주의점

7.2

Redo Apply의 위치와 동작 원리

- 전송과 적용은 별개의 서비스
- 미디어 복구를 끝없이 이어 가는 구조
- Standby 프로세스의 역할 분담

전송과 적용은 분리된 서비스

도착했다와 반영됐다는 다른 진술

- 전송: 리두를 SRL·아카이브 로그에 기록하면 끝
- 적용: 기록된 리두를 데이터 파일 블록에 반영
- 적용이 멈춰도 전송은 계속된다
- 데이터 손실 여부는 전송, 전환 속도는 적용이 좌우

미디어 복구로서의 Redo Apply

백업 복구와 원리가 같고, 끝나지 않을 뿐

- 물리 Standby는 미디어 복구 기술로 적용
- 목표 시점 없이 다음 리두를 기다리며 계속
- 블록 수준에서 Primary와 동일, 데이터 유형 제약 없음
- 논리 Standby의 SQL Apply와는 구조가 다름
- Snapshot Standby는 받기만 하고 적용하지 않음

Standby 쪽 프로세스

적용의 주체는 MRP0와 복구 슬레이브

- RFS: Primary의 전송 프로세스와 짝을 이뤄 수신
- ARCn: 다 찬 SRL을 아카이브 로그로 만듦
- MRP0: Redo Apply 전체를 조율하는 코디네이터
- PRnn: logmerger와 apply slave로 병렬 적용
- MRP0가 없으면 적용이 돌지 않는 것으로 판단

V$DATAGUARD_PROCESS로 보는 역할

12.2 이후 권장 뷰

- ROLE 열: managed recovery, recovery logmerger 등
- ACTION 열: 지금 하는 일(APPLYING_LOG 등)
- V$MANAGED_STANDBY는 19c에서 디프리케이트
- 새 점검 스크립트는 V$DATAGUARD_PROCESS 기준

적용 속도를 좌우하는 요소

Primary보다 적용 쪽에 부담이 몰리는 순간

- Standby I/O: 캐시에 없는 블록을 읽어야 함
- CPU 수: 복구 슬레이브 병렬도에 영향
- 리두의 성격: 인덱스 많은 대량 DML, LOB 변경
- Broker의 Apply Rate로 처리 속도 확인

적용 단계의 손상 검증

리두를 그대로 덮어쓰는 장치가 아니다

- RFS, ARCn, MRP, DBWn이 단계별로 정합성 검사
- 손상된 리두는 갭 해소 경로로 다시 받아 옴
- DB_LOST_WRITE_PROTECT: 양쪽 TYPICAL 이상 권장
- Primary lost write는 ORA-00752로 드러남

7.3

Redo Apply 기동과 중지

- 기동 명령과 선행 조건
- 상태에 맞지 않는 명령의 오류
- 열린 모드와 적용 상태의 조합

기동 명령과 선행 조건

RECOVER MANAGED STANDBY DATABASE

- Standby가 최소 MOUNT 상태여야 함
- 기본형: ... RECOVER MANAGED STANDBY DATABASE DISCONNECT
- SYSDBA 또는 SYSDG로 접속해 실행
- 처음 기동 후 Primary 로그 전환으로 흐름 확인
- Broker 구성에서는 DGMGRL로 상태 변경

DISCONNECT의 의미

누가 복구를 수행하는가

- DISCONNECT: MRP0가 백그라운드에서 적용
- 생략하면 명령 세션이 직접 복구, 프롬프트 반환 없음
- FROM SESSION은 지금도 허용되지만 불필요
- 사실상 운영에서는 항상 DISCONNECT 사용

중지와 상태 오류

두 오류 모두 데이터에는 영향 없음

- 중지: ... RECOVER MANAGED STANDBY DATABASE CANCEL
- ORA-01153: 이미 도는데 다시 기동
- ORA-16136: 이미 멈췄는데 CANCEL
- ORA-10456: 마운트 상태에서 적용 중 OPEN 시도
- 재기동 스크립트는 CANCEL 후 기동, 16136은 무시

열린 모드와 적용 상태 조합

라이선스 경계가 걸린 곳

- MOUNTED + 적용: 전통적인 Standby
- READ ONLY + 적용 중지: 조회 가능, 데이터 정지
- READ ONLY WITH APPLY: Active Data Guard 옵션 필요
- 열린 Standby를 마운트로: 재기동 후 STARTUP MOUNT

기동 직후 확인할 세 가지

켜졌다가 아니라 적용하고 있다를 본다

- 프로세스: managed recovery 역할의 MRP0
- 모드: RECOVERY_MODE가 기대한 값인지
- 진행: 로그 전환 후 적용 위치가 움직이는지
- 프로세스가 떠 있어도 진행이 멈춰 있을 수 있음

ACTION 값 읽기

MRP0가 지금 무엇을 하는가

- APPLYING_LOG: 적용 중
- WAIT_FOR_LOG: 다음 리두 대기, 오류 아님
- WAIT_FOR_GAP: 빠진 로그 때문에 진행 불가
- APPLIED 열은 로그 단위 표시, 초 단위 판단 불가

7.4

실시간 적용 (Real-Time Apply)

- 로그 전환을 기다리지 않는 적용
- SRL이 전제인 이유
- 19c 기본값과 판별 방법

아카이브 기반 적용 vs 실시간 적용

언제 리두를 읽는가의 차이

- 아카이브 기반: 로그가 아카이브된 뒤 읽기 시작
- 실시간: RFS가 SRL에 쓰는 즉시 MRP0가 읽음
- 밀리면 아카이브 로그로 따라잡고 다시 SRL로
- RECOVERY_MODE: MANAGED 대 MANAGED REAL TIME APPLY

SRL이 전제인 이유

쓰는 중인 파일을 동시에 읽는 구조

- RFS가 SRL에 쓰지 않으면 실시간 적용 불가
- SRL 없음, 크기 부족, 모두 아카이브 대기 시 강등
- 4장 규칙: ORL과 같은 크기, 그룹 하나 더
- 적용 중인 SRL 삭제 시 ORA-00261

소리 없는 강등

오류 없이 아카이브 기반으로 떨어진다

- 기동 명령은 성공, MRP0도 정상으로 보임
- 실제로는 로그 전환 때마다 계단식으로 적용
- Primary 로그를 키우고 SRL을 그대로 두면 발생
- 판별 수단은 RECOVERY_MODE와 apply lag

19c 기본값과 명령 절

12.1부터 실시간 적용이 기본

- 기본형 기동만으로 실시간 적용
- USING CURRENT LOGFILE: 디프리케이트, 효과 같음
- USING ARCHIVED LOGFILE: 실시간 적용 끄기
- PARALLEL n, NODELAY, FINISH도 기동 시점에 결정

실시간 적용 vs 실시간 조회

이름은 비슷, 기능과 라이선스는 다름

- 실시간 적용: MRP0가 SRL을 바로 읽는 적용 방식
- 실시간 조회: READ ONLY로 연 채 적용을 계속
- 실시간 적용은 기본 기능, 조회는 ADG 옵션
- 마운트 Standby도 실시간 적용은 돌아감

실시간 적용 판별

1차는 뷰, 2차는 기능 시험

- V$ARCHIVE_DEST_STATUS.RECOVERY_MODE 확인
- 값: MANAGED REAL TIME APPLY / MANAGED / IDLE
- 반드시 Standby 인스턴스에서 조회
- 기능 시험: 커밋 후 로그 전환 없이 Standby 조회
- 기능 시험은 ADG 라이선스 환경에서만

실시간 적용의 이점

운영상 손해는 거의 없다

- Switchover, Failover 시 남은 적용량 감소
- Failover 시간(RTO) 단축
- 실시간 조회 결과가 Primary와 초 단위로 근접
- 큰 로그 파일을 써도 적용 지연이 늘지 않음

7.5

적용을 떠받치는 Standby 설정

- Standby의 로컬 아카이브 목적지
- 목적지 STATUS 읽기
- FAL과 파일 관리 파라미터

로컬 아카이브 목적지와 VALID_FOR

다 찬 SRL은 어디로 가는가

- Standby ARCn이 SRL을 아카이브 로그로 만듦
- 위치는 Standby의 LOG_ARCHIVE_DEST_1 등이 결정
- VALID_FOR에 SRL(ALL_LOGFILES 등)이 포함돼야 함
- ONLINE_LOGFILES만 지정하면 WRONG VALID_TYPE
- STANDBY_ARCHIVE_DEST는 디프리케이트

목적지 STATUS 읽기

INACTIVE와 BAD PARAM을 혼동하지 말 것

- VALID: 정상 사용 중
- INACTIVE: 정의되지 않은 목적지, 정상
- BAD PARAM: 문자열을 해석하지 못함
- ERROR: 문법은 맞지만 기록·연결 실패
- DEFERRED: 수동 보류 상태

FAL과 파일 관리 파라미터

기동 성공 뒤에도 적용을 좌우하는 설정

- FAL_SERVER: 빠진 로그를 요청할 대상
- FAL_CLIENT: 19c에서는 선택, 설정 시 올바른 이름
- Primary에도 반대 방향으로 미리 설정
- STANDBY_FILE_MANAGEMENT=AUTO (상세는 8장)
- 파일 이름 변환·OMF 설정도 적용의 일부

사례 — 기동은 성공, 적용은 제자리

진단 순서 따라가기

- 증상: MRP0는 보이는데 적용 위치가 거의 고정
- 1단계: RECOVERY_MODE가 MANAGED인지 확인
- 2단계: Standby 로컬 목적지 STATUS 확인
- 원인: BAD PARAM으로 SRL 아카이브 불가
- 교정 후 재기동, FAL 설정도 함께 점검

7.6

아카이브 갭과 FAL

- 갭이 생기는 원인
- 자동 해소의 두 경로
- 수동 해소와 예방

갭이 생기는 원인

적용은 갭을 발견하고 기다리는 쪽

- 네트워크 단절로 일정 시간 전송 불가
- Standby 인스턴스 다운 중 로그 전환
- Standby 디스크 부족으로 기록 실패
- Primary 아카이브 로그가 전송 전 삭제
- 갭 해소 중에는 apply lag가 일시적으로 커짐

자동 해소의 두 경로

Primary가 먼저, 또는 Standby가 요청

- ARCn ping: Primary가 주기적으로 누락을 확인해 전송
- FAL: MRP0가 부재를 발견해 FAL_SERVER에 요청
- 요청 받은 쪽 ARCn이 Standby RFS로 전송
- ping은 재연결에, FAL은 손상·유실에 강함

갭 확인과 수동 해소

자동으로 안 될 때의 순서

- Standby에서 V$ARCHIVE_GAP 조회
- 몇 분 간격으로 다시 봐서 줄어드는지 확인
- 원본 복사 후 ALTER DATABASE REGISTER PHYSICAL LOGFILE
- 원본이 없으면 RECOVER STANDBY DATABASE FROM SERVICE
- 갭이 있으면 Switchover 불가

갭과 보호 모드

설정값과 실제 보장 수준은 다를 수 있다

- Maximum Availability: 단절 시 비동기화로 계속 운영
- 갭이 모두 해소돼야 동기화 상태로 복귀
- PROTECTION_MODE는 설정, PROTECTION_LEVEL은 실제
- Maximum Performance는 갭이 있어도 Primary 무반응

삭제 정책으로 갭을 예방

가장 어려운 갭은 스스로 만든 갭

- 원인: FRA 공간 압박, 백업 후 일괄 삭제
- APPLIED ON ALL STANDBY 정책 권장
- 최소한 SHIPPED TO ALL STANDBY
- Primary와 Standby 양쪽에 같은 정책

7.7

지연 적용 (DELAY)

- 일부러 늦게 적용하는 이유
- DELAY 속성과 실시간 적용의 관계
- 지연 Standby를 고르는 기준

왜 늦게 적용하는가

사용자 오류로부터 데이터를 지키는 시간 창

- Data Guard는 잘못된 DROP, DELETE도 그대로 복제
- 실시간 적용이면 몇 초 안에 Standby에도 반영
- 지연 구간 안에 발견하면 적용을 멈추고 데이터 회수
- 전송은 지연되지 않아 손실 관점 보호는 유지

DELAY 설정

Primary 목적지의 속성

- Primary의 LOG_ARCHIVE_DEST_n에 DELAY=분
- 값 없이 DELAY만 쓰면 기본 30분
- 리두가 Standby에 아카이브된 시점부터 계산
- Broker 구성에서는 DelayMins 속성

실시간 적용과의 관계

둘은 함께 성립하지 않는다

- 실시간 적용 중이면 DELAY가 무시됨(교재·문서)
- 지연을 쓰려면 USING ARCHIVED LOGFILE로 기동
- 기동 후 RECOVERY_MODE가 MANAGED인지 확인
- NODELAY: 이번 기동에서만 지연 무시

지연 Standby 선택 기준

역할 전환용과 오류 대비용을 나눈다

- 지연 Standby는 Failover 시간이 길어짐
- 지연은 보통 두 번째 Standby에 건다
- Switchover 대상이라면 전환 전 지연 제거
- Standby의 Flashback Database가 대안이자 보완

사례 — 오후에 실행된 잘못된 DELETE

지연 Standby로 데이터 회수

- 조건 없는 DELETE, 40분 뒤 발견
- 실시간 Standby boston에는 이미 반영
- 4시간 지연 Standby의 적용부터 CANCEL
- READ ONLY로 열어 행 조회 후 Primary로 복원
- 복구 후 USING ARCHIVED LOGFILE로 재기동

7.8

동기화 상태를 판단하는 지표

- V$DATAGUARD_STATS의 네 지표
- 지표의 신선도
- lag 조합으로 지연 구간 좁히기

V$DATAGUARD_STATS의 네 지표

Standby에서 조회

- transport lag: 아직 도착하지 않은 리두의 시간 환산
- apply lag: Standby 데이터가 얼마나 과거인가
- apply finish time: 받은 리두 적용 완료 예상 시간
- estimated startup time: 기동·오픈 예상 시간
- apply lag는 항상 transport lag 이상

지표의 신선도

DATUM_TIME과 TIME_COMPUTED

- DATUM_TIME: 계산에 쓴 데이터를 마지막으로 받은 시각
- TIME_COMPUTED: 지표를 계산한 시각
- 두 시각 차이는 30초 미만이어야 신뢰
- 연결이 끊기면 lag가 작은 값에 멈춰 정상처럼 보임

lag 조합으로 구간 좁히기

어느 쪽 lag가 큰가

- 둘 다 작음: 정상, 단 신선도 확인
- transport 작고 apply만 큼: 적용 쪽 문제
- 둘 다 큼: 전송 구간 문제
- 둘 다 증가 추세: 리두 생성량이 처리 능력 초과
- apply lag 큰데 WAIT_FOR_LOG면 공급 문제

지표가 없을 때의 보조 확인

순번 비교와 Broker

- Primary 현재 순번과 Standby 마지막 적용 순번 비교
- 실시간 적용 중이면 차이 0~1이 보통
- 순번 차이는 로그 개수일 뿐 시간이 아님
- Broker SHOW DATABASE의 Transport Lag, Apply Lag
- 임계값 경보: ApplyLagThreshold 등(14장)

기동 후 동기화 점검 순서

앞 단계가 이상하면 뒤 단계 값은 믿지 않는다

- 1. 역할과 열린 모드 (V$DATABASE)
- 2. 프로세스: MRP0, RFS와 ACTION
- 3. 목적지 STATUS와 RECOVERY_MODE
- 4. 갭: V$ARCHIVE_GAP
- 5. 지연 지표와 DATUM_TIME 신선도

7.9

운영 시 주의점

- 멈춘 적용은 경보 없이 방치되기 쉽다
- 방식 변경은 CANCEL 후 재기동
- 값, 신선도, 추세로 감시

운영 시 주의점

경보가 울리지 않는 문제들

- CANCEL한 적용의 재기동을 체크리스트에
- SQL 관리 구성은 재기동 절차에 적용 기동 포함
- Primary 로그 변경 시 SRL도 함께 재구성
- Standby 자원은 Primary보다 넉넉하게
- 적용 중지 기간에는 Primary 삭제 정책·FRA 확인

요약

- 동기화 = 전송(도착) + Redo Apply(반영)
- MRP0가 조율, 복구 슬레이브가 병렬 적용
- 기동 DISCONNECT, 중지 CANCEL / 01153·16136
- 19c는 실시간 적용 기본, SRL이 전제, RECOVERY_MODE로 판별
- 갭은 ARCn ping·FAL로 자동 해소, DELAY는 적용만 늦춤
- transport lag는 손실, apply lag는 과거 정도, 신선도 함께

퀴즈 (1/2)

- Q1. 적용 중에 다시 기동하면? → ORA-01153
- Q2. 19c 실시간 적용의 필수 조건은? → 알맞은 크기의 SRL
- Q3. RECOVERY_MODE가 MANAGED이면? → 아카이브 기반 적용 중

퀴즈 (2/2)

- Q4. Primary에서 지워진 로그도 FAL로 복구되나? → 아니다
- Q5. DELAY=240의 효과는? → 전송은 즉시, 적용만 240분 지연
- Q6. lag 0초인데 DATUM_TIME이 20분 전이면? → 연결 단절 의심

실습 개요

- 01 적용 프로세스 관찰: RFS·MRP0·ACTION (7.2, 7.3)
- 02 적용 제어와 오류: ORA-01153, ORA-16136 (7.3)
- 03 BAD PARAM 진단과 교정: 로컬 목적지, FAL (7.5)
- 04 실시간 적용 판별 / 05 갭 만들기와 해소 (7.4, 7.6)
- 06 지연 읽기: 네 지표와 신선도 (7.8)
- 07 SRL과 적용: 실시간 적용의 전제 (7.4)

Oracle Database 19c Active Data Guard 과정

Primary 구조 변경의 Standby 반영

리두가 따라가는 변경, 사람이 따라가야 하는 변경

제8장

학습 목표

- 구조 변경이 리두로 Standby에 전달되는 경로를 설명한다
- STANDBY_FILE_MANAGEMENT와 파일 이름 결정 규칙을 안다
- 파일 추가·삭제·크기·이름 변경의 수동 개입 여부를 판단한다
- UNNAMED 파일을 진단하고 CREATE DATAFILE로 복구한다
- 리두 로그와 PDB 변경을 Standby에서 처리한다
- NOLOGGING 흔적과 리두 밖 항목을 관리한다

Agenda

- 8.2 구조 변경이 Standby에 전달되는 원리
- 8.3 STANDBY_FILE_MANAGEMENT와 파일 이름 결정
- 8.4 테이블스페이스와 데이터 파일 변경 / 8.5 UNNAMED 파일과 복구
- 8.6 온라인 리두 로그와 Standby Redo Log
- 8.7 멀티테넌트 환경의 PDB 변경 / 8.8 NOLOGGING 작업
- 8.9 리두로 전달되지 않는 것 / 8.10 운영 시 주의점

8.2

구조 변경이 Standby에 전달되는 원리

- 리두에 실리는 변경과 실리지 않는 변경
- Standby에서 직접 바꿀 수 없는 이유
- 자동 반영과 수동 개입 분류

리두에 실리는 구조 변경

지시는 리두가, 파일 위치는 Standby가

- CREATE TABLESPACE, ADD DATAFILE, RESIZE, DROP
- MRP0가 Standby 컨트롤 파일에 같은 변경을 가함
- 필요하면 Standby 서버에 실제 파일을 만듦
- 리두에는 파일 번호와 크기만, 경로는 Standby가 정함

리두에 실리지 않는 변경

데이터베이스 밖의 파일과 설정

- 초기화 파라미터(spfile), Oracle Net 설정
- TDE 키스토어, 운영체제 디렉터리
- 패스워드 파일은 12.2부터 리두로 갱신
- 이름 변경은 Standby 쪽 이름에 손대지 않음
- NOLOGGING은 블록 내용 없이 표시만 남김

Standby에서 직접 바꿀 수 없는 이유

구조 변경은 Primary에서 한 방향으로

- MOUNT 또는 READ ONLY 상태라 쓰기 불가
- 열린 Standby에서 ADD DATAFILE은 ORA-16000
- 블록 단위 동일성을 지키려면 Primary가 원천
- 예외: 로그 그룹, SRL, 파일 이름은 Standby에서

자동 반영과 수동 개입 분류

이 장 전체의 지도

- 자동: 파일 생성·추가·크기 변경, PDB 생성·삭제
- 일부 수동: 테이블스페이스 삭제 후 비OMF 파일 정리
- 수동: 파일 이름 변경, 로그 그룹, TTS, 플러그인
- 수동: NOLOGGING 복구, 파라미터, TDE 키스토어
- 자동이라도 적용이 돌고 있어야 반영됨

8.3

STANDBY_FILE_MANAGEMENT와 파일 이름 결정

- AUTO와 MANUAL의 차이
- 새 파일 이름이 정해지는 순서
- AUTO에서 막히는 명령

AUTO와 MANUAL

동작을 정하는 것은 Standby 쪽 값

- AUTO: Standby가 파일을 만들고 적용을 계속
- MANUAL(기본값): UNNAMED 항목만 남기고 적용 중지
- Standby 역할일 때만 의미, Primary 값은 전환 대비
- 동적 파라미터, SCOPE=BOTH로 즉시 변경

새 파일 이름이 정해지는 순서

Standby 설정만 보고 결정

- DB_CREATE_FILE_DEST 있음: OMF 이름, PDB는 GUID 디렉터리
- 없으면 DB_FILE_NAME_CONVERT로 경로 변환
- 둘 다 없으면 Primary와 같은 경로 시도
- 디렉터리가 없으면 생성 실패, UNNAMED로
- 두 방식을 섞지 말고 한쪽으로 통일

DB_FILE_NAME_CONVERT 작성 요령

정적 파라미터, 문자열 치환

- 왼쪽 문자열을 찾아 오른쪽으로 바꿈
- 여러 쌍 나열 가능, 처음 일치한 쌍 사용
- 디렉터리 끝 슬래시까지 맞춰 적기
- 대소문자 그대로 비교
- 변경 후 Standby 재기동 필요

AUTO에서 막히는 명령

자동 관리와 사람의 개입이 부딪히지 않게

- RENAME FILE, ADD/DROP LOGFILE, CREATE DATAFILE
- AUTO 상태에서는 ORA-01275로 거부
- 적용 중지 → MANUAL → 작업 → AUTO → 적용 기동
- 19c Broker: StandbyFileManagement 속성 디프리케이트

8.4

테이블스페이스와 데이터 파일 변경

- 생성·추가·크기 변경은 자동
- 삭제 뒤의 운영체제 파일
- 이름 변경과 TTS는 수동

생성, 추가, 크기 변경

AUTO면 사람이 할 일이 없다

- 같은 파일 번호로, Standby 규칙의 이름으로 생성
- RESIZE, AUTOEXTEND로 늘어난 크기도 전달
- 파일 번호는 연속 보장 없음, FILE#로 비교
- Standby 디스크 부족이면 Standby만 실패
- 임시 파일은 리두로 만들어지지 않음

삭제와 운영체제 파일

항목은 사라지지만 파일은 남을 수 있다

- DROP TABLESPACE는 Standby 컨트롤 파일에서도 제거
- OMF 파일은 함께 삭제
- 비OMF: INCLUDING CONTENTS AND DATAFILES + AUTO면 삭제
- 그 밖에는 Standby에 고아 파일이 남음
- 정리는 삭제 확인 뒤, 지우지 말고 옮겨 두기

이름 변경과 이동

AUTO여도 Standby에 전달되지 않음

- MOVE DATAFILE, RENAME FILE은 Primary만 바꿈
- 적용은 파일 번호로 계속, 이름만 어긋남
- 오프라인: 마운트에서 OS 이동 후 RENAME FILE
- 온라인: READ ONLY로 열고 MOVE DATAFILE
- 적용을 켠 채 이동은 Active Data Guard 옵션 필요

전송 가능 테이블스페이스

파일 내용이 리두에서 오지 않는다

- 플러그인 리두는 연결 선언만 담음
- 적용 시점에 Standby에 같은 파일이 있어야 함
- 데이터 파일 사본을 양쪽에 놓고 impdp로 플러그인
- 경로가 다르면 실패 후 MANUAL, RENAME FILE, 재기동

시나리오: 새 디스크로 옮긴 파일

Primary만 옮기고 잊은 Standby

- Primary에서 큰 파일을 /u02로 온라인 이동
- Standby 파일은 /u01에 그대로, 적용은 정상
- 석 달 뒤 Standby /u01 가득 참, 자동 확장 실패
- 적용 중지, Primary에는 아무 신호 없음

8.5

UNNAMED 파일과 복구

- UNNAMED가 생기는 경로
- 증상 읽기: ORA-01274, ORA-01111
- CREATE DATAFILE로 복구

UNNAMED 파일이 생기는 경로

파일을 만들지 못하면 자리만 남는다

- 컨트롤 파일에 $ORACLE_HOME/dbs/UNNAMEDnnnnn 등록
- 실제 파일은 없음, nnnnn은 파일 번호
- 원인 ①: MANUAL 상태에서 파일 추가
- 원인 ②: 디렉터리 없음, 변환 규칙 누락
- 원인 ③: 공간 부족, 권한 문제

증상 읽기

오류 번호가 진행 단계를 알려 준다

- 처음 멈출 때: ORA-01274 (만들 수 없었다)
- 재기동 시: ORA-01111 (이름 모를 파일)
- V$DATAFILE: UNNAMED, STATUS=RECOVER
- V$DATAFILE_HEADER.ERROR = FILE MISSING
- Primary 목적지는 VALID, apply lag만 증가

CREATE DATAFILE로 복구

빈 파일을 만들고 적용이 채운다

- MANUAL로 내림 (AUTO면 ORA-01275)
- CREATE DATAFILE 'UNNAMED...' AS '실제 경로'
- OMF 구성은 CREATE DATAFILE n AS NEW
- AUTO 복귀, 적용 재기동, V$DATAFILE 확인

재발 방지와 확인 루틴

구조 변경 작업은 Standby 확인으로 닫는다

- MANUAL 원인: AUTO 고정, 복귀 확인을 절차서에
- 경로 원인: 변환 규칙 보완 또는 Standby OMF
- 공간 원인: 양쪽 같은 디스크 감시 기준
- 작업 후: 로그 전환 → FILE# 비교 → 적용 확인

8.6

온라인 리두 로그와 Standby Redo Log

- Standby에 있는 두 종류의 로그
- Primary 로그 변경은 전달되지 않음
- Standby에서의 추가·삭제와 오류

Standby에 있는 두 종류의 로그

지금 쓰는 로그와 전환 뒤에 쓸 로그

- SRL: Standby 역할에서 RFS가 쓰는 곳
- ORL: Standby 역할에서는 쓰이지 않음
- ORL은 역할 전환 뒤 LGWR이 쓰기 시작
- V$LOG 상태: UNUSED, CLEARING, CLEARING_CURRENT

Primary 로그 변경은 전달되지 않는다

AUTO여도 마찬가지

- ADD/DROP LOGFILE GROUP은 Standby ORL을 안 바꿈
- 전환 뒤 그룹 수 부족 → 로그 전환 대기 증가
- ORL만 키우고 SRL을 두면 실시간 적용 강등
- 함께 볼 대상 네 가지: 양쪽 ORL과 SRL

Standby에서 로그 그룹 추가와 삭제

적용 중지와 MANUAL이 전제

- 적용 중지 → MANUAL 전환
- 추가: Primary와 같은 크기로 ADD LOGFILE
- 삭제: CLEAR LOGFILE GROUP n 후 DROP
- CURRENT, CLEARING_CURRENT 그룹은 전환 뒤 삭제
- AUTO 복귀 → 적용 재기동

로그 작업에서 만나는 오류

대부분 상태를 알려 주는 오류

- ORA-01275: AUTO 상태에서 로그 작업
- ORA-01623: CURRENT 그룹 삭제 시도
- ORA-01624: ACTIVE 그룹, 체크포인트 후 재시도
- ORA-00359: 이미 없는 그룹, 조치 불필요
- ORA-01156: 적용이 도는 중에 로그 작업

로그 그룹을 지워도 파일은 남는다

순서를 바꾸면 인스턴스가 내려간다

- 비OMF DROP LOGFILE은 OS 파일을 지우지 않음
- 삭제 성공 확인 전에 OS 파일부터 지우면 위험
- CURRENT 그룹 파일이면 다음 전환에서 인스턴스 종료
- 순서: DB에서 제거 → 조회 확인 → 옮겨 두기

8.7

멀티테넌트 환경의 PDB 변경

- PDB 생성 방법별 파일 확보
- ENABLED_PDBS_ON_STANDBY와 STANDBYS 절
- 제외된 PDB와 PDB 삭제

PDB 생성이 전달되는 방식

파일을 어디서 얻느냐가 갈림길

- 같은 CON_ID, 같은 GUID로 Standby에 등록
- 시드에서 생성: Standby의 PDB$SEED 복사, 자동
- 로컬 클론: Standby의 원본 PDB 복사
- 원격 클론, XML 플러그인: 원본이 Standby에 없음
- 새 PDB는 Standby에서 MOUNTED로 시작

18c의 자동 복사 파라미터

원격 클론과 플러그인의 수작업 줄이기

- 12c: Primary 생성 전에 Standby에 파일 직접 복사
- STANDBY_PDB_SOURCE_FILE_DIRECTORY: 플러그인 파일 위치
- STANDBY_PDB_SOURCE_FILE_DBLINK: 원격 클론 원본 링크
- DB 링크는 원본 PDB를 가리키고 원본이 READ ONLY여야 함
- 조건이 안 맞으면 여전히 수동 복사

ENABLED_PDBS_ON_STANDBY

Standby마다 복제할 PDB 고르기

- Standby에 설정, Primary에서는 무시
- 미지정이면 모든 PDB("*")
- 와일드카드 *, 앞에 -를 붙이면 제외
- 여러 패턴이 겹치면 오른쪽이 우선
- Primary 쪽 선택: CREATE PLUGGABLE DATABASE ... STANDBYS

제외된 PDB와 나중에 포함하기

RECOVERY_STATUS로 구분

- 제외된 PDB도 이름과 메타데이터는 등록
- 데이터 파일은 OFFLINE, 리두 적용 대상에서 빠짐
- V$PDBS.RECOVERY_STATUS: ENABLED / DISABLED
- 포함: FROM SERVICE로 파일 복원 → ENABLE RECOVERY
- 제외된 PDB는 역할 전환 뒤 새 Primary에서 못 엶

PDB 삭제와 시나리오

개발 PDB 때문에 멈춘 적용

- DROP PLUGGABLE DATABASE는 Standby에도 반영
- OMF는 빈 GUID 디렉터리가 남을 수 있음, rmdir로 정리
- 시나리오: 원격 클론 개발 PDB에서 적용 중지
- 대책: '*','-DEV*' 설정과 이름 규칙 통일

8.8

NOLOGGING 작업과 복구 불가 블록

- Standby에서 일어나는 일
- 영향받은 파일 찾기
- 복구와 예방

Standby에서 무엇이 일어나는가

적용은 조용히 지나가고 조회에서 터진다

- 직접 경로 적재 + NOLOGGING: 블록 내용 없음
- Standby는 블록을 복구 불가로 표시하고 진행
- 조회나 Failover 뒤 업무에서 오류 발생
- ORA-01578 + ORA-26040 = NOLOGGING 흔적

영향받은 파일 찾기

UNRECOVERABLE_CHANGE# 비교

- V$DATAFILE.UNRECOVERABLE_CHANGE#, _TIME
- 양쪽 값을 FILE#별로 비교
- Primary 값이 크면 Standby가 내용을 못 가진 파일
- 12.2 이후 Standby의 V$NONLOGGED_BLOCK

복구 방법

파일 전체 또는 블록만

- 교재 방식: BEGIN BACKUP 후 파일 복사
- RMAN RECOVER DATAFILE n FROM SERVICE (12c 이후)
- RMAN NONLOGGED BLOCK 복구 (12.2 이후)
- 적용 중지 상태에서 진행, 끝나면 재기동
- ORA-00308, ORA-01152면 갭부터 해소

예방

FORCE LOGGING이 기본 방어선

- Primary FORCE LOGGING: 객체 설정과 무관하게 리두 생성
- 켠 이후 작업만 보호, 기존 흔적은 별도 복구
- CDB 단위 설정, V$DATABASE.FORCE_LOGGING 확인
- 18c Standby nologging 모드: ADG 옵션 필요

8.9

리두로 전달되지 않는 것

- 초기화 파라미터
- 패스워드 파일과 Far Sync
- TDE 키스토어와 그 밖의 항목

초기화 파라미터

spfile 변경은 리두에 없다

- Primary의 ALTER SYSTEM은 Standby에 반영 안 됨
- 같아야 할 값: 메모리, PROCESSES, 옵티마이저
- 달라야 할 값: DB_UNIQUE_NAME, FAL_SERVER, 목적지
- COMPATIBLE은 Standby 먼저, Primary 나중

패스워드 파일

12.2부터 리두로 갱신, Far Sync는 예외

- 리두 전송 인증에 패스워드 파일 사용
- 12.2+: 권한 부여·회수, 암호 변경이 리두로 전달
- Far Sync는 적용하지 않으므로 수동 복사
- 적용이 밀린 Standby는 변경을 아직 모름

TDE 키스토어

암호화된 리두는 가지만 키는 가지 않는다

- 암호화 데이터는 리두에도 암호화된 채 전달
- 마스터 키는 데이터베이스 밖 키스토어 파일에
- SET KEY로 키 재설정 후 Standby에 키스토어 복사
- 자동 로그인 키스토어(cwallet.sso)도 함께
- 외부 키 관리(OKV)면 복사 불필요

그 밖의 항목과 점검표

역할 전환 연습 때마다 비교

- CREATE CONTROLFILE 후 Standby 컨트롤 파일 재생성
- tnsnames.ora, listener.ora, OS 사용자와 권한
- 점검: 성능 파라미터, COMPATIBLE
- 점검: V$PWFILE_USERS, 키스토어 갱신 시각
- 점검: ORL·SRL 구성, 임시 파일

8.10

운영 시 주의점

- 구조 변경 계획서에 Standby 단계
- 양쪽 AUTO, Standby에서 확인
- 리두 밖 항목은 전환 연습 때 비교

운영 시 주의점

자동 반영에도 사람의 확인이 필요하다

- 구조 변경 계획서에 Standby 확인 단계 포함
- 양쪽 AUTO, 값은 Standby에서 조회
- 경로 방식 Standby는 새 디렉터리마다 규칙 점검
- 로그 구성 변경은 네 군데를 한 작업으로
- FORCE LOGGING과 UNRECOVERABLE_CHANGE# 정기 점검

요약

- 구조 변경은 리두로 전달, 파일 위치는 Standby 설정이 결정
- AUTO면 파일 생성·삭제 자동, MANUAL이면 UNNAMED와 적용 중지
- 이름 변경, 로그 그룹, TTS·플러그인 파일은 수동
- UNNAMED: ORA-01274 → ORA-01111, MANUAL에서 CREATE DATAFILE
- PDB 복제 대상은 ENABLED_PDBS_ON_STANDBY, STANDBYS 절
- NOLOGGING은 FORCE LOGGING으로 예방, 파라미터·TDE는 리두 밖

퀴즈 (1/2)

- Q1. Primary만 AUTO, Standby는 MANUAL이면? → UNNAMED와 적용 중지
- Q2. AUTO Standby에서 RENAME FILE 오류는? → ORA-01275
- Q3. Primary에서 MOVE DATAFILE 후 Standby는? → 원래 경로, 적용 계속

퀴즈 (2/2)

- Q4. Primary에 로그 그룹 추가 후 Standby는? → MANUAL로 직접 추가
- Q5. "*","-PDB*A","PDB2A"의 결과는? → PDB1B, PDB2A
- Q6. ORA-01578 + ORA-26040이면? → NOLOGGING 흔적, 다시 가져옴

실습 개요

- 01 자동 반영 확인 / 02 파일 추가와 크기 변경 (8.3, 8.4)
- 03 MANUAL과 UNNAMED 파일 (8.3, 8.5)
- 04 UNNAMED 파일 복구 (8.5)
- 05 PDB 추가와 삭제 (8.7)
- 06 반영되지 않는 것: 온라인 리두 로그 (8.6)
- 07 사고 분석: CURRENT 리두 삭제, 읽기 자료 (8.6)

Oracle Database 19c Active Data Guard 과정

Primary 구조 변경의 Standby 반영

리두가 따라가는 변경, 사람이 따라가야 하는 변경

제8장

학습 목표

- 구조 변경이 리두로 Standby에 전달되는 경로를 설명한다
- STANDBY_FILE_MANAGEMENT와 파일 이름 결정 규칙을 안다
- 파일 추가·삭제·크기·이름 변경의 수동 개입 여부를 판단한다
- UNNAMED 파일을 진단하고 CREATE DATAFILE로 복구한다
- 리두 로그와 PDB 변경을 Standby에서 처리한다
- NOLOGGING 흔적과 리두 밖 항목을 관리한다

Agenda

- 8.2 구조 변경이 Standby에 전달되는 원리
- 8.3 STANDBY_FILE_MANAGEMENT와 파일 이름 결정
- 8.4 테이블스페이스와 데이터 파일 변경 / 8.5 UNNAMED 파일과 복구
- 8.6 온라인 리두 로그와 Standby Redo Log
- 8.7 멀티테넌트 환경의 PDB 변경 / 8.8 NOLOGGING 작업
- 8.9 리두로 전달되지 않는 것 / 8.10 운영 시 주의점

8.2

구조 변경이 Standby에 전달되는 원리

- 리두에 실리는 변경과 실리지 않는 변경
- Standby에서 직접 바꿀 수 없는 이유
- 자동 반영과 수동 개입 분류

리두에 실리는 구조 변경

지시는 리두가, 파일 위치는 Standby가

- CREATE TABLESPACE, ADD DATAFILE, RESIZE, DROP
- MRP0가 Standby 컨트롤 파일에 같은 변경을 가함
- 필요하면 Standby 서버에 실제 파일을 만듦
- 리두에는 파일 번호와 크기만, 경로는 Standby가 정함

리두에 실리지 않는 변경

데이터베이스 밖의 파일과 설정

- 초기화 파라미터(spfile), Oracle Net 설정
- TDE 키스토어, 운영체제 디렉터리
- 패스워드 파일은 12.2부터 리두로 갱신
- 이름 변경은 Standby 쪽 이름에 손대지 않음
- NOLOGGING은 블록 내용 없이 표시만 남김

Standby에서 직접 바꿀 수 없는 이유

구조 변경은 Primary에서 한 방향으로

- MOUNT 또는 READ ONLY 상태라 쓰기 불가
- 열린 Standby에서 ADD DATAFILE은 ORA-16000
- 블록 단위 동일성을 지키려면 Primary가 원천
- 예외: 로그 그룹, SRL, 파일 이름은 Standby에서

자동 반영과 수동 개입 분류

이 장 전체의 지도

- 자동: 파일 생성·추가·크기 변경, PDB 생성·삭제
- 일부 수동: 테이블스페이스 삭제 후 비OMF 파일 정리
- 수동: 파일 이름 변경, 로그 그룹, TTS, 플러그인
- 수동: NOLOGGING 복구, 파라미터, TDE 키스토어
- 자동이라도 적용이 돌고 있어야 반영됨

8.3

STANDBY_FILE_MANAGEMENT와 파일 이름 결정

- AUTO와 MANUAL의 차이
- 새 파일 이름이 정해지는 순서
- AUTO에서 막히는 명령

AUTO와 MANUAL

동작을 정하는 것은 Standby 쪽 값

- AUTO: Standby가 파일을 만들고 적용을 계속
- MANUAL(기본값): UNNAMED 항목만 남기고 적용 중지
- Standby 역할일 때만 의미, Primary 값은 전환 대비
- 동적 파라미터, SCOPE=BOTH로 즉시 변경

새 파일 이름이 정해지는 순서

Standby 설정만 보고 결정

- DB_CREATE_FILE_DEST 있음: OMF 이름, PDB는 GUID 디렉터리
- 없으면 DB_FILE_NAME_CONVERT로 경로 변환
- 둘 다 없으면 Primary와 같은 경로 시도
- 디렉터리가 없으면 생성 실패, UNNAMED로
- 두 방식을 섞지 말고 한쪽으로 통일

DB_FILE_NAME_CONVERT 작성 요령

정적 파라미터, 문자열 치환

- 왼쪽 문자열을 찾아 오른쪽으로 바꿈
- 여러 쌍 나열 가능, 처음 일치한 쌍 사용
- 디렉터리 끝 슬래시까지 맞춰 적기
- 대소문자 그대로 비교
- 변경 후 Standby 재기동 필요

AUTO에서 막히는 명령

자동 관리와 사람의 개입이 부딪히지 않게

- RENAME FILE, ADD/DROP LOGFILE, CREATE DATAFILE
- AUTO 상태에서는 ORA-01275로 거부
- 적용 중지 → MANUAL → 작업 → AUTO → 적용 기동
- 19c Broker: StandbyFileManagement 속성 디프리케이트

8.4

테이블스페이스와 데이터 파일 변경

- 생성·추가·크기 변경은 자동
- 삭제 뒤의 운영체제 파일
- 이름 변경과 TTS는 수동

생성, 추가, 크기 변경

AUTO면 사람이 할 일이 없다

- 같은 파일 번호로, Standby 규칙의 이름으로 생성
- RESIZE, AUTOEXTEND로 늘어난 크기도 전달
- 파일 번호는 연속 보장 없음, FILE#로 비교
- Standby 디스크 부족이면 Standby만 실패
- 임시 파일은 리두로 만들어지지 않음

삭제와 운영체제 파일

항목은 사라지지만 파일은 남을 수 있다

- DROP TABLESPACE는 Standby 컨트롤 파일에서도 제거
- OMF 파일은 함께 삭제
- 비OMF: INCLUDING CONTENTS AND DATAFILES + AUTO면 삭제
- 그 밖에는 Standby에 고아 파일이 남음
- 정리는 삭제 확인 뒤, 지우지 말고 옮겨 두기

이름 변경과 이동

AUTO여도 Standby에 전달되지 않음

- MOVE DATAFILE, RENAME FILE은 Primary만 바꿈
- 적용은 파일 번호로 계속, 이름만 어긋남
- 오프라인: 마운트에서 OS 이동 후 RENAME FILE
- 온라인: READ ONLY로 열고 MOVE DATAFILE
- 적용을 켠 채 이동은 Active Data Guard 옵션 필요

전송 가능 테이블스페이스

파일 내용이 리두에서 오지 않는다

- 플러그인 리두는 연결 선언만 담음
- 적용 시점에 Standby에 같은 파일이 있어야 함
- 데이터 파일 사본을 양쪽에 놓고 impdp로 플러그인
- 경로가 다르면 실패 후 MANUAL, RENAME FILE, 재기동

시나리오: 새 디스크로 옮긴 파일

Primary만 옮기고 잊은 Standby

- Primary에서 큰 파일을 /u02로 온라인 이동
- Standby 파일은 /u01에 그대로, 적용은 정상
- 석 달 뒤 Standby /u01 가득 참, 자동 확장 실패
- 적용 중지, Primary에는 아무 신호 없음

8.5

UNNAMED 파일과 복구

- UNNAMED가 생기는 경로
- 증상 읽기: ORA-01274, ORA-01111
- CREATE DATAFILE로 복구

UNNAMED 파일이 생기는 경로

파일을 만들지 못하면 자리만 남는다

- 컨트롤 파일에 $ORACLE_HOME/dbs/UNNAMEDnnnnn 등록
- 실제 파일은 없음, nnnnn은 파일 번호
- 원인 ①: MANUAL 상태에서 파일 추가
- 원인 ②: 디렉터리 없음, 변환 규칙 누락
- 원인 ③: 공간 부족, 권한 문제

증상 읽기

오류 번호가 진행 단계를 알려 준다

- 처음 멈출 때: ORA-01274 (만들 수 없었다)
- 재기동 시: ORA-01111 (이름 모를 파일)
- V$DATAFILE: UNNAMED, STATUS=RECOVER
- V$DATAFILE_HEADER.ERROR = FILE MISSING
- Primary 목적지는 VALID, apply lag만 증가

CREATE DATAFILE로 복구

빈 파일을 만들고 적용이 채운다

- MANUAL로 내림 (AUTO면 ORA-01275)
- CREATE DATAFILE 'UNNAMED...' AS '실제 경로'
- OMF 구성은 CREATE DATAFILE n AS NEW
- AUTO 복귀, 적용 재기동, V$DATAFILE 확인

재발 방지와 확인 루틴

구조 변경 작업은 Standby 확인으로 닫는다

- MANUAL 원인: AUTO 고정, 복귀 확인을 절차서에
- 경로 원인: 변환 규칙 보완 또는 Standby OMF
- 공간 원인: 양쪽 같은 디스크 감시 기준
- 작업 후: 로그 전환 → FILE# 비교 → 적용 확인

8.6

온라인 리두 로그와 Standby Redo Log

- Standby에 있는 두 종류의 로그
- Primary 로그 변경은 전달되지 않음
- Standby에서의 추가·삭제와 오류

Standby에 있는 두 종류의 로그

지금 쓰는 로그와 전환 뒤에 쓸 로그

- SRL: Standby 역할에서 RFS가 쓰는 곳
- ORL: Standby 역할에서는 쓰이지 않음
- ORL은 역할 전환 뒤 LGWR이 쓰기 시작
- V$LOG 상태: UNUSED, CLEARING, CLEARING_CURRENT

Primary 로그 변경은 전달되지 않는다

AUTO여도 마찬가지

- ADD/DROP LOGFILE GROUP은 Standby ORL을 안 바꿈
- 전환 뒤 그룹 수 부족 → 로그 전환 대기 증가
- ORL만 키우고 SRL을 두면 실시간 적용 강등
- 함께 볼 대상 네 가지: 양쪽 ORL과 SRL

Standby에서 로그 그룹 추가와 삭제

적용 중지와 MANUAL이 전제

- 적용 중지 → MANUAL 전환
- 추가: Primary와 같은 크기로 ADD LOGFILE
- 삭제: CLEAR LOGFILE GROUP n 후 DROP
- CURRENT, CLEARING_CURRENT 그룹은 전환 뒤 삭제
- AUTO 복귀 → 적용 재기동

로그 작업에서 만나는 오류

대부분 상태를 알려 주는 오류

- ORA-01275: AUTO 상태에서 로그 작업
- ORA-01623: CURRENT 그룹 삭제 시도
- ORA-01624: ACTIVE 그룹, 체크포인트 후 재시도
- ORA-00359: 이미 없는 그룹, 조치 불필요
- ORA-01156: 적용이 도는 중에 로그 작업

로그 그룹을 지워도 파일은 남는다

순서를 바꾸면 인스턴스가 내려간다

- 비OMF DROP LOGFILE은 OS 파일을 지우지 않음
- 삭제 성공 확인 전에 OS 파일부터 지우면 위험
- CURRENT 그룹 파일이면 다음 전환에서 인스턴스 종료
- 순서: DB에서 제거 → 조회 확인 → 옮겨 두기

8.7

멀티테넌트 환경의 PDB 변경

- PDB 생성 방법별 파일 확보
- ENABLED_PDBS_ON_STANDBY와 STANDBYS 절
- 제외된 PDB와 PDB 삭제

PDB 생성이 전달되는 방식

파일을 어디서 얻느냐가 갈림길

- 같은 CON_ID, 같은 GUID로 Standby에 등록
- 시드에서 생성: Standby의 PDB$SEED 복사, 자동
- 로컬 클론: Standby의 원본 PDB 복사
- 원격 클론, XML 플러그인: 원본이 Standby에 없음
- 새 PDB는 Standby에서 MOUNTED로 시작

18c의 자동 복사 파라미터

원격 클론과 플러그인의 수작업 줄이기

- 12c: Primary 생성 전에 Standby에 파일 직접 복사
- STANDBY_PDB_SOURCE_FILE_DIRECTORY: 플러그인 파일 위치
- STANDBY_PDB_SOURCE_FILE_DBLINK: 원격 클론 원본 링크
- DB 링크는 원본 PDB를 가리키고 원본이 READ ONLY여야 함
- 조건이 안 맞으면 여전히 수동 복사

ENABLED_PDBS_ON_STANDBY

Standby마다 복제할 PDB 고르기

- Standby에 설정, Primary에서는 무시
- 미지정이면 모든 PDB("*")
- 와일드카드 *, 앞에 -를 붙이면 제외
- 여러 패턴이 겹치면 오른쪽이 우선
- Primary 쪽 선택: CREATE PLUGGABLE DATABASE ... STANDBYS

제외된 PDB와 나중에 포함하기

RECOVERY_STATUS로 구분

- 제외된 PDB도 이름과 메타데이터는 등록
- 데이터 파일은 OFFLINE, 리두 적용 대상에서 빠짐
- V$PDBS.RECOVERY_STATUS: ENABLED / DISABLED
- 포함: FROM SERVICE로 파일 복원 → ENABLE RECOVERY
- 제외된 PDB는 역할 전환 뒤 새 Primary에서 못 엶

PDB 삭제와 시나리오

개발 PDB 때문에 멈춘 적용

- DROP PLUGGABLE DATABASE는 Standby에도 반영
- OMF는 빈 GUID 디렉터리가 남을 수 있음, rmdir로 정리
- 시나리오: 원격 클론 개발 PDB에서 적용 중지
- 대책: '*','-DEV*' 설정과 이름 규칙 통일

8.8

NOLOGGING 작업과 복구 불가 블록

- Standby에서 일어나는 일
- 영향받은 파일 찾기
- 복구와 예방

Standby에서 무엇이 일어나는가

적용은 조용히 지나가고 조회에서 터진다

- 직접 경로 적재 + NOLOGGING: 블록 내용 없음
- Standby는 블록을 복구 불가로 표시하고 진행
- 조회나 Failover 뒤 업무에서 오류 발생
- ORA-01578 + ORA-26040 = NOLOGGING 흔적

영향받은 파일 찾기

UNRECOVERABLE_CHANGE# 비교

- V$DATAFILE.UNRECOVERABLE_CHANGE#, _TIME
- 양쪽 값을 FILE#별로 비교
- Primary 값이 크면 Standby가 내용을 못 가진 파일
- 12.2 이후 Standby의 V$NONLOGGED_BLOCK

복구 방법

파일 전체 또는 블록만

- 교재 방식: BEGIN BACKUP 후 파일 복사
- RMAN RECOVER DATAFILE n FROM SERVICE (12c 이후)
- RMAN NONLOGGED BLOCK 복구 (12.2 이후)
- 적용 중지 상태에서 진행, 끝나면 재기동
- ORA-00308, ORA-01152면 갭부터 해소

예방

FORCE LOGGING이 기본 방어선

- Primary FORCE LOGGING: 객체 설정과 무관하게 리두 생성
- 켠 이후 작업만 보호, 기존 흔적은 별도 복구
- CDB 단위 설정, V$DATABASE.FORCE_LOGGING 확인
- 18c Standby nologging 모드: ADG 옵션 필요

8.9

리두로 전달되지 않는 것

- 초기화 파라미터
- 패스워드 파일과 Far Sync
- TDE 키스토어와 그 밖의 항목

초기화 파라미터

spfile 변경은 리두에 없다

- Primary의 ALTER SYSTEM은 Standby에 반영 안 됨
- 같아야 할 값: 메모리, PROCESSES, 옵티마이저
- 달라야 할 값: DB_UNIQUE_NAME, FAL_SERVER, 목적지
- COMPATIBLE은 Standby 먼저, Primary 나중

패스워드 파일

12.2부터 리두로 갱신, Far Sync는 예외

- 리두 전송 인증에 패스워드 파일 사용
- 12.2+: 권한 부여·회수, 암호 변경이 리두로 전달
- Far Sync는 적용하지 않으므로 수동 복사
- 적용이 밀린 Standby는 변경을 아직 모름

TDE 키스토어

암호화된 리두는 가지만 키는 가지 않는다

- 암호화 데이터는 리두에도 암호화된 채 전달
- 마스터 키는 데이터베이스 밖 키스토어 파일에
- SET KEY로 키 재설정 후 Standby에 키스토어 복사
- 자동 로그인 키스토어(cwallet.sso)도 함께
- 외부 키 관리(OKV)면 복사 불필요

그 밖의 항목과 점검표

역할 전환 연습 때마다 비교

- CREATE CONTROLFILE 후 Standby 컨트롤 파일 재생성
- tnsnames.ora, listener.ora, OS 사용자와 권한
- 점검: 성능 파라미터, COMPATIBLE
- 점검: V$PWFILE_USERS, 키스토어 갱신 시각
- 점검: ORL·SRL 구성, 임시 파일

8.10

운영 시 주의점

- 구조 변경 계획서에 Standby 단계
- 양쪽 AUTO, Standby에서 확인
- 리두 밖 항목은 전환 연습 때 비교

운영 시 주의점

자동 반영에도 사람의 확인이 필요하다

- 구조 변경 계획서에 Standby 확인 단계 포함
- 양쪽 AUTO, 값은 Standby에서 조회
- 경로 방식 Standby는 새 디렉터리마다 규칙 점검
- 로그 구성 변경은 네 군데를 한 작업으로
- FORCE LOGGING과 UNRECOVERABLE_CHANGE# 정기 점검

요약

- 구조 변경은 리두로 전달, 파일 위치는 Standby 설정이 결정
- AUTO면 파일 생성·삭제 자동, MANUAL이면 UNNAMED와 적용 중지
- 이름 변경, 로그 그룹, TTS·플러그인 파일은 수동
- UNNAMED: ORA-01274 → ORA-01111, MANUAL에서 CREATE DATAFILE
- PDB 복제 대상은 ENABLED_PDBS_ON_STANDBY, STANDBYS 절
- NOLOGGING은 FORCE LOGGING으로 예방, 파라미터·TDE는 리두 밖

퀴즈 (1/2)

- Q1. Primary만 AUTO, Standby는 MANUAL이면? → UNNAMED와 적용 중지
- Q2. AUTO Standby에서 RENAME FILE 오류는? → ORA-01275
- Q3. Primary에서 MOVE DATAFILE 후 Standby는? → 원래 경로, 적용 계속

퀴즈 (2/2)

- Q4. Primary에 로그 그룹 추가 후 Standby는? → MANUAL로 직접 추가
- Q5. "*","-PDB*A","PDB2A"의 결과는? → PDB1B, PDB2A
- Q6. ORA-01578 + ORA-26040이면? → NOLOGGING 흔적, 다시 가져옴

실습 개요

- 01 자동 반영 확인 / 02 파일 추가와 크기 변경 (8.3, 8.4)
- 03 MANUAL과 UNNAMED 파일 (8.3, 8.5)
- 04 UNNAMED 파일 복구 (8.5)
- 05 PDB 추가와 삭제 (8.7)
- 06 반영되지 않는 것: 온라인 리두 로그 (8.6)
- 07 사고 분석: CURRENT 리두 삭제, 읽기 자료 (8.6)

Oracle Database 19c Active Data Guard 과정

Active Data Guard ① 실시간 조회와 DML 리다이렉션

열린 채 적용하는 Standby에 무엇을 맡길 것인가

제9장

학습 목표

- 옵션의 범위와 과금 경계를 설명한다
- 실시간 조회를 켜고 끄고 판별한다
- 쿼리 SCN으로 읽기 일관성을 설명한다
- 오프로드 가능 작업을 기준으로 가른다
- 지연 한도와 강제 동기화로 신선도를 통제한다
- 임시 테이블·시퀀스·DML 리다이렉션을 쓴다

Agenda

- 9.2 Active Data Guard 옵션과 라이선스
- 9.3 Real-Time Query의 동작
- 9.4 Standby에서 되는 일과 안 되는 일
- 9.5 데이터 신선도 관리 / 9.6 임시 데이터와 시퀀스
- 9.7 DML 리다이렉션 / 9.8 자동 블록 손상 복구와 관련 기능
- 9.9 옵션 없이 쓸 수 있는 대안 / 9.10 운영 시 주의점

9.2

Active Data Guard 옵션과 라이선스

- Standby를 쉬게 두지 않으려는 이유
- 옵션에 묶인 기능
- 어떤 상태가 옵션 사용인가

Standby를 쉬게 두지 않으려는 이유

보호용 사본을 평소에도 일하게

- 마운트 Standby는 장애 전까지 업무 기여가 없음
- 보고서·추출·임의 조회를 Standby로 오프로드
- 별도 복제 도구 없이, 데이터 유형 제약 없이
- 갱신은 Primary, 조회는 여러 Standby로 분산
- 보고서용 인덱스·MV는 만들 수 없음

옵션에 묶인 기능

하나라도 쓰면 옵션 사용

- Real-Time Query, 임시 테이블·시퀀스 지원
- DML·PL/SQL 리다이렉션
- 자동 블록 손상 복구, Standby In-Memory
- Standby의 블록 변경 추적
- Far Sync, Real-Time Cascading (10장)

과금 경계

열린 채로 적용하면 옵션 사용

- 마운트 상태에서 적용: 기본 기능
- 적용을 멈추고 READ ONLY: 기본 기능
- READ ONLY 상태에서 적용 기동: 옵션 사용
- STARTUP만으로 기동하면 READ ONLY로 열림
- 옵션 없는 사이트는 STARTUP MOUNT로 고정

사용 흔적 확인

상태는 OPEN_MODE, 기록은 기능 사용 통계

- 지금 상태: V$DATABASE.OPEN_MODE
- READ ONLY WITH APPLY면 옵션 사용 중
- 기록: CDB_FEATURE_USAGE_STATISTICS
- CDB에서 DBA_ 뷰는 현재 컨테이너만 보여 줌
- 통계는 늦게 갱신되므로 0이 미사용 증거는 아님

9.3

Real-Time Query의 동작

- 실시간 적용과 실시간 조회의 차이
- 켜고 끄는 절차, 열린 모드 판별
- 쿼리 SCN과 읽기 일관성

실시간 적용과 실시간 조회

이름은 비슷하지만 다른 축

- 실시간 적용: 리두를 언제 읽어 적용하는가
- 실시간 조회: 적용 중에 조회할 수 있는가
- 실시간 적용은 SRL 전제, 19c 기본, 기본 기능
- 실시간 조회는 명시적 열기, 옵션 필요
- 둘을 함께 써야 초 단위 신선도

켜는 절차

멈추고, 열고, 다시 기동

- RECOVER MANAGED STANDBY DATABASE CANCEL
- ALTER DATABASE OPEN READ ONLY
- CDB면 ALTER PLUGGABLE DATABASE ALL OPEN READ ONLY
- RECOVER MANAGED STANDBY DATABASE DISCONNECT
- Broker 구성은 열기만 하면 적용 상태 유지

끄는 절차

열린 DB를 마운트로 바로 내리는 명령은 없음

- SHUTDOWN IMMEDIATE
- STARTUP MOUNT
- RECOVER MANAGED STANDBY DATABASE DISCONNECT
- 적용만 CANCEL하면 READ ONLY로 남음
- COMPATIBLE 11.0 이상이 전제 (19c는 충족)

열린 모드 판별

세 위치가 같은 사실을 보여 줌

- MOUNTED / READ ONLY / READ ONLY WITH APPLY
- Primary의 RECOVERY_MODE 끝에 WITH QUERY
- Broker SHOW DATABASE: Real Time Query ON
- Standby 쪽 하나, Primary 쪽 하나를 함께 확인

읽기 일관성과 쿼리 SCN

Standby 결과 = 조금 과거의 Primary 결과

- 커밋 전 변경도 리두로 Standby 블록에 반영됨
- 언두 블록도 함께 적용되어 있음
- 쿼리 SCN 기준으로 블록을 되돌려 읽음
- 적용이 멈추면 쿼리 SCN도 멈춤
- 긴 조회는 ORA-01555 가능, Primary 언두 설정과 연동

9.4

Standby에서 되는 일과 안 되는 일

- 쓰기가 막히는 이유
- 오프로드에 적합한 작업
- 판단 기준: 영구 파일·딕셔너리를 바꾸는가

쓰기가 막히는 이유

데이터 파일은 Primary의 리두로만 바뀐다

- Standby가 블록을 바꾸면 다음 리두와 어긋남
- 일반 DML·DDL: ORA-16000
- 통계 수집: 딕셔너리 기록이라 ORA-16000
- 로컬 AWR 스냅샷: ORA-13516
- 옵티마이저는 Primary에서 넘어온 통계를 사용

오프로드에 적합한 작업

파일을 바꾸지 않거나 임시 공간만 쓰는 작업

- 조회·보고서, 정렬·해시 조인(TEMP 사용)
- V$ 뷰, DISPLAY_CURSOR, EXPLAIN PLAN
- 파일 추출, 네트워크 링크 기반 Data Pump
- RMAN 백업(기본 기능), BCT는 옵션
- 임시 파일은 리두로 안 만들어짐, 직접 추가

막히는 작업과 우회 경로

기준 하나로 결과를 예측

- 일반 DML → 리다이렉션 또는 Primary 접속
- DDL·인덱스·통계 → Primary에서 수행
- 로컬 AWR → 원격 AWR(UMF)
- GTT·PTT·CACHE 시퀀스 → Standby에서 성공
- 보고서용 인덱스·MV → 논리 Standby

9.5

데이터 신선도 관리

- 지연 관찰과 히스토그램
- STANDBY_MAX_DATA_DELAY, zero lag
- SYNC WITH PRIMARY, 트리거, 서비스

지연의 원인과 분포

순간값보다 분포로 한도를 정한다

- 원인: CPU·I/O 부족, 네트워크 지연, 대역폭
- transport lag와 apply lag로 구간 구분(7.8절)
- V$STANDBY_EVENT_HISTOGRAM: 값별 관측 횟수
- 누적값이므로 시작·끝을 저장해 차이를 냄

STANDBY_MAX_DATA_DELAY

세션이 허용할 지연을 초 단위로

- ALTER SESSION SET STANDBY_MAX_DATA_DELAY = 5
- 세션 파라미터, 시스템 수준 설정 없음
- 한도 초과 시 ORA-03172, 경고 로그에도 기록
- 기본 NONE: 지연과 무관하게 실행
- SYS에는 적용되지 않음 (일반 사용자로 검증)

zero lag

한도 0은 전제 조건이 붙는 별도 모드

- 쿼리 SCN이 Primary의 현재 SCN에 닿을 때까지 대기
- 결과가 같은 시점 Primary 결과와 동일
- 보호 모드: Maximum Availability 이상
- 해당 Standby로 SYNC 전송
- 실시간 조회 활성

ALTER SESSION SYNC WITH PRIMARY

세션 시작 시점에 한 번 맞추기

- 실행 시점의 Primary까지 따라잡을 때까지 대기
- 돌아온 뒤의 쿼리는 기다리지 않음
- 적용이 없거나 중간에 취소되면 ORA-03173
- SYNC 전송 구성을 전제로 사용

역할 인식 AFTER LOGON 트리거

애플리케이션 소스를 고치지 않고 정책 적용

- SYS_CONTEXT('USERENV','DATABASE_ROLE')
- PHYSICAL STANDBY일 때만 ALTER SESSION 실행
- Primary에서 생성, 리두로 Standby에 전달
- 역할 전환 뒤에도 다시 만들 필요 없음
- SYNC WITH PRIMARY 트리거는 전용 연결에서만

역할 기반 서비스

서버 이름 대신 서비스 이름으로 접속

- 갱신용 sales_rw는 Primary, 조회용 sales_ro는 Standby
- Grid·Restart: srvctl add service -role
- 단일 인스턴스: DBMS_SERVICE + 시작 트리거
- 역할 전환 대비 DB_ROLE_CHANGE 트리거
- MOUNTED로 내리면 조회 서비스 접속은 ORA-12514

사례: 신선도 요구가 다른 두 업무

서비스와 트리거로 정책을 나눈다

- 대시보드: 5초 한도 트리거, 초과 시 Primary로 재시도
- 정산 확인: zero lag 조건의 SYNC Standby에서만
- 조건이 없으면 정산은 처음부터 Primary 서비스
- 정책 변경은 서비스 단위로 끝냄

9.6

Standby에서의 임시 데이터와 시퀀스

- 전역 임시 테이블과 임시 언두
- 전용 임시 테이블
- 전역 시퀀스와 세션 시퀀스

GTT의 DML과 임시 언두

언두를 TEMP에 두면 리두가 생기지 않는다

- GTT 데이터 변경 자체는 리두 없음
- 문제는 언두: 언두 기록이 리두를 만듦
- 임시 언두: GTT 언두를 임시 테이블스페이스에
- ADG Standby에서는 항상 켜짐(파라미터 무시)
- Primary에서 TEMP_UNDO_ENABLED=TRUE로 리두 감소

GTT의 생성과 삭제

정의는 딕셔너리에 있으므로 Primary에서

- GTT 정의는 Primary에서 만들어 리두로 전달
- 교재: Standby의 GTT DDL을 Primary로 투명하게 전달
- 숨은 파라미터 _enable_proxy_adg_redirect 사용
- 운영 정석은 Primary에서 미리 생성
- 일반 테이블 DDL은 리다이렉션 대상 아님

전용 임시 테이블 (PTT)

정의가 메모리에 있어 Standby에서 직접 생성

- 18c 도입, 정의·데이터 모두 세션 전용
- 이름은 ORA$PTT_ 접두어로 시작
- ON COMMIT DROP DEFINITION (기본, 트랜잭션)
- ON COMMIT PRESERVE DEFINITION (세션)
- 같은 이름, 다른 열 구성을 세션마다 가질 수 있음

시퀀스

전역 시퀀스는 범위 할당, 세션 시퀀스는 세션 안에서만

- CACHE·NOORDER 시퀀스는 Standby에서 NEXTVAL 가능
- Primary가 캐시 크기만큼 범위를 떼어 줌
- 번호는 유일하지만 연속적이지 않음
- ORDER·NOCACHE 시퀀스는 사용 불가
- CREATE SEQUENCE ... SESSION: 세션 범위, 비영속

9.7

DML 리다이렉션

- 동작 흐름과 가시성
- ADG_REDIRECT_DML, ADG_REDIRECT_PLSQL
- 제약, 오류 번호, 쓸 곳과 피할 곳

동작 흐름

기록은 전부 Primary에서 일어난다

- 1 Standby 세션이 DML 실행
- 2 Standby가 Primary로 연결해 문장 전달
- 3 Primary에서 실행, 리두 생성
- 4 리두가 평소 경로로 Standby에 적용
- 5 적용 후 세션에 제어 반환

ADG_REDIRECT_DML

시스템 수준과 세션 수준

- 초기화 파라미터, 기본값 FALSE
- ALTER SESSION ENABLE ADG_REDIRECT_DML
- 세션 설정이 시스템 설정보다 우선
- 세션 수준 상태는 V$PARAMETER에 안 보임
- 도입 초기에는 세션 수준 권장

ADG_REDIRECT_PLSQL

호출을 통째로 Primary로

- ALTER SESSION ENABLE ADG_REDIRECT_PLSQL
- 세션 수준에서만 설정 가능
- 바인드 변수를 쓰는 블록은 대상 아님
- 블록 안 DML 리다이렉션과 구분: 왕복이 적음

제약과 오류 번호

16000은 미시도, 16397은 시도 후 실패

- Primary로 새 연결 후 현재 사용자로 재인증
- SYS(SYSDBA)는 재인증 불가 → ORA-16397
- Standby의 LOG_ARCHIVE_DEST_n 접속 정보 사용
- XA 분산 트랜잭션, 일반 DDL은 대상 아님
- 16397 메시지는 원인을 말해 주지 않음

쓸 곳과 피할 곳

거의 읽기만 하고 가끔 쓰는 애플리케이션

- 적합: 실행 이력 한 행, 조회 시각 갱신
- 부적합: 대량 DML, 잦은 갱신
- 문장마다 왕복·적용 대기 → Primary 직접보다 느림
- 리다이렉션이 늘면 오프로드 목적이 뒤집힘
- 갱신이 늘 것 같으면 서비스로 접속 분리

9.8

자동 블록 손상 복구와 관련 기능

- 자동 블록 손상 복구
- Standby의 In-Memory, 원격 AWR
- 그 밖의 옵션 기능

자동 블록 손상 복구

상대편의 정상 블록으로 자동 교체

- Primary 손상 → 실시간 조회 Standby에서 정상 블록
- Standby 손상 → Primary에서 받아 적용 지속
- 세션은 오류 대신 약간의 지연만 겪음
- 감지된 손상만 대상, 검사 파라미터와 병행
- 마운트 Standby라면 RMAN 블록 복구로 수동 처리

Standby의 In-Memory 열 저장소

12.2부터 지원, 두 옵션이 모두 필요

- Standby 인스턴스에 INMEMORY_SIZE 할당
- INMEMORY_ADG_ENABLED (기본 TRUE)
- Primary와 Standby에 서로 다른 객체 적재 가능
- Expression·ILM은 Primary 활동 기준
- FastStart, Join Group 미지원

Standby의 성능 진단과 원격 AWR

로컬 스냅샷은 실패, 이력은 Primary에 저장

- 로컬 AWR 스냅샷: ORA-13516
- UMF(Remote Management Framework), 12.2부터
- DBMS_UMF로 토폴로지 구성, 원격 DB 등록
- CREATE_REMOTE_SNAPSHOT으로 스냅샷
- 일상 진단은 V$ 뷰와 ASH 메모리로 충분

그 밖의 관련 기능

이름과 라이선스 위치를 알아 둔다

- Standby의 블록 변경 추적: 증분 백업 가속
- Far Sync, Real-Time Cascading (10장)
- DBMS_ROLLING 롤링 업그레이드
- Global Data Services, Application Continuity
- 설치 스위치 없음, 쓰는 순간 옵션 사용

9.9

옵션 없이 쓸 수 있는 대안

- MOUNTED Standby
- 적용을 멈추고 READ ONLY로 여는 방식
- 선택지 비교

MOUNTED Standby

보호 수준과 역할 전환 능력은 그대로

- 사용자 테이블 조회: ORA-01219
- 고정 테이블·V$ 뷰는 조회 가능
- Broker 명령 사용 가능, Real Time Query OFF
- RMAN 백업 가능, BCT만 불가
- 조회 서비스 접속은 ORA-12514

적용을 멈추고 READ ONLY로 열기

조회 창 운영

- 적용을 멈추고 열면 옵션 없이 조회
- 데이터는 멈춘 시점에 고정
- 전송은 계속되어 손실 보호는 유지
- 대가는 apply lag와 늘어나는 Failover 시간
- 재개 전 반드시 재기동해 마운트로

선택지 비교

무엇을 포기하는지 알고 고른다

- MOUNTED: 조회 불가, 감시만, 기본 기능
- READ ONLY(적용 정지): 고정 데이터, 기본 기능
- READ ONLY WITH APPLY: 초 단위 신선도, 옵션
- Snapshot Standby: 갱신 가능, 나중에 폐기 (11장)
- Logical Standby: 별도 객체 가능 (12장)

9.10

운영 시 주의점

- 조회 부하와 apply lag
- 신선도 한도와 언두 보존
- 검증 계정과 임시 파일

운영 시 주의점

조회 부하가 보호 수준을 깎지 않게

- 조회 부하가 크면 적용 자원이 줄어 apply lag 증가
- Resource Manager 또는 조회 전용 Standby 분리
- 신선도 요구가 있으면 지연 한도를 반드시 설정
- 긴 보고서는 Primary 언두 보존과 함께 조정
- 검증은 일반 사용자로, 임시 파일은 직접 확인

요약

- READ ONLY WITH APPLY가 대표적인 옵션 사용 상태
- 실시간 적용과 실시간 조회는 다른 축
- 언두와 쿼리 SCN으로 커밋된 데이터만 보임
- 영구 파일·딕셔너리를 바꾸면 Standby에서 불가
- 지연 한도 ORA-03172, zero lag는 SYNC·MaxAvail 전제
- 리다이렉션: 16000 미시도, 16397 시도 후 실패, SYS 불가

퀴즈 (1/2)

- Q1. 옵션 사용 상태는? → READ ONLY에서 적용 중
- Q2. 커밋 전 행이 Standby에 보이나? → 보이지 않는다
- Q3. STANDBY_MAX_DATA_DELAY의 특징은? → 세션 단위, 초과 시 ORA-03172

퀴즈 (2/2)

- Q4. zero lag 전제가 아닌 것은? → ADG_REDIRECT_DML
- Q5. Standby GTT INSERT가 되는 이유는? → 임시 언두
- Q6. 리다이렉션 켜고 CREATE INDEX하면? → ORA-16000

실습 개요

- 01 ADG 상태 확인 / 02 읽기 일관성 검증 (9.2, 9.3)
- 03 되는 것과 안 되는 것 (9.4, 9.6)
- 04 DML 리다이렉션 / 05 리다이렉션의 제약 (9.7)
- 06 지연 한도 / 07 지연 히스토그램 (9.5)
- 08 옵션 없는 대안 경로 (9.9)

Oracle Database 19c Active Data Guard 과정

Active Data Guard ② Far Sync와 Real-Time Cascading

멀리 있는 Standby까지 손실 없이, Primary는 가볍게

제10장

학습 목표

- 원거리 SYNC 전송이 커밋에 더하는 비용을 설명한다
- Far Sync가 가진 파일과 없는 파일을 구분한다
- Far Sync 컨트롤 파일·파라미터·SRL을 준비한다
- Primary→Far Sync SYNC, Far Sync→Terminal ASYNC 경로를 잇는다
- ALTERNATE와 GROUP·PRIORITY로 대체 경로를 설계한다
- 보호 모드와 Real-Time Cascading의 관계를 설명한다

Agenda

- 10.2 원거리 Zero Data Loss의 문제
- 10.3 Far Sync 인스턴스의 정체 / 10.4 생성
- 10.5 리두 경로 구성 / 10.6 대체 경로와 고가용성
- 10.7 보호 모드와 Far Sync
- 10.8 Cascading과 Real-Time Cascading
- 10.9 언제 무엇을 쓰는가 / 10.10 운영 시 주의점

10.2

원거리 Zero Data Loss의 문제

- SYNC 전송의 커밋 대기
- 거리가 만드는 왕복 지연
- Far Sync 이전의 선택지

SYNC 전송이 커밋에 더하는 시간

커밋마다 원격 응답을 기다린다

- LGWR은 NSS가 Standby 응답을 받아야 커밋 완료
- AFFIRM이면 SRL 디스크 기록까지 대기
- 왕복 지연 몇 ms가 처리량 저하로 번진다
- 대기 이벤트: SYNC Remote Write, log file sync

거리와 왕복 지연의 한계

대역폭을 늘려도 줄지 않는 지연

- 광케이블 속도에는 물리적 상한이 있다
- 거리가 늘면 왕복 지연도 거의 비례해 는다
- 실용 거리 변수: 쓰기 크기, 대역폭, RTT, SRL I/O
- 가까운 Standby는 지역 재해를 막지 못한다

Far Sync 이전의 선택지

둘 다 무언가를 포기했다

- 원격 ASYNC 직송: 성능은 좋지만 손실 가능
- 로컬 SYNC + 원격 ASYNC: 다중 Standby 표준형
- 다중 Standby는 Primary가 두 곳에 전송
- 로컬과 Primary를 함께 잃으면 원격은 손실

Far Sync가 바꾸는 계산

SYNC 구간과 ASYNC 구간을 나눈다

- Primary 가까이에 경량 중계 인스턴스를 둔다
- Primary → Far Sync: SYNC (짧은 왕복)
- Far Sync → 원격 Standby: ASYNC (긴 거리)
- Primary 장애 시 Far Sync 리두로 무손실 Failover

10.3

Far Sync 인스턴스의 정체

- 가진 것과 없는 것
- 역할을 구분하는 표지
- 라이선스와 배치 조건

가진 것과 없는 것

데이터베이스 없는 인스턴스

- 있음: SPFILE, Far Sync 컨트롤 파일, 패스워드 파일
- 있음: Standby Redo Log, 아카이브 로그
- 없음: 데이터 파일, 온라인 리두 사용, Redo Apply
- 항상 MOUNTED, OPEN 불가
- Primary가 될 수 없어 Failover 대상이 아니다

역할을 구분하는 표지

V$DATABASE와 V$DATAGUARD_CONFIG

- DATABASE_ROLE = FAR SYNC
- CONTROLFILE_TYPE = FARSYNC, OPEN_MODE = MOUNTED
- 적용 시도 시 ORA-01665 (컨트롤 파일 유형)
- DEST_ROLE = FAR SYNC INSTANCE, PARENT_DBUN

Far Sync 안의 리두 흐름

수신은 Standby와 같고, 출구가 다르다

- RFS가 받아 SRL에 기록
- TTnn이 SRL을 읽어 즉시 다음 목적지로 전달
- ARCn이 다 찬 SRL을 로컬 아카이브
- 원격 목적지 최대 30개, 압축 전송 오프로드

라이선스와 배치 조건

Active Data Guard 옵션 기능

- Far Sync는 Active Data Guard 옵션 기능
- 12.1부터 제공, 구성원은 같은 19c 릴리스로
- DB_NAME은 Primary와 같고 DB_UNIQUE_NAME만 다름
- 가깝되 같은 재해를 피할 만큼 떨어진 곳
- SYNC NOAFFIRM(Fast Sync)로 거리 여유 확보

10.4

Far Sync 인스턴스 생성

- SQL 방식의 흐름
- 컨트롤 파일·파라미터·SRL
- RMAN DUPLICATE FOR FARSYNC

SQL 방식의 전체 흐름

chicago, chicagoFS, boston

- Primary: Far Sync 컨트롤 파일, PFILE, 패스워드 파일
- Far Sync: 파라미터 수정, 디렉터리 생성
- 세 서버: 정적 등록, tnsnames, LOG_ARCHIVE_CONFIG
- Far Sync: STARTUP MOUNT, SRL 준비, 전달 목적지
- 마지막에 Primary 목적지를 Far Sync로 변경

Far Sync용 컨트롤 파일

CREATE FAR SYNC INSTANCE CONTROLFILE

- Primary가 MOUNT 또는 OPEN이면 실행 가능
- 파일 유형이 FARSYNC로 기록된다
- 데이터 파일·로그·SRL 정의가 그대로 담김
- 이후 Primary 구조 변경의 영향 없음

파라미터 파일

Primary PFILE에서 고칠 것

- DB_UNIQUE_NAME=chicagoFS, CONTROL_FILES
- LOG_ARCHIVE_CONFIG에 세 구성원
- DEST_2: SERVICE=boston ASYNC
- VALID_FOR=(STANDBY_LOGFILES,STANDBY_ROLE)
- FAL_SERVER=chicago, LOG_FILE_NAME_CONVERT

Standby Redo Log 준비

Far Sync의 입구이자 출구

- 크기는 Primary 최대 온라인 로그 이상
- 개수는 스레드별 온라인 그룹 수 + 1
- 정의가 따라왔으면 ADD는 ORA-01184
- 파일이 없으면 CLEAR LOGFILE로 생성
- Terminal Standby에도 SRL 필수

Oracle Net과 기동

STARTUP MOUNT 한 번

- 리스너 정적 등록(Broker용 _DGMGRL 포함)
- 세 서버의 tnsnames.ora에 세 별칭
- NOMOUNT 단계 없이 바로 MOUNT
- 기동 후 DATABASE_ROLE로 역할 확인

받는 쪽의 명단을 먼저 넓힌다

ORA-16057

- LOG_ARCHIVE_CONFIG는 구성원 명단
- 명단에 없으면 송신 쪽에 ORA-16057
- 오류는 보내는 쪽, 원인은 받는 쪽
- 세 곳을 넓힌 뒤 Primary 목적지 변경
- 남은 오류는 DEFER 후 ENABLE로 재시도

RMAN DUPLICATE FOR FARSYNC

12.2부터 지원

- FOR STANDBY 자리에 FOR FARSYNC
- 액티브 복제: 컨트롤 파일과 SPFILE만 복사
- 백업 기반 복제 가능, DORECOVER는 불가
- SPFILE SET으로 DB_UNIQUE_NAME 변경 필수
- Primary에 SRL이 있으면 자동 생성

10.5

리두 경로 구성

- Primary → Far Sync: SYNC
- Far Sync → Terminal: ASYNC
- 대칭 구성과 경로 확인

Primary에서 Far Sync까지

SYNC AFFIRM

- Far Sync는 Primary에게 평범한 목적지 하나
- DEST_2를 boston에서 chicagoFS로 변경
- boston은 이제 Far Sync를 통해서만 수신
- NOAFFIRM이면 메모리 도착 시점에 응답
- TRANSMIT_MODE가 PARALLELSYNC로 보일 수 있음

Far Sync에서 Terminal까지

ASYNC로 원거리 구간을 흘린다

- TTnn은 boston 응답을 기다리지 않는다
- 원거리 지연이 Primary 커밋에서 빠진다
- boston의 lag는 Primary 기준으로 계산
- Terminal 추가는 DEST_3, DEST_4로 확장

Terminal Standby 쪽 설정

boston에서 바꿀 것

- LOG_ARCHIVE_CONFIG에 chicagoFS 추가
- FAL_SERVER='chicagoFS','chicago'
- Redo Apply·실시간 적용 설정은 그대로
- RFS 상대가 chicagoFS로 바뀐다

역할 전환에 대비한 대칭 구성

bostonFS를 미리 둔다

- boston이 Primary가 될 때를 위한 bostonFS
- boston에 PRIMARY_ROLE 목적지로 정의
- Standby 동안은 쉬다가 전환 후 활성화
- 대칭이 없으면 전환 후 보호 수준이 내려감

경로를 확인하는 뷰

어느 구성원에서 무엇을 보는가

- V$DATAGUARD_CONFIG: PARENT_DBUN으로 경로 트리
- V$ARCHIVE_DEST: STATUS와 VALID_NOW
- V$ARCHIVE_DEST_STATUS: ERROR, GAP_STATUS
- V$DATAGUARD_PROCESS: RFS·TT 연결 상대
- Terminal의 V$DATAGUARD_STATS: 최종 지연

10.6

대체 경로와 Far Sync 고가용성

- 단일 장애점 문제
- ALTERNATE 속성
- 19c의 GROUP과 PRIORITY

Far Sync가 단일 장애점이 되는 문제

한 대가 경로 전체를 쥔다

- Far Sync가 멈추면 Terminal 전송이 끊긴다
- Primary는 NET_TIMEOUT 후 대기를 포기
- 업무는 계속, 보호 수준은 하락
- 복구 전 Primary까지 잃으면 빈 구간 발생

ALTERNATE 속성

공식 교재의 대체 경로 예

- DEST_2: chicagoFS SYNC, MAX_FAILURE=1, ALTERNATE=DEST_3
- DEST_3: boston ASYNC 직송, ALTERNATE=DEST_2
- DEST_STATE_3='ALTERNATE'로 평소 휴면
- 12.2부터 Far Sync 복구 시 자동 복귀

19c의 GROUP과 PRIORITY

우선순위로 대체 경로를 표현

- 목적지를 GROUP으로 묶고 PRIORITY 1~8 부여
- 숫자가 작을수록 우선
- 상위 순위가 모두 실패하면 다음 순위로
- 상위 순위가 돌아오면 자동 복귀
- GROUP과 ALTERNATE는 함께 쓰지 않음

고가용성 설계 패턴

Far Sync 장애 때 무엇을 지키는가

- Terminal 직송 폴백: 전송 유지, 무손실은 중단
- Far Sync 이중화: 두 번째가 SYNC 승계
- RAC Far Sync: 인스턴스 장애 흡수
- 예비 Far Sync는 다른 건물·전원 계통에

대체 경로는 장애를 일으켜 시험한다

설정이 받아들여졌다고 넘어가는 것은 아니다

- 대체 경로 설정은 오류 없이 받아들여진다
- Far Sync를 멈추고 활성 목적지 변화를 확인
- 복구 후 원래 경로 복귀와 갭 확인
- 시험 결과를 운영 절차서에 기록

10.7

보호 모드와 Far Sync

- Maximum Availability
- Maximum Performance
- Maximum Protection 미지원

Maximum Availability와 Far Sync

동기 지점이 Far Sync로 옮겨 간다

- Primary → Far Sync SYNC, 모드는 MaxAvailability
- Primary만 잃으면 Terminal로 무손실 Failover
- 열린 채로 MAXIMIZE AVAILABILITY 실행
- 상향 직후 잠시 RESYNCHRONIZATION

Maximum Performance와 Far Sync

무손실 대신 분배를 맡는다

- Primary → Far Sync도 ASYNC
- Far Sync를 원격 지역 한가운데에 둘 수 있다
- Primary는 목적지 하나만 상대
- 대륙 간 회선에는 리두가 한 번만 지나감

Maximum Protection과 Far Sync

지원하지 않는 조합

- Far Sync는 Maximum Protection 미지원
- 이 모드는 SYNC 실패 시 Primary를 멈춘다
- SYNC 상대는 데이터를 가진 Standby
- 무손실 + 업무 연속성: MaxAvailability + Far Sync

Far Sync 구성에서의 Failover

마지막 리두까지 넘겨받는다

- Far Sync가 남은 리두를 Terminal로 마저 전송
- 도착 확인 후 FAILOVER TO boston
- Broker는 이 마무리 전송을 확인해 줌
- FSFO: Primary→Far Sync SYNC, Far Sync→대상 ASYNC

구성원마다 다른 PROTECTION_LEVEL

MODE는 목표, LEVEL은 관점

- PROTECTION_MODE는 모든 구성원에 같은 값
- PROTECTION_LEVEL은 구성원별 달성 수준
- Terminal만 RESYNCHRONIZATION일 수 있다
- 판단 기준은 Primary의 LEVEL과 Far Sync 목적지

10.8

Cascading과 Real-Time Cascading

- Cascading Standby
- 전통 방식과 실시간 방식
- 구성 방법과 제약

Cascading Standby

데이터를 가진 Standby가 전달까지 맡는다

- Primary → cascading Standby → cascaded Standby
- 전달할 수 있는 것은 물리 Standby와 Far Sync
- 논리·Snapshot Standby는 전달 불가
- 조회·백업·전환 대상까지 겸하는 대신 자원이 든다

전통적인 cascading

로그 전환을 기다린 뒤 전달

- SRL이 아카이브된 뒤에야 다음 Standby로
- 전달 주기는 Primary 로그 전환 주기와 같다
- 한가한 시간대에는 몇 시간 지연도 가능
- 강제 로그 전환으로 주기를 맞추던 운영

Real-Time Cascading

SRL에 쓰는 대로 바로 넘긴다

- 12.1부터 제공
- 로그 전환 대기가 사라져 지연이 크게 줄어듦
- Active Data Guard 옵션 기능
- 전통 방식(아카이브 후 전달)은 기본 기능
- Primary 전송량이 절반으로 준다

Cascading 구성 방법

전달 목적지는 cascading Standby에

- SERVICE=london ASYNC
- VALID_FOR=(STANDBY_LOGFILES,STANDBY_ROLE)
- cascading Standby에 SRL 필수
- london의 FAL_SERVER는 boston 먼저
- 세 구성원 LOG_ARCHIVE_CONFIG에 london

사례 — 오류 없이 멈춰 있던 cascading

STATUS는 VALID였다

- 로컬 Standby에서 해외 DR로 전달 구성
- 변경은 성공, STATUS도 VALID
- DR 센터 transport lag가 계속 증가
- VALID_NOW = WRONG VALID_TYPE
- VALID_FOR 수정 후 UNKNOWN을 거쳐 YES

10.9

언제 무엇을 쓰는가

- 구성별 비교
- 판단 기준과 사례

구성별 비교

원격 무손실, Primary 부담, 비용

- SYNC 직송: 무손실, 원거리 왕복 지연
- ASYNC 직송: 가볍지만 손실 가능
- 로컬 SYNC + 원격 ASYNC: 목적지 두 곳
- 로컬 + Real-Time Cascading: 목적지 한 곳, ADG
- Far Sync: 원격 무손실, 가까운 왕복만, ADG

판단 기준

질문 순서대로 고른다

- 원격지에도 무손실이 필요한가
- Primary가 원격 왕복 지연을 견디는가
- 로컬에 조회·백업용 Standby가 필요한가
- 원격 목적지가 여러 곳인가
- Far Sync 장애 때 무엇을 지킬 것인가

사례 — 서울 본사와 해외 재해 복구 센터

Far Sync 두 대와 직송 예비

- 해외 센터 SYNC 직송: 커밋 지연 허용치 초과
- 서울 안 다른 건물에 Far Sync, SYNC AFFIRM
- 해외 센터는 Far Sync에서 ASYNC로 수신
- 두 번째 Far Sync를 같은 우선순위로 추가
- 해외 직송을 최하위 우선순위로 예비

10.10

운영 시 주의점

- 아카이브 정리, SRL 재구성
- 설정 순서와 구간별 지연 확인

운영 시 주의점

조용히 무너지는 지점들

- Far Sync 아카이브도 RMAN 삭제 정책으로 정리
- Primary 로그 변경 시 Far Sync·Terminal SRL 재구성
- 받는 쪽 LOG_ARCHIVE_CONFIG부터 넓힌다
- lag 증가 시 두 구간으로 나눠 진단
- SCOPE=MEMORY 시험 후 SPFILE 반영

요약

- SYNC는 커밋마다 원격 왕복 지연을 더한다
- Far Sync: 컨트롤 파일·SRL·아카이브만, 데이터 파일 없음, ADG
- 생성: CREATE FAR SYNC INSTANCE CONTROLFILE 또는 DUPLICATE FOR FARSYNC
- 경로: Primary→Far Sync SYNC, Far Sync→Terminal ASYNC
- 대체 경로: ALTERNATE 또는 GROUP·PRIORITY, 자동 복귀
- MaxProtection 미지원 / Real-Time Cascading은 ADG

퀴즈 (1/2)

- Q1. Far Sync에 대해 옳은 것은? → SYNC 수신, RMAN DUPLICATE 생성
- Q2. Far Sync 전달 목적지의 VALID_FOR는? → (STANDBY_LOGFILES,STANDBY_ROLE)
- Q3. Far Sync를 쓸 수 없는 보호 모드는? → Maximum Protection

퀴즈 (2/2)

- Q4. MAX_FAILURE=1 ALTERNATE 구성에서 Far Sync 장애 시? → DEST_3로 전환
- Q5. GROUP·PRIORITY에서 틀린 것은? → 상위 순위 복구 시 수동 복귀
- Q6. Real-Time Cascading의 특징은? → SRL 기록 즉시 전달

실습 개요

- 01 Far Sync 노드 준비 / 02 기동과 역할 확인 (10.3, 10.4)
- 03 SRL 실체화: ORA-01184, CLEAR LOGFILE (10.4.4)
- 04 Oracle Net 구성: 정적 등록 (10.4.5)
- 05 리두 경로 전환: ORA-16057, 변경 순서 (10.5)
- 06 보호 모드 상향: PROTECTION_LEVEL, ORA-01665 (10.7)
- 07 Cascading과 원복: VALID_FOR, WRONG VALID_TYPE (10.8)

Oracle Database 19c Active Data Guard 과정

Snapshot Standby

운영 데이터를 잠시 빌려 쓰고 그대로 돌려놓기

제11장

학습 목표

- Snapshot Standby가 필요한 상황과 RAT 연계를 설명한다
- 보장 복원 지점과 Flashback의 역할을 설명한다
- 받기는 계속, 적용만 멈추는 구조를 설명한다
- FRA·Flashback 로그 공간을 계획하고 감시한다
- 전환·열기·되돌리기와 순서 오류를 해석한다
- 세 가지 활용 방식 중 요구에 맞는 것을 고른다

Agenda

- 11.2 Snapshot Standby가 필요한 상황
- 11.3 동작 원리
- 11.4 전제 조건과 공간 계획
- 11.5 전환과 되돌리기 명령
- 11.6 제약과 위험
- 11.7 세 가지 방식 비교와 선택 / 11.8 운영 시 주의점

11.2

Snapshot Standby가 필요한 상황

- 운영 크기의 데이터로 시험해야 하는 이유
- Real Application Testing과의 연계
- 물리 Standby만 전환 대상

운영과 같은 데이터로 시험해야 하는 이유

검증 DB에서 빠르던 SQL이 운영에서 느린 까닭

- 실행 계획은 데이터 양과 분포에 따라 바뀜
- 복제본 생성은 수 TB면 몇 시간, 회수도 필요
- 물리 Standby는 운영 크기 데이터를 이미 보유
- 전환과 되돌리기가 각각 명령 한 줄
- Standby 서버 사양 차이는 결과 해석 때 반영

Real Application Testing과의 연계

캡처 직전 전환으로 시작 시점 데이터를 고정

- RAT는 별도 라이선스 옵션
- Database Replay: Primary 부하 캡처 후 재생
- 캡처 시작 직전에 Snapshot으로 전환
- SPA: SQL Tuning Set을 변경 전후로 비교
- 재생 클라이언트(wrc)는 별도 호스트에서 접속

어떤 Standby를 바꿀 수 있는가

물리 Standby 전용, ADG 옵션 불필요

- 전환 대상은 물리 Standby뿐
- 논리 Standby, Far Sync는 불가
- 구성원 자격 유지, 리두도 계속 수신
- 여러 Standby 중 하나만 돌리는 배치가 일반적
- EE의 Data Guard 기본 기능

이름이 비슷한 다른 기능들

스토리지 스냅샷, PDB 스냅샷 복제와 구분

- 스토리지 스냅샷: 장비 수준 블록 복사
- PDB 스냅샷 복제: PDB 하나를 쓰기 시 복사
- Snapshot Standby: DB 전체, Flashback으로 복귀
- 시험 후 남는 복사본이 없음

11.3

동작 원리

- 보장 복원 지점과 Flashback Database
- 받기는 계속, 적용만 멈춤
- 되돌릴 때의 네 단계

보장 복원 지점

되돌아갈 SCN에 붙인 이름표

- 이름: SNAPSHOT_STANDBY_REQUIRED_ + 시각
- 그 SCN까지 Flashback 로그를 지우지 않고 보존
- DB_FLASHBACK_RETENTION_TARGET과 무관
- 되돌린 뒤 자동 삭제, 직접 지울 대상 아님
- 이름을 틀리면 ORA-38780, 아무 일 없음

Flashback이 꺼져 있어도 된다

RESTORE POINT ONLY 로깅

- 전환 전 FLASHBACK_ON = NO여도 전환 가능
- 전환 후 FLASHBACK_ON = RESTORE POINT ONLY
- 복원 지점 이후 처음 바뀐 블록만 기록
- 공간은 변경 횟수보다 건드린 블록 수에 비례
- 되감기 시간은 DB 크기 아닌 변경량에 비례

받기는 계속, 적용만 멈춤

전송 서비스는 아무것도 바뀌지 않음

- RFS 수신, SRL 기록, 아카이브는 그대로
- MRP0가 없어 데이터 파일 반영은 멈춤
- MRP 기동 시 ORA-01665
- 자체 변경은 자기 온라인 리두 로그에 기록
- Primary가 사라져도 받은 리두는 남아 있음

두 지연이 갈라진다

transport lag는 그대로, apply lag만 증가

- transport lag: 평소처럼 수 초 이내
- apply lag: 전환 후 흐른 시간만큼 증가
- V$ARCHIVED_LOG의 APPLIED는 NO로 쌓임
- 7장에서 나눈 두 지표가 가장 뚜렷이 갈림

되돌릴 때 일어나는 일

Flashback, 역할 복귀, 적용 재개, 실시간 복귀

- 1) 복원 지점 SCN으로 Flashback, 로컬 변경 폐기
- 2) 컨트롤 파일 STANDBY, 역할 복귀, 복원 지점 삭제
- 3) 쌓인 아카이브 로그를 순서대로 적용
- 4) 따라잡은 뒤 실시간 적용으로 복귀
- 남길 결과는 되돌리기 전에 추출

전환을 반복할 때

매번 새 복원 지점, 새 출발점

- 전환할 때마다 새 보장 복원 지점 생성
- 출발점은 그 시점까지 적용된 Primary 데이터
- 이전 시험의 객체는 남지 않음
- 같은 시험 반복은 사용자 복원 지점이 빠름

상태 값의 변화

V$DATABASE 네 열로 단계 판별

- DATABASE_ROLE: PHYSICAL ↔ SNAPSHOT STANDBY
- CONTROLFILE_TYPE: STANDBY ↔ CURRENT
- FLASHBACK_ON: NO ↔ RESTORE POINT ONLY
- OPEN_MODE: MOUNTED → READ WRITE
- V$RESTORE_POINT 행 유무

11.4

전제 조건과 공간 계획

- 전환 명령의 조건은 단순
- 사고는 FRA 공간에서 시작
- 감시 기준과 점검 순서

전환 전에 갖춰야 할 조건

FRA, 적용 중지, MOUNT

- FRA 구성: DB_RECOVERY_FILE_DEST, _SIZE
- Redo Apply 중지 (아니면 ORA-38784)
- MOUNT 상태 (ADG였다면 재기동)
- RAC는 인스턴스 하나만 남기는 편이 안전
- Primary 쪽 설정은 손대지 않음

FRA를 함께 쓰는 두 파일

Flashback 로그 + 적용 못 한 아카이브 로그

- Flashback 로그: 시험이 건드린 블록 수에 비례
- 행 하나를 고쳐도 블록 전체 이미지
- 아카이브 로그: 리두 생성량 × 전환 기간
- 하루 200GB × 3일 = 600GB
- 적용 전 로그는 삭제 대상이 아님

삭제 정책과 Primary FRA

APPLIED ON ALL STANDBY의 영향

- 적용 기준 정책이면 Primary에서도 보존
- Primary와 Standby FRA가 함께 차오름
- 시험 기간에만 정책을 푸는 것은 위험
- 지워진 로그는 되돌린 뒤 갭이 됨

FRA가 가득 차면

시험 DB의 공간 문제가 리두 보관까지 막음

- 보장 복원 지점의 로그는 자동 삭제 불가
- Snapshot 쪽 쓰기 정지
- 리두 아카이브 실패로 Primary 목적지 오류
- 해결: FRA 확장 또는 시험 중단 후 되돌리기
- 복원 지점을 지우면 되돌릴 길이 사라짐

공간 감시

FRA 전체 사용률을 기준으로

- V$RECOVERY_FILE_DEST: 전체 한도와 사용량
- V$RECOVERY_AREA_USAGE: 유형별 비율
- V$RESTORE_POINT.STORAGE_SIZE: 복원 지점 몫
- 85%, 97%에서 alert log 경고
- 80% 같은 중단 기준을 미리 정함

전환 전 점검 순서

명령 성공 조건과 전환 후 위험을 함께

- 1) 역할이 PHYSICAL STANDBY인가
- 2) FRA 경로·크기, 3) 여유 공간
- 4) 적용 중지, 5) MOUNTED
- 6) 재해 대비를 맡을 다른 Standby가 있는가
- apply lag가 충분히 작은지도 확인

11.5

전환과 되돌리기 명령

- 전환, 열기, 되돌리기
- 순서를 어기면 나는 오류들
- Broker의 CONVERT DATABASE

Snapshot Standby로 전환

CANCEL, CONVERT, OPEN

- RECOVER MANAGED STANDBY DATABASE CANCEL
- ALTER DATABASE CONVERT TO SNAPSHOT STANDBY
- 명령 직후 MOUNT 상태로 남음
- ALTER DATABASE OPEN으로 읽기·쓰기
- 적용 중 전환은 ORA-38784 + ORA-01153

열기와 PDB

CDB를 열어도 PDB는 MOUNTED

- OPEN 순간부터 Primary와 갈라짐
- PDB는 ALL OPEN으로 따로 열기
- 열기 전 PDB 조회는 ORA-01219
- DDL·DML 모두 가능, ORA-16000 없음
- SCN은 시험 작업에 따라 독자적으로 진행

열려 있는 동안

자동 작업과 서비스 점검

- 자동 통계 수집이 계획을 바꿀 수 있음
- 비교 시험이면 DBMS_AUTO_TASK_ADMIN으로 중지
- 스케줄러 잡은 JOB_QUEUE_PROCESSES로 차단
- 역할 기반 서비스로 운영 접속 차단

Physical Standby로 되돌리기

MOUNT에서 CONVERT TO PHYSICAL STANDBY

- SHUTDOWN IMMEDIATE, STARTUP MOUNT
- ALTER DATABASE CONVERT TO PHYSICAL STANDBY
- 열린 상태면 ORA-01126
- 한 번도 열지 않았으면 ORA-16433
- RAC는 인스턴스 하나만 남김

ADG로 복귀하는 순서

열고 나서 적용

- STARTUP MOUNT 후 OPEN READ ONLY
- 그다음 RECOVER MANAGED STANDBY DATABASE
- 적용을 먼저 기동하면 OPEN에서 ORA-10456
- ADG를 안 쓰면 MOUNT에서 적용만 기동
- apply lag가 줄 때까지 Failover 준비 미완

전환 기간 중 상태 확인

수신은 정상인지, 목적지는 VALID인지

- Standby: V$DATAGUARD_STATS의 두 lag
- Standby: V$ARCHIVED_LOG 시퀀스 증가
- Primary: V$ARCHIVE_DEST_STATUS가 VALID
- ERROR가 보이면 FRA 부족부터 의심

자주 만나는 오류

대부분 상태와 명령의 불일치

- 38784+01153: 적용 중 전환
- 01665: Snapshot에서 MRP 기동
- 01219: 닫힌 PDB 조회
- 16433: 안 열고 되돌리기 / 01126: 열린 채 되돌리기
- 10456: 적용 먼저, 열기 나중

Broker에서의 전환

CONVERT DATABASE 한 줄

- CONVERT DATABASE 'boston' TO SNAPSHOT STANDBY
- CONVERT DATABASE 'boston' TO PHYSICAL STANDBY
- 적용 중지·재기동·적용 재개를 Broker가 처리
- Broker 구성에서는 SQL로 바꾸지 않음
- SHOW CONFIGURATION에 Snapshot standby 표시

11.6

제약과 위험

- 역할 전환과 FSFO 제약
- 보호 모드 영향과 RTO 증가
- 되돌리기를 막는 작업

역할 전환과 Fast-Start Failover

Switchover와 FSFO 대상 불가

- Switchover 대상이 될 수 없음
- Fast-Start Failover 대상이 될 수 없음
- Failover는 되돌리기와 적용을 거친 뒤 가능
- Broker는 Failover 안에서 이 단계를 처리
- 다른 물리 Standby가 있으면 그쪽이 대상

보호 모드에 미치는 영향

최대 보호에서는 유일한 Standby 불가

- 최대 보호: 유일한 Standby로 전환 불가
- 그런 상황을 만드는 구성 변경도 거부
- 최대 가용성·성능: 전환 제약 없음
- SYNC 대상이면 수신 확인은 계속 보냄
- 손실 보호는 유지, 복구 시간은 늘어남

지연된 적용과 RTO

되감기 + 밀린 적용 + 역할 전환

- 되감기: 시험이 건드린 블록 양에 비례
- 적용: 전환 기간 리두 양에 비례
- 사흘 Snapshot이면 사흘 치 리두 적용
- 적용 속도가 3배여도 하루 가까이 소요
- RTO가 분 단위라면 재해 대비 역할 불가

손상 로그 위험

적용할 때에야 발견되는 손상

- 받은 로그는 적용 시점에 검증됨
- 손상은 되돌린 뒤 MRP가 읽을 때 발견
- Primary가 없으면 다시 받을 방법 없음
- Flashback 로그 손상 시 되돌리기 실패 가능
- 대책: Primary 로그 보존, 다른 물리 Standby

되돌리기를 막는 작업

Flashback이 넘지 못하는 선

- 컨트롤 파일 재생성·복원
- 데이터 파일 축소(RESIZE로 줄이기)
- OPEN RESETLOGS, Flashback 로그 수동 삭제
- 실패 시 Standby를 새로 생성(5·6장)
- 시험 계정에 ALTER DATABASE·SYSTEM 권한 금지

11.7

세 가지 방식 비교와 선택

- Snapshot Standby
- Real-Time Query (ADG)
- Logical Standby

세 방식 비교

쓰기, 데이터 시점, 전환 대상, 라이선스

- Snapshot: 전부 쓰기, 전환 시점 고정, EE 기본
- Real-Time Query: 읽기만, 최신, ADG 옵션
- Logical: 일부 객체 쓰기·유지, 최신, 유형 제약
- Switchover 대상: Snapshot만 불가
- DML 리다이렉션은 Primary에 기록

선택 기준

바꾸는가, 남기는가, 얼마나 오래인가

- 바꾸고 버려도 됨 → Snapshot Standby
- 최신 데이터 읽기만 → Real-Time Query
- 추가 객체를 계속 유지 → Logical Standby
- 장기 개발 환경 → RMAN 복제·PDB 클론
- 번갈아 쓰면 전환마다 조회 서비스 중단

역할을 나눠 둔 구성

재해 대비, 조회, 시험을 분리

- Standby A: SYNC·FASTSYNC, FSFO 대상
- Standby B: 평소 Real-Time Query
- 시험 기간에만 B를 Snapshot으로 전환
- 재해 대비 수준이 시험 중에도 유지
- 최대 보호의 유일한 Standby 제약도 회피

11.8

운영 시 주의점

- 전환해 둔 사실을 잊는 사고
- 되돌려도 취소되지 않는 외부 작업
- 되돌린 뒤의 검증

전환해 둔 사실을 잊지 않기

명령보다 기억이 사고 원인

- 감시 목록에 Snapshot 상태를 별도 표시
- 전환 기간 상한, FRA 사용률 상한 명시
- 담당자와 되돌릴 시각을 운영 일지에 기록
- 시각이 지나면 경보를 상위 단계로
- 반복 시험은 사용자 복원 지점 활용

되돌려도 취소되지 않는 일

DB 밖으로 나간 것은 돌아오지 않음

- DB 링크 INSERT, 메일 발송, 외부 인터페이스
- 시험 전에 잡과 링크를 막아 둠
- AWR·재생 보고서는 되돌리기 전에 추출
- 운영 데이터이므로 접근 통제는 운영 수준
- 되돌린 뒤 역할·복원 지점·lag 순서로 확인

요약

- 물리 Standby를 임시로 읽기·쓰기, 끝나면 변경 폐기
- 보장 복원 지점 자동 생성, RESTORE POINT ONLY 로깅
- 수신·아카이브 계속, 적용만 멈춤 → apply lag 증가
- 전제: FRA, 적용 중지, MOUNT / 공간은 두 파일이 함께
- 되돌리기: 한 번 연 뒤(16433), MOUNT에서(01126)
- Switchover·FSFO 불가, 최대 보호 유일 Standby 불가

퀴즈 (1/2)

- Q1. 논리 Standby로 Snapshot Standby를 만들 수 있나? → 아니다
- Q2. 적용 중 전환하면? → ORA-38784 + ORA-01153
- Q3. 전환 기간의 Primary 리두는? → 수신·아카이브, 되돌린 뒤 적용

퀴즈 (2/2)

- Q4. FLASHBACK_ON = NO면 전환 불가? → FRA만 있으면 가능
- Q5. 장애 시 Primary가 될 수 없다? → 되돌리고 적용하면 가능
- Q6. ADG 복귀 순서는? → OPEN READ ONLY 후 적용 기동

실습 개요

- 01 전제 조건 확인 / 02 전환 실패와 성공 (11.4, 11.5.1)
- 03 복원 지점 읽기 (11.3.1)
- 04 열고 쓰기, PDB와 ORA-01219 (11.5.2)
- 05 전환 중의 관찰, ORA-01665와 FRA (11.3.2, 11.5.4)
- 06 되돌리기와 ORA-10456 (11.5.3)
- 07 검증과 정리 (11.3.3, 11.8)

Oracle Database 19c Active Data Guard 과정

Logical Standby와 SQL Apply

리두를 SQL로 되살려 적용하는 열린 Standby

제12장

학습 목표

- Logical Standby의 용도와 Physical과의 차이를 비교한다
- SQL Apply의 단계별 프로세스와 LCR 흐름을 설명한다
- 미지원 객체와 행 식별 문제를 전환 전에 점검한다
- Physical을 거쳐 Logical로 전환하는 절차를 수행한다
- DATABASE GUARD와 SKIP 규칙으로 복제를 통제한다
- APPLY_SET으로 적용 성능을 조정하고 롤링 업그레이드와 연결한다

Agenda

- 12.2 Logical Standby의 용도와 Physical과의 차이
- 12.3 SQL Apply 아키텍처 / 12.4 지원 범위 점검
- 12.5 Logical Standby 생성 절차
- 12.6 SQL Apply 기동·중지와 실시간 적용 / 12.7 DATABASE GUARD
- 12.8 선택적 복제와 필터링 / 12.9 SQL Apply 성능 조정
- 12.10 롤링 업그레이드와의 관계 / 12.11 운영 시 주의점

12.2

Logical Standby의 용도와 Physical과의 차이

- 열린 채로 갱신되는 Standby
- 보고 분산, 추가 구조, 손상 차단
- 고르는 기준과 라이선스

열린 채로 갱신되는 Standby

같은 행과 값, 다른 블록

- 받은 리두를 SQL로 되살려 실행 (SQL Apply)
- READ WRITE로 열린 채 복제 테이블이 갱신됨
- 블록 배치와 ROWID는 Primary와 다름
- Standby에만 인덱스, MV, 보고용 스키마 추가 가능
- 미지원 유형이 있고 적용 비용이 큼

Logical Standby가 주는 이점

보고 분산부터 업그레이드까지

- 월말 집계, 대량 조회를 Primary에서 덜어 냄
- 보고에 맞춘 인덱스·요약 테이블을 Standby에만
- Primary의 블록 손상이 Standby로 번지지 않음
- Switchover·Failover로 재해 복구 역할 수행
- 버전이 다른 DB 사이에서도 동작해 롤링 업그레이드에 쓰임

Physical과의 비교

적용 방식이 나머지 차이를 만든다

- 적용: Redo Apply(블록) 대 SQL Apply(SQL)
- 열린 모드: MOUNT·READ ONLY 대 READ WRITE
- 데이터 유형 제약: 없음 대 일부 제외
- 컨트롤 파일: STANDBY 대 CURRENT
- 주된 쓰임: 재해 복구 대 보고 DB, 롤링 업그레이드

언제 Logical Standby를 고르는가

요구와 제약을 같이 본다

- 쓰기 가능한 보고 공간이 필요하고 약간의 지연이 허용될 때
- 중요 테이블이 미지원 유형이면 고르지 않음
- 조회만 필요하면 ADG 실시간 조회가 더 단순
- Logical Standby 자체는 EE 기본 기능
- DBMS_ROLLING은 ADG 옵션 필요

12.3

SQL Apply 아키텍처

- LogMiner와 LogMiner 딕셔너리
- 마이닝 단계와 적용 단계
- 커밋 순서와 일관성

리두를 SQL로 바꾸는 LogMiner

리두에는 이름이 없다

- 리두에는 객체 번호, 블록, 슬롯 같은 물리 정보만 있음
- 이름과 번호를 잇는 사전이 LogMiner 딕셔너리
- Primary가 딕셔너리를 리두에 실어 보냄
- Standby ROWID가 달라 키 값으로 행을 찾음

로그 마이닝 단계

Reader, Preparer, Builder

- Reader: SRL·아카이브 로그에서 리두를 읽음
- Preparer: 블록 변경을 LCR로 변환, 병렬 가능
- Builder: 낱개 LCR을 트랜잭션으로 조립
- 메모리가 모자라면 LCR을 디스크로 spill

적용 단계

Analyzer, Coordinator, Applier

- Analyzer: 트랜잭션 사이 의존 관계 계산
- Coordinator(LSP0): 배분과 커밋 순서 승인
- Applier: LCR을 SQL로 실행하고 커밋, 병렬
- V$LOGSTDBY_PROCESS의 TYPE으로 확인

커밋 순서와 일관성

PRESERVE_COMMIT_ORDER

- 기본: Primary와 같은 순서로 커밋 (TRUE)
- 조회 사용자는 실제 있었던 상태만 봄
- FALSE: 처리량은 오르나 중간 상태가 보일 수 있음
- 블록을 되돌리는 물리 Standby에는 없는 문제

12.4

지원 범위 점검

- 내부 스키마와 미지원 유형
- 넘어가지 않는 SQL과 패키지
- 행 식별과 보충 로깅

내부 스키마는 자동으로 빠진다

DBA_LOGSTDBY_SKIP의 INTERNAL SCHEMA

- SYS, SYSTEM, XDB 등은 기본 SKIP 규칙에 포함
- 여기 만든 사용자 테이블은 복제되지 않음
- 미지원 목록에도 나오지 않아 놓치기 쉬움
- CDB 루트와 각 PDB에서 따로 조회

지원되지 않는 유형과 객체

오류 없이 테이블 전체가 빠진다

- BFILE, ROWID·UROWID, VARRAY·중첩 테이블
- REF·중첩 테이블을 가진 객체, 일부 Spatial 유형
- 전역 임시 테이블, MV 지원 테이블, SYS 객체
- DBA_LOGSTDBY_UNSUPPORTED로 열 단위 원인 확인
- 중요 테이블이 있으면 물리 Standby 검토

Standby로 넘어가지 않는 SQL과 패키지

Standby에서 따로 실행해야 하는 작업

- ALTER SYSTEM, ALTER SESSION, ALTER DATABASE
- CREATE·DROP DATABASE LINK, MV와 MV LOG
- LOCK TABLE, SET ROLE, SET TRANSACTION, PURGE
- 메타데이터를 바꾸는 패키지: DBMS_JAVA, DBMS_AQ 등
- 롤링 업그레이드 전용 지원: DBA_LOGSTDBY_PLSQL_SUPPORT

행 식별과 DBA_LOGSTDBY_NOT_UNIQUE

ROWID 대신 키로 행을 찾는다

- 기본 키나 NOT NULL 고유 인덱스로 행을 찾음
- 키가 없으면 모든 열 비교, 전체 스캔 발생
- BAD_COLUMN = Y: LOB 등 때문에 구별 불가, 키 필수
- BAD_COLUMN = N: 동작은 하나 비효율, 키 권장

RELY 제약과 보충 로깅

키 열 값을 리두에 남기는 장치

- RELY DISABLE 기본 키: 검사·인덱스 유지 비용 없음
- 고른 열이 실제로 유일하지 않으면 적용 실패
- Standby에는 같은 열에 고유 인덱스를 따로 생성
- 보충 로깅: 변경과 함께 키 열 값을 기록
- BUILD가 PK·UI 보충 로깅을 자동으로 켬

전환 전 점검 순서

모두 Primary에서, 컨테이너마다

- 내부 스키마에 사용자 테이블이 없는지
- 미지원 테이블 목록을 업무 담당자와 검토
- BAD_COLUMN = Y 테이블부터 키 정비
- ARCHIVELOG, FORCE LOGGING, DB_NAME 길이 확인
- 원복 수단(스냅샷, 재생성 절차) 준비

12.5

Logical Standby 생성 절차

- Physical Standby에서 출발
- BUILD와 RECOVER TO LOGICAL STANDBY
- 열기와 첫 확인

생성 절차 일곱 단계

Physical Standby를 거쳐 간다

- 1 물리 Standby 생성과 동기화
- 2 Redo Apply 중지 3 Primary 목적지 준비(선택)
- 4 Primary에서 DBMS_LOGSTDBY.BUILD
- 5 RECOVER TO LOGICAL STANDBY, MOUNT 재기동
- 6 OPEN RESETLOGS, SQL Apply 기동 7 확인

Redo Apply 중지와 목적지 분리

딕셔너리를 지나치지 않게

- BUILD 전에 MRP를 멈춰 딕셔너리 위치를 지킴
- Broker 구성은 STATE='APPLY-OFF'
- 논리 Standby는 자기 리두를 직접 생성
- 자기 Online Redo Log용과 받은 SRL용 목적지 분리
- FRA를 쓰면 두 종류를 자동으로 구분

LogMiner 딕셔너리 빌드

EXECUTE DBMS_LOGSTDBY.BUILD

- Primary 딕셔너리를 리두에 기록
- PK·고유 인덱스 보충 로깅 자동 활성화
- 진행 중인 트랜잭션이 끝날 때까지 대기
- Standby의 DBA_LOGSTDBY_LOG에 DICT_BEGIN·DICT_END로 표시

RECOVER TO LOGICAL STANDBY

이름을 주는 형식과 KEEP IDENTITY

- 딕셔너리 위치까지 적용한 뒤 논리 Standby로 전환
- 새 이름: DB_NAME·DBID 변경, 8자 초과는 ORA-02084
- KEEP IDENTITY: 이름·DBID 유지, 롤링 업그레이드용
- 전환 후 LOGICAL STANDBY, CURRENT, GUARD ALL
- SHUTDOWN 후 MOUNT로 재기동, 목적지 조정

전환 명령이 끝나지 않을 때

무엇을 기다리는지부터 본다

- 오류 없이 프롬프트가 돌아오지 않음
- 알림 로그: 특정 시퀀스를 기다리는 메시지
- Primary가 조용하면 다음 로그가 안 생김
- BUILD 전에 MRP를 멈추지 않았으면 딕셔너리를 지나침
- 조치: 로그 전환, 그래도 안 되면 BUILD 재실행

열기와 첫 확인

OPEN RESETLOGS 다음 할 일

- ALTER DATABASE OPEN RESETLOGS
- START LOGICAL STANDBY APPLY IMMEDIATE
- Primary 로그 전환 후 DBA_LOGSTDBY_LOG 등록 확인
- V$LOGSTDBY_PROGRESS의 APPLIED_SCN, LATEST_SCN
- 초기 INITIALIZING은 딕셔너리 적재 중

12.6

SQL Apply 기동·중지와 실시간 적용

- IMMEDIATE와 실시간 적용
- STOP과 ABORT
- 상태 뷰와 아카이브 자동 삭제

기동과 실시간 적용

IMMEDIATE를 붙여야 실시간

- START LOGICAL STANDBY APPLY IMMEDIATE
- IMMEDIATE 없으면 아카이브된 로그부터 마이닝
- 전제는 물리 Standby와 같은 SRL 구성
- NODELAY: 설정된 지연을 무시
- SKIP FAILED TRANSACTION: 실패 트랜잭션 하나 건너뜀

중지: STOP과 ABORT

기다리느냐, 바로 멈추느냐

- STOP: 진행 중인 트랜잭션을 마무리하고 멈춤
- 대형 트랜잭션 처리 중이면 몇 분 걸릴 수 있음
- ABORT: 즉시 멈춤, 미커밋 부분은 재기동 후 재적용
- RECOVER MANAGED ... CANCEL은 쓰지 않음

상태를 보는 뷰

어디서 무엇을 보는지만 먼저

- V$LOGSTDBY_STATE: 전체 상태, 실시간 여부
- V$LOGSTDBY_PROCESS: 프로세스별 상태 메시지
- V$LOGSTDBY_PROGRESS: 적용 위치(SCN)
- DBA_LOGSTDBY_EVENTS: 실제 오류 이력
- ORA-16111, 16116, 16242는 정상 상태 메시지

아카이브 로그 자동 삭제

LOG_AUTO_DELETE와 보관 시간

- 적용 끝난 원격 아카이브 로그를 자동 삭제(기본 TRUE)
- LOG_AUTO_DEL_RETENTION_TARGET: 기본 1,440분 보관
- Flashback·시점 복구 때 과거 로그가 다시 필요
- FRA를 쓰지 않을 때만 의미가 있는 설정

12.7

DATABASE GUARD

- ALL, STANDBY, NONE
- SYS 예외와 세션 단위 해제
- Standby에서 도는 작업 관리

가드의 세 수준

열린 Standby를 보호하는 장치

- ALL: SYS 외 모든 데이터 변경 차단 (Logical 기본)
- STANDBY: SQL Apply가 유지하는 테이블만 차단
- NONE: 차단 없음 (Primary 기본)
- CDB 루트에서 설정, PDB에서는 ORA-65040
- V$DATABASE.GUARD_STATUS로 확인

SYS 예외와 세션 단위 해제

가장 위험한 것은 인식하지 못한 해제

- 가드는 SYS에 적용되지 않음
- ALTER SESSION DISABLE GUARD: 그 세션만 해제
- 세션 해제도 권한 필요, 없으면 ORA-01031
- GUARD NONE은 다시 켜는 것을 잊기 쉬움
- Standby 작업용 계정을 따로 둠

사례: SYS로 복제 테이블을 고친 뒤

다음 리두에서 멈춘다

- SYS로 복제 테이블에 열 추가, 가드가 막지 않음
- 다음 INSERT에서 SQL Apply 중지
- DBA_LOGSTDBY_EVENTS에 ORA-26676 (열 개수 불일치)
- 구조 원복 후 INSTANTIATE_TABLE로 재적재
- 재발 방지: 전용 계정, GUARD STANDBY

Standby에서 도는 작업 관리

Scheduler의 DATABASE_ROLE

- 작업은 만든 DB의 현재 역할을 DATABASE_ROLE로 가짐
- Primary 작업은 복제되지만 Standby에서는 실행 안 됨
- Standby에서 만든 작업은 LOGICAL STANDBY 역할
- 양쪽 모두 돌려면 역할별 사본 두 개
- DBA_SCHEDULER_JOB_ROLES로 확인

12.8

선택적 복제와 필터링

- SKIP 규칙
- SKIP_ERROR, SKIP_TRANSACTION
- INSTANTIATE_TABLE

SKIP 규칙

일부 스키마·문장을 빼고 받는다

- DBMS_LOGSTDBY.SKIP(stmt, schema_name, object_name)
- stmt: DML, SCHEMA_DDL, CONTAINER 등
- 와일드카드 %, USE_LIKE로 표시
- SQL Apply를 멈춘 상태에서 추가
- 빠진 객체는 그 시점부터 Primary와 달라짐

SKIP_ERROR와 SKIP_TRANSACTION

멈추는 대신 넘어가게 하기

- SKIP_ERROR: 지정한 문장 종류의 오류를 기록만 하고 계속
- 실패해도 데이터에 영향 없는 문장(GRANT 등)에만
- SKIP_TRANSACTION: XIDUSN·XIDSLT·XIDSQN으로 하나 지정
- 트랜잭션 ID는 DBA_LOGSTDBY_EVENTS에서 확인
- 건너뛴 DDL은 Standby에서 손으로 처리

규칙 확인과 제거

DBA_LOGSTDBY_SKIP과 UNSKIP

- ERROR 열: N은 문장 건너뜀, Y는 오류 건너뜀
- INTERNAL SCHEMA 기본 규칙이 처음부터 있음
- 트랜잭션 규칙은 DBA_LOGSTDBY_SKIP_TRANSACTION
- UNSKIP 계열은 추가 때와 같은 인수로

INSTANTIATE_TABLE

Primary에서 테이블을 다시 가져온다

- Standby 테이블을 지우고 Primary 현재 데이터로 재생성
- DB 링크를 통한 Data Pump 사용
- 쓰는 때: SKIP 해제 후 재포함, 사고 후 재동기화
- SQL Apply를 멈춘 상태에서 실행
- 링크 계정 권한 부족 시 테이블이 사라진 채 남을 수 있음

12.9

SQL Apply 성능 조정

- APPLY_SET과 파라미터
- 서버 수의 관계
- MAX_SGA와 병목 판단

APPLY_SET과 파라미터

DBA_LOGSTDBY_PARAMETERS로 확인

- 변경 APPLY_SET, 기본값 복귀 APPLY_UNSET
- MAX_SERVERS, APPLY_SERVERS, PREPARE_SERVERS
- MAX_SGA: LCR 캐시 크기(MB)
- PRESERVE_COMMIT_ORDER, LOG_AUTO_DELETE 등
- 프로세스 수 변경은 SQL Apply 중지 후

서버 수의 관계

그릇과 내용물

- MAX_SERVERS가 상한(그릇)
- APPLY + PREPARE + 고정 3개(Reader, Builder, Analyzer)
- 상한을 넘으면 ORA-16081
- 늘릴 때는 MAX_SERVERS 먼저, 줄일 때는 APPLY_SERVERS 먼저
- PARALLEL_MAX_SERVERS의 여유도 함께 확인

MAX_SGA와 병목 판단

늘리기 전에 원인부터

- LCR 캐시가 모자라면 spill 증가, 적용 정체
- MAX_SGA를 키우면 공유 풀 크기도 검토
- Applier 한가: 마이닝 쪽, PREPARE_SERVERS
- Applier 모두 바쁨: APPLY_SERVERS, Standby 인덱스
- 특정 테이블 정체: 키 없는 테이블 정비

12.10

롤링 업그레이드와의 관계

- 버전 차이를 넘는 SQL Apply
- Transient Logical Standby와 DBMS_ROLLING
- 운영 시 주의점

버전이 다른 두 DB를 잇는 SQL Apply

Standby를 먼저 올리고 역할을 바꾼다

- Redo Apply는 같은 버전끼리만 가능
- SQL Apply는 낮은 버전의 변경을 높은 버전에 적용
- Standby 업그레이드 동안 Primary는 업무 계속
- 업무 중단은 Switchover 시간 정도
- 미지원 유형 제약은 업그레이드 중에도 그대로

Transient Logical Standby와 DBMS_ROLLING

업그레이드 기간에만 논리로

- 물리 Standby를 KEEP IDENTITY로 잠시 논리 전환
- 이름·DBID 유지 → 끝나면 옛 Primary를 물리로 복귀
- 수동 절차는 physru 스크립트로 자동화
- DBMS_ROLLING: 계획·전환·마무리를 명령 몇 개로
- DBMS_ROLLING은 ADG 옵션 라이선스 필요

운영 시 주의점

조용히 어긋나는 지점들

- 전환은 되돌릴 수 없음, 원복 수단 먼저
- 새 테이블이 생기면 미지원 목록 다시 확인
- SYS 접속은 장애 대응으로 한정
- 순서: BUILD 전 적용 중지, 규칙 전 SQL Apply 중지
- Primary의 링크·MV·ALTER SYSTEM은 Standby에서 따로

요약

- Logical Standby = SQL Apply로 갱신되는 열린 Standby
- Reader·Preparer·Builder → Analyzer·LSP0·Applier, 커밋 순서 유지
- 전환 전 내부 스키마, UNSUPPORTED, NOT_UNIQUE 점검
- 적용 중지 → BUILD → RECOVER TO LOGICAL STANDBY → OPEN RESETLOGS
- START ... IMMEDIATE로 실시간 적용, 중지는 STOP·ABORT
- GUARD ALL 기본·SYS 예외, SKIP·INSTANTIATE_TABLE·APPLY_SET

퀴즈 (1/2)

- Q1. Logical Standby에 대해 옳은 것은? → READ WRITE, 고유 인덱스 추가 가능
- Q2. LCR을 트랜잭션으로 조립하는 프로세스는? → Builder
- Q3. BAD_COLUMN = Y의 조치는? → 기본 키·고유 키 추가

퀴즈 (2/2)

- Q4. BUILD 전에 Redo Apply를 멈추는 이유는? → 딕셔너리를 지나치지 않게
- Q5. GUARD ALL에서 복제 테이블을 바꿀 수 있는 세션은? → SYS
- Q6. 서버 수를 기본값으로 되돌리는 순서는? → APPLY_SERVERS 먼저

실습 개요

- 01 전제 조건 점검 / 02 LogMiner 딕셔너리 (12.4, 12.5)
- 03 전환: 이름 형식, 대기 진단, 전환 후 값 (12.5)
- 04 SQL Apply 기동과 복제 확인 (12.3, 12.6)
- 05 골라서 복제하기와 조정: SKIP, APPLY_SET (12.8, 12.9)
- 06 데이터베이스 가드와 SYS 사고 (12.7)
- 07 되살리기와 원복: INSTANTIATE_TABLE, 스냅샷 (12.8)

Oracle Database 19c Active Data Guard 과정

Data Guard Broker 개요와 구성 생성

흩어진 파라미터를 하나의 구성으로 묶는다

제13장

학습 목표

- Broker가 덜어 주는 관리 부담을 설명한다
- DMON, 구성 파일, DG_BROKER_START의 역할을 구분한다
- 구성·멤버·인스턴스 객체 모델을 명령과 연결한다
- _DGMGRL 정적 서비스와 전제 조건을 점검한다
- 구성을 만들고 속성·상태·보호 모드를 바꾼다
- SQL 변경의 충돌을 이해하고 구성을 제거한다

Agenda

- 13.2 Broker가 해결하는 문제 / 13.3 Broker 아키텍처
- 13.4 구성 객체 모델 / 13.5 구성 생성 전제 조건
- 13.6 구성 생성 절차
- 13.7 속성과 EDIT 명령
- 13.8 Broker 관리 파라미터와 SQL 충돌
- 13.9 상태 전환과 보호 모드 / 13.10 비활성화와 제거 / 13.11 운영 시 주의점

13.2

Broker가 해결하는 문제

- SQL만으로 관리할 때의 부담
- Broker가 대신 맡는 일
- Broker 유무에 따른 비교

SQL만으로 관리할 때의 부담

설정은 여러 인스턴스의 파라미터에 흩어져 있다

- 전송은 Primary의 LOG_ARCHIVE_DEST_n에
- 갭 해소 대상은 Standby의 FAL_SERVER에
- 짝이 맞지 않으면 사람이 양쪽을 비교해야 한다
- 감시도 인스턴스마다 따로 조회
- 역할 전환은 순서가 정해진 SQL 여러 개

Broker가 대신 맡는 일

새 기능을 더하기보다 다루는 방식을 바꾼다

- 구성 생성과 Standby·Far Sync 추가 자동화
- 아무 멤버에 접속해도 구성 전체를 관리
- DMON이 상태를 계속 감시하고 통계 수집
- SWITCHOVER, FAILOVER 명령 한 줄
- Fast-Start Failover는 Broker가 전제

Broker 유무에 따른 관리 방식

EE 기본 기능, ADG 옵션 불필요

- 관리 단위: 데이터베이스별 → 구성 전체
- 전송·적용 설정: 파라미터 → 속성
- 감시: 인스턴스별 V$ 뷰 → 구성 단위 상태
- 역할 전환: 여러 사이트의 명령 → 명령 하나
- 사용은 선택, 단 한 구성에서 섞어 쓰지 않는다

13.3

Broker 아키텍처

- 클라이언트: DGMGRL, EM Cloud Control
- 서버: DMON 프로세스와 구성 파일
- DG_BROKER_START와 Broker 로그

클라이언트 구성 요소

명령을 전달할 뿐, 정보는 서버 쪽에 있다

- DGMGRL: 명령행 인터페이스, 스크립트 가능
- 구성의 어느 멤버에 접속해도 전체를 관리
- EM Cloud Control: 마법사, 버튼식 역할 전환
- EM은 이벤트를 메일 등으로 통지
- 실습은 DGMGRL과 SQL*Plus로 진행

DMON 프로세스

Broker가 관리하는 모든 인스턴스에 하나씩

- Primary, Standby, Far Sync 모두에서 실행
- 명령을 받아 로컬 처리, 다른 DMON과 통신
- 모든 멤버의 구성 파일을 같은 내용으로 유지
- 역할 전환 때 파라미터를 고쳐 쓴다
- RSM0, NSVn 등 보조 프로세스와 함께 동작

브로커 구성 파일

멤버마다 두 벌, 바이너리, 편집 불가

- dr1<db_unique_name>.dat, dr2<db_unique_name>.dat
- 기본 위치: Linux $ORACLE_HOME/dbs
- 멤버 목록, 역할, 속성, 의도한 상태를 기록
- 두 벌인 이유: 마지막 유효 상태 보존
- RAC는 DG_BROKER_CONFIG_FILEn을 공유 위치로

DG_BROKER_START와 Broker 로그

켜기만 해서는 아무것도 바뀌지 않는다

- 동적 파라미터, SCOPE=BOTH로 모든 멤버에서
- TRUE로 바꾸면 DMON 기동
- 기존 전송·적용에는 영향 없음
- 로그: trace 디렉터리의 drc<SID>.log
- 클라이언트 추적 SET TRACE_LEVEL, 서버는 TraceLevel

13.4

구성 객체 모델

- 구성, 멤버, 인스턴스
- 리두 경로의 표현
- 의도한 상태와 현재 상태

구성, 멤버, 인스턴스

명령 이름이 곧 대상 객체다

- 구성: 멤버 프로필의 집합, 보호 모드 보유
- 데이터베이스 멤버: Primary, 각종 Standby
- Far Sync 멤버: 리두를 받아 넘기는 인스턴스
- 멤버 이름은 DB_UNIQUE_NAME
- Primary 1 + Standby·Far Sync 최대 30

리두 경로의 표현

들여쓰기 한 단계 = 위 멤버에게서 받는다

- SHOW CONFIGURATION은 들여쓰기로 경로 표시
- 기본은 Primary가 모든 Standby로 직접 전송
- 경로 변경은 RedoRoutes 속성(10장)
- 기본값 NULL은 (LOCAL : ALL)
- RedoRoutes의 전송 방식이 LogXptMode보다 우선

의도한 상태와 현재 상태

Broker는 "어때야 하는가"를 기억한다

- 의도한 상태(intended state)를 구성 파일에 저장
- Standby는 보통 APPLY-ON
- 실제와 비교해 SUCCESS, WARNING, ERROR 판정
- 재기동 뒤에도 의도한 상태대로 적용 기동
- SQL로 켠 적용은 재기동하면 사라졌다

13.5

구성 생성 전제 조건

- 데이터베이스 측 조건
- Oracle Net과 _DGMGRL 정적 서비스
- 수동 원격 목적지 정리와 접속 권한

데이터베이스 측 조건

하나라도 빠지면 생성이나 첫 전환에서 막힌다

- Enterprise Edition, Primary는 ARCHIVELOG
- 모든 멤버가 SPFILE로 기동
- 모든 멤버가 MOUNT 이상
- 모든 멤버에서 DG_BROKER_START=TRUE
- COMPATIBLE 19.0.0 이상은 신기능용(필수 아님)

_DGMGRL 정적 서비스

꺼진 인스턴스에 접속할 통로

- 이름: <db_unique_name>_DGMGRL.<db_domain>
- 리스너 SID_LIST에 GLOBAL_DBNAME으로 등록
- 동적 서비스는 인스턴스가 꺼지면 사라진다
- Switchover 중 재기동에 필요
- 상태 UNKNOWN은 정적 등록의 정상 모습

수동 원격 목적지 정리

12.1부터 SERVICE 목적지가 있으면 추가 불가

- 대상: SERVICE 속성이 있는 LOG_ARCHIVE_DEST_n
- 남아 있으면 ORA-16698
- 로컬 LOCATION 목적지는 그대로 둔다
- Primary 목적지를 비우면 전송이 멈춘다
- 활성화 후 갭 해소로 따라온다

DGMGRL 접속과 권한

SYSDG 먼저, 실패하면 SYSDBA

- CONNECT sys@chicago, AS 절은 쓰지 않는다
- 원격 접속은 비밀번호 파일 인증
- 운영 계정은 SYSDG만 부여 권장
- ORA-16525: DMON 없음
- ORA-16532: DMON은 있으나 구성 없음

구성 생성 전 점검 순서

명령을 내기 전에 각 멤버에서 확인

- SPFILE 기동 여부
- DG_BROKER_START=TRUE
- 리스너의 _DGMGRL 정적 서비스
- 연결 식별자 양방향 해석과 SYSDBA 원격 접속
- SERVICE 목적지 유무

13.6

구성 생성 절차

- CREATE CONFIGURATION
- ADD DATABASE, ADD FAR_SYNC
- ENABLE CONFIGURATION과 Broker의 인수

CREATE CONFIGURATION

구성 파일이 이 순간 처음 생긴다

- Primary에 접속한 상태에서 실행
- PRIMARY DATABASE IS 뒤에는 DB_UNIQUE_NAME
- CONNECT IDENTIFIER는 다른 멤버가 쓸 식별자
- 식별자는 DGConnectIdentifier 속성으로 저장
- 생성 직후는 DISABLED, MaxPerformance

ADD DATABASE와 ADD FAR_SYNC

멤버 종류는 Broker가 판별한다

- ADD DATABASE 'boston' AS CONNECT IDENTIFIER IS boston
- 이름은 Standby의 DB_UNIQUE_NAME
- 연결 식별자 절은 사실상 필수
- 추가 직후 ORA-16905: 아직 활성화 전
- ADD FAR_SYNC도 같은 형식

ENABLE CONFIGURATION

이때부터 Broker가 책임진다

- 구성과 모든 멤버를 Broker 관리로 전환
- 속성을 파라미터로 내려 쓴다
- Standby에서 Redo Apply 기동(SRL 있으면 실시간)
- 처리는 비동기, 안정되기까지 수십 초
- 멤버 단위는 ENABLE DATABASE, ENABLE FAR_SYNC

활성화 후 Broker가 다시 쓰는 파라미터

손으로 쓴 것과 모양이 다르다

- 목적지: 기본값까지 모두 적은 긴 문자열
- VALID_FOR=(ONLINE_LOGFILE,ALL_ROLES)
- LOG_ARCHIVE_CONFIG를 멤버 목록으로 재작성
- Primary의 FAL_SERVER는 비워질 수 있다
- 적용은 의도한 상태에 따라 Broker가 기동

SQL 구성을 Broker로 옮기는 작업

운영 중단 없이 저녁 한 시간

- 전날: 전제 조건 점검, DG_BROKER_START 켜기
- 당일: 기존 목적지·LOG_ARCHIVE_CONFIG 값 기록
- 원격 목적지 비우기 → 생성 → 추가 → 활성화
- 비운 시점부터 활성화까지 전송 중단 구간
- 로그 전환으로 적용 진행 확인, 스크립트 정비

13.7

속성과 EDIT 명령

- 속성의 분류
- EDIT 명령과 주요 전송 속성
- 19c의 SET PARAMETER

속성의 분류

설정하는 값과 읽기만 하는 값

- 구성 속성: OperationTimeout, TraceLevel 등
- 설정 속성: LogXptMode, DGConnectIdentifier 등
- 상태 속성: StatusReport, InconsistentProperties 등
- 설정 속성은 구성 파일에 저장된다
- 상태 속성은 조회 시점에 계산, 변경 불가

EDIT 명령의 형태

객체 모델과 그대로 대응

- EDIT CONFIGURATION / DATABASE / FAR_SYNC / INSTANCE
- SET PROPERTY로 변경, RESET PROPERTY로 기본값
- 구성 파일, SPFILE, 메모리를 함께 수정
- 정적 파라미터 대응 속성은 재기동 후 반영
- 비활성 상태에서도 변경 가능, 활성화 때 반영

전송과 적용의 주요 설정 속성

목적지 문자열의 조각마다 이름이 붙었다

- LogXptMode, DGConnectIdentifier, LogShipping
- NetTimeout, ReopenSecs, MaxFailure, Binding
- RedoCompression(압축은 별도 옵션), DelayMins
- ApplyLagThreshold, TransportLagThreshold
- 속성은 리두를 받는 멤버 쪽에 설정

LogXptMode와 목적지 속성

세 모드 모두 SRL이 필요하다

- ASYNC = ASYNC NOAFFIRM
- FASTSYNC = SYNC NOAFFIRM (12.2 추가)
- SYNC = SYNC AFFIRM
- MaxProtection은 SYNC만 허용
- MaxAvailability는 SYNC 또는 FASTSYNC

19c의 SET PARAMETER

파라미터 계열 속성 일부가 디프리케이트

- LogArchiveTrace, StandbyFileManagement 등 9개
- DbFileNameConvert, LogFileNameConvert 포함
- 구성 파일에 저장되지 않는다
- EDIT DATABASE ... SET PARAMETER / RESET PARAMETER
- SET PROPERTY는 속성, SET PARAMETER는 파라미터

13.8

Broker 관리 파라미터와 SQL 충돌

- Broker가 관리하는 대상
- SQL로 바꾸면 생기는 일
- 회복과 운영 원칙

Broker가 관리하는 대상

파라미터와 적용 제어 SQL까지

- LOG_ARCHIVE_DEST_n, _STATE_n, LOG_ARCHIVE_CONFIG
- ARCHIVE_LAG_TARGET, LOG_ARCHIVE_* 계열
- STANDBY_FILE_MANAGEMENT, *_FILE_NAME_CONVERT
- 논리 Standby의 SQL Apply 설정
- RECOVER MANAGED STANDBY DATABASE 문장

SQL로 바꾸면 생기는 일

명령은 성공하고 문제는 나중에 온다

- ALTER SYSTEM은 그대로 성공한다
- 구성 파일 값과 실제 값이 어긋난다
- 즉시 경고가 뜬다는 보장이 없다
- 재활성화, 역할 전환, 속성 변경 때 덮어쓴다
- 필요한 변경이었다면 조용히 사라진다

어긋남의 회복과 운영 원칙

Broker를 켰으면 Broker로만 바꾼다

- 같은 값으로 전송 속성 하나를 다시 설정
- Broker가 목적지 문자열 전체를 다시 쓴다
- 필요한 변경은 먼저 속성에 넣고 나서
- 속성 있는 설정은 SET PROPERTY
- 디프리케이트된 파라미터는 SET PARAMETER

13.9

상태 전환과 보호 모드

- 멤버 상태 네 가지
- APPLY-OFF와 SQL 중지의 차이
- 보호 모드 변경 순서

멤버 상태 네 가지

EDIT DATABASE ... SET STATE

- TRANSPORT-ON / TRANSPORT-OFF: Primary 전용
- APPLY-ON / APPLY-OFF: Standby 전용
- APPLY-OFF는 의도한 상태를 바꾼다
- SQL로 멈추면 Broker는 이상으로 판정
- Standby 하나만 멈출 땐 LogShipping=OFF

열린 Standby와 상태 전환

라이선스 경계를 함께 본다

- READ ONLY WITH APPLY에서 APPLY-OFF → READ ONLY
- 다시 APPLY-ON → 실시간 조회 재개
- 실시간 조회는 Active Data Guard 옵션 필요
- 마운트 Standby의 APPLY-ON은 기본 기능
- 유지 보수 후 APPLY-ON을 체크리스트에

보호 모드 변경

EDIT CONFIGURATION SET PROTECTION MODE AS

- 보호 모드는 구성 전체의 설정
- 먼저 Standby의 LogXptMode를 SYNC로
- Primary에도 LogXptMode를 정해 둔다
- 그다음 MaxAvailability로 올린다
- 순서가 틀리면 ORA-16627

보호 모드를 올리고 내리는 순서

요구 조건이 빠지는 순간을 만들지 않는다

- 올릴 때: 전송 방식 → 보호 모드
- 내릴 때: 보호 모드 → 전송 방식
- Protection Mode와 PROTECTION_LEVEL 함께 확인
- SYNC 전환 후 커밋 응답 시간 재측정
- 원거리라면 FASTSYNC나 Far Sync 검토

13.10

구성 비활성화와 제거

- DISABLE: 관리만 멈춘다
- REMOVE: 구성 정보까지 지운다
- EXPORT / IMPORT CONFIGURATION

비활성화

Data Guard는 계속 돈다

- DISABLE CONFIGURATION / DATABASE / FAR_SYNC
- 전송과 적용은 SPFILE 설정대로 계속
- 사라지는 것은 Broker로 관리하는 능력
- Primary는 구성 전체로만 비활성화
- 구성 파일 위치 변경 전에는 반드시 비활성화

제거

기본 동작은 목적지까지 지운다

- REMOVE DATABASE, REMOVE FAR_SYNC
- REMOVE CONFIGURATION: 프로필과 구성 정보 삭제
- 데이터베이스 자체는 영향 없음
- PRESERVE DESTINATIONS로 목적지 보존
- Fast-Start Failover가 켜져 있으면 제거 불가

구성 메타데이터 보존

구성 파일은 RMAN 백업에 들어가지 않는다

- EXPORT CONFIGURATION TO '파일명'
- trace 디렉터리에 텍스트로 저장
- IMPORT CONFIGURATION FROM '파일명'
- 버전 다운그레이드 후 구성 복원에 사용
- 내보낸 파일은 별도 위치로 복사 보관

13.11

운영 시 주의점

- 혼합 운영 금지
- 평소에 드러나지 않는 설정 점검
- 의도한 상태를 절차에 반영

운영 시 주의점

평소에는 티가 나지 않는 것들

- 모든 멤버를 Broker로, 혼합 운영 금지
- 기존 스크립트의 SQL 적용 제어를 DGMGRL로
- 리스너 변경 때 _DGMGRL 정적 서비스 확인
- 연결 식별자는 모든 멤버에서 같게
- DG_BROKER_START는 SCOPE=BOTH로

요약

- Broker: 생성·관리·감시·역할 전환을 구성 단위로
- DMON + 구성 파일 dr1/dr2, DG_BROKER_START, drc 로그
- 멤버 이름은 DB_UNIQUE_NAME, Primary 1 + 최대 30
- 전제: SPFILE, _DGMGRL 정적 서비스, SERVICE 목적지 정리
- CREATE → ADD → ENABLE, 속성은 SET PROPERTY
- SQL 변경은 덮어쓰임, 상태 전환과 보호 모드 순서

퀴즈 (1/2)

- Q1. 접속 후 ORA-16525이면? → DG_BROKER_START 확인
- Q2. ADD DATABASE에서 ORA-16698이면? → SERVICE 목적지 정리
- Q3. 구성 파일은 멤버당 몇 벌? → 두 벌

퀴즈 (2/2)

- Q4. _DGMGRL 정적 서비스가 필요한 이유는? → 꺼진 인스턴스 기동
- Q5. 관리 파라미터를 SQL로 바꾸면? → 다음 속성 적용 때 덮어씀
- Q6. 보호 모드 변경에서 ORA-16627이면? → LogXptMode 먼저

실습 개요

- 01 전제 조건 점검 / 02 Broker 켜기 (13.3, 13.5)
- 03 구성 만들기와 ORA-16698 (13.5.3, 13.6)
- 04 활성화와 인수 확인 (13.6.3, 13.6.4)
- 05 상태 제어와 속성 (13.7, 13.9)
- 06 SQL로 바꾸면 (13.8)
- 07 재기동 검증과 스냅샷 (13.4.3)

Oracle Database 19c Active Data Guard 과정

Broker 모니터링과 검증 명령

지금 괜찮은가, 지금 넘길 수 있는가

제14장

학습 목표

- DMON의 상태 수집과 SUCCESS·WARNING·ERROR 집계를 설명한다
- SHOW 계열 출력과 ORA-166xx 코드로 문제 위치를 좁힌다
- 모니터링 전용 속성으로 경고의 원인을 찾는다
- 지연 임계값 속성으로 경고 기준을 정한다
- VALIDATE 여섯 가지의 검사 대상을 구분한다
- SHOW와 VALIDATE의 차이, 구성 내보내기, Broker 로그를 다룬다

Agenda

- 14.2 Broker 모니터링의 원리
- 14.3 SHOW 명령으로 상태 읽기
- 14.4 모니터링 전용 속성 / 14.5 지연 임계값과 경고
- 14.6 VALIDATE 계열 검증 명령
- 14.7 SHOW와 VALIDATE의 차이 / 14.8 구성 내보내기와 가져오기
- 14.9 Broker 로그와 경고 로그 / 14.10 운영 시 주의점

14.2

Broker 모니터링의 원리

- DMON이 상태를 모으는 방식
- 상태 값과 집계 규칙
- ORA-166xx 코드 읽는 법

DMON과 상태 수집

화면의 상태는 대개 저장된 점검 결과

- 구성원마다 DMON 하나, 구성 파일을 메모리에 보관
- Primary DMON이 주기적으로 상태 점검을 주도
- 각 구성원 DMON이 자기 쪽 사실을 확인해 회신
- SHOW CONFIGURATION은 보관된 결과를 표시
- status updated 시간으로 결과의 신선도 확인

상태 값과 집계 규칙

가장 심각한 것이 위로 올라간다

- SUCCESS: 의도대로 동작, 경고 조건 없음
- WARNING: 동작하지만 주의 조건 있음
- ERROR: 의도한 상태를 만족하지 못함
- 구성원 상태 = 그 구성원의 가장 심각한 문제
- 구성 상태 = 구성원 상태를 다시 집계

Intended State가 판정 기준

의도와 실제를 견주는 구조

- Primary: TRANSPORT-ON / TRANSPORT-OFF
- 물리 Standby: APPLY-ON / APPLY-OFF
- APPLY-OFF에서 적용 중지는 정상
- APPLY-ON에서 적용 중지는 ORA-16766, ERROR
- 일부러 꺼 둔 기능은 경고 대상에서 빠진다

ORA-166xx 코드 읽는 법

구성, 구성원, 원인 순으로 내려간다

- 구성: 16607 실패, 16608 경고
- 구성원: 16809, 16810 여러 문제 묶음
- 원인: 16766 적용 중지, 16737 전송 오류
- 원인: 16853·16855 지연 초과, 16857 연결 끊김
- 원인: 16714·16715 속성 불일치, 16789 SRL

묶음 코드와 통신 코드

화면에 다 나오지 않는 경우

- 16809, 16810은 StatusReport로 펼친다
- 16664는 결과를 받지 못했다는 뜻
- 통신 오류 때 다른 상태 값은 오래된 정보
- DGM- 접두어는 DGMGRL 클라이언트 오류
- 정확한 문구는 oerr ora 번호로 확인

14.3

SHOW 명령으로 상태 읽기

- 구성에서 구성원, 인스턴스로 좁힌다
- 요약 항목과 VERBOSE 속성 목록
- Far Sync와 자동 전환의 표시

SHOW CONFIGURATION

구성 전체를 한 화면에

- 구성 이름과 보호 모드
- 들여쓰기는 리두가 흐르는 방향
- 문제 구성원 아래 Warning·Error 줄과 코드
- 쓰이지 않는 Far Sync는 (inactive)
- Fast-Start Failover 켜짐 여부, 구성 상태

SHOW CONFIGURATION의 변형

더 많이, 또는 지연만

- VERBOSE: 구성 수준 속성까지 표시
- TraceLevel, CommunicationTimeout 등
- OperationTimeout, FSFO 관련 속성
- 19c LAG 형태: 구성원별 지연을 모아 보기

SHOW DATABASE

구성원 하나의 요약

- Role, Intended State
- Transport Lag, Apply Lag와 computed 시점
- Average Apply Rate: 리두 생성 속도와 비교
- Real Time Query: ADG 옵션 기능(9장)
- 경고·오류 목록과 Database Status

SHOW DATABASE VERBOSE

속성 전체와 (monitor) 표시

- 요약 뒤에 속성 전체를 이름 = 값으로 나열
- 설정 속성: 값을 바꿀 수 있음
- 모니터링 속성: 값 자리에 (monitor)만 표시
- 모니터링 속성은 이름을 지정해 따로 조회
- 철자가 틀리면 ORA-16538

Far Sync, 인스턴스, 자동 전환

모니터링 관점에서만

- SHOW FAR_SYNC: Apply Lag 항목이 없음
- Far Sync 문제는 뒤의 Standby 지연으로 번짐
- SHOW INSTANCE: RAC처럼 인스턴스가 여럿일 때
- SHOW FAST_START FAILOVER: 설정과 Observer
- ORA-16819, 16820: Observer 부재 경고

SHOW로 문제를 좁히는 순서

위에서 아래로, 한 단계씩

- 1. SHOW CONFIGURATION: 문제 구성원과 갱신 시각
- 2. SHOW DATABASE / FAR_SYNC: 원인 코드
- 3. 묶음 코드면 StatusReport
- 3. 전송이면 LogXptStatus, 불일치면 Inconsistent
- 4. 그래도 모호하면 drc 로그와 경고 로그

14.4

모니터링 전용 속성

- 읽기 전용, 조회할 때 계산
- 보내는 쪽과 받는 쪽의 큐
- 속성 불일치와 대기 이벤트

설정 속성과 모니터링 속성

값을 주면 거부된다

- 모니터링 속성은 읽기 전용
- ORA-16538: 그런 속성이 없음
- ORA-16790: 값이 허용 범위 밖
- ORA-16568: 읽기 전용 속성에 값을 줌
- 조회 때마다 통신하므로 반복 조회는 피함

StatusReport와 LogXptStatus

문제 목록과 전송 상태

- StatusReport: 인스턴스별 심각도와 오류 문장
- 행이 없으면 정상
- LogXptStatus: Primary에서 조회
- 목적지별 STATUS와 ERROR
- Net 계열 오류면 3장의 접속 구성부터

속성 불일치 두 가지

메모리, SPFILE, 구성 파일

- InconsistentProperties: 세 값이 다른 항목
- MEMORY_VALUE, SPFILE_VALUE, BROKER_VALUE
- InconsistentLogXptProps: 전송 속성만 비교
- 경고 ORA-16714, 16715로 드러남
- Broker 관리 파라미터는 SQL로 바꾸지 않음

SendQEntries와 RecvQEntries

보낼 것과 적용할 것

- SendQEntries: Primary에서 아직 못 보낸 로그
- 정상이면 CURRENT 한 줄 정도
- RecvQEntries: Standby가 받고 아직 적용 안 한 로그
- 보낼 큐가 길면 전송 구간 문제
- 받은 큐가 길면 적용 구간 문제

큐 두 개로 병목 가리기

야간 배치 중 apply lag 경고

- 경고: ORA-16853, 처음 의심은 네트워크
- SendQEntries: CURRENT 한 줄, 전송 정상
- RecvQEntries: NOT_APPLIED 다수, 적용 지연
- TopWaitEvents: 데이터 파일 읽기 대기 상위
- 대응: 배치 분산, Standby I/O 점검

TopWaitEvents와 논리 Standby 속성

대기 성격과 SQL Apply 정보

- TopWaitEvents: 대기 시간 상위 5개 이벤트
- Standby에서는 유휴 대기가 상위인 것이 보통
- 적용 병목이면 I/O 계열이 올라옴
- LsbyFailedTxnInfo: SQL Apply를 멈춘 트랜잭션
- LsbyParameters: MAX_SGA, MAX_SERVERS

14.5

지연 임계값과 경고

- 세 임계값 속성
- 경고가 상태로 번지는 경로
- 업무 기준으로 값을 정한다

세 임계값 속성

19c 기본값은 모두 30초

- ApplyLagThreshold: apply lag, ORA-16853
- TransportLagThreshold: transport lag, ORA-16855
- TransportDisconnectedThreshold: ORA-16857
- 단위는 초, 구성원마다 지정
- 0이면 그 항목을 감시하지 않음

경고가 상태로 번지는 경로

구성원에서 구성 전체로

- 임계값 초과 → 구성원 WARNING
- 구성원 WARNING → 구성 WARNING
- 다음 상태 점검에서 반영, 즉시는 아님
- 내려가면 SUCCESS로 복귀, 이력은 로그에
- 임계값은 판정만 바꾸고 전송·적용은 그대로

임계값을 정하는 기준

RPO와 RTO에서 거꾸로

- TransportLagThreshold는 RPO 쪽
- ApplyLagThreshold는 RTO 쪽
- 평소 지연 분포를 먼저 기록
- 너무 짧으면 경고 피로, 너무 길면 무의미
- ASYNC 원거리 Standby는 별도 기준

지연 적용과 임계값

DelayMins를 쓰면 함께 바꾼다

- DelayMins: 실수 대비 시간을 산다
- 대가: Failover 전 밀린 리두 적용 시간
- ApplyLagThreshold는 지연 시간보다 넉넉하게
- TransportLagThreshold는 그대로 둔다
- 전송 지연은 여전히 실제 문제

14.6

VALIDATE 계열 검증 명령

- 여섯 가지 형태
- 전환 준비, 파라미터, 블록, 접속 경로
- 결과 파일을 남기는 명령

VALIDATE 여섯 가지

무엇을 검사하는가

- DATABASE: 역할 전환 준비 상태
- DATABASE ... SPFILE: 파라미터 차이
- DATABASE ... DATAFILE: 블록 비교, lost write
- FAR_SYNC: Far Sync 준비 상태
- NETWORK CONFIGURATION, STATIC CONNECT IDENTIFIER

VALIDATE DATABASE

역할 전환을 기준으로 한 점검

- Ready for Switchover / Ready for Failover
- Flashback Database Status: Reinstate 조건
- Managed by Clusterware
- 정적 접속 식별자로 실제 접속 시험
- Log Files Cleared: 전환 후 쓸 로그 준비

VERBOSE와 로그 구성 계산

현재와 전환 후를 함께

- 용량 정보, 임시 파일 수, 온라인 이동 여부
- 적용·전송 정보, Gap Status
- Current: 지금의 ORL과 상대 SRL 비교
- Future: 역할이 바뀐 뒤의 ORL과 SRL 비교
- Primary에도 SRL이 필요한 이유

VALIDATE DATABASE SPFILE

Primary와 Standby의 파라미터 차이

- 값이 다른 파라미터만 나열
- 접속은 DGConnectIdentifier 사용
- 전환 후 성능 차이를 미리 발견
- 원래 달라야 하는 항목도 섞여 나옴
- 표기만 다른 항목도 보고됨

VALIDATE DATABASE DATAFILE

블록 단위 비교와 lost write

- 내부적으로 DBMS_DBCOMP 호출
- Primary나 ALL이면 모든 Standby와 비교
- OUTPUT 필수, 디렉터리 포함 시 DGM-17138
- 결과는 대상 호스트의 trace 디렉터리
- 시작 메시지만 내고 돌아온다

DATAFILE 결과 읽기

0이어야 하는 열

- SAMEV, SAMEV&C: 같은 블록 쌍
- DIFFV: 적용 중이면 생길 수 있음
- LWLOC, LWRMT: lost write, 0이어야 함
- DIFFPAIR: 버전 같은데 내용 다름, 0이어야 함
- SKIPPED, ENCERR: 건너뛴 이유 확인

VALIDATE NETWORK CONFIGURATION

구성원 사이 접속을 양방향으로

- chicago에서 boston, boston에서 chicago
- 접속 문자열은 DGConnectIdentifier에서
- 한 방향만 되는 경우를 잡는다
- 정적 접속 식별자 시험도 함께 수행
- ALL 대신 구성원 이름으로 범위 지정

VALIDATE STATIC CONNECT IDENTIFIER

정적 서비스만으로 접속되는가

- StaticConnectIdentifier 속성 값 사용
- STATIC_SERVICE=TRUE를 붙여 새로 접속
- 동적 등록 서비스는 쓰지 않음
- 실패하면 ORA-12514와 Failed
- SHOW 상태 값에는 반영되지 않음

VALIDATE FAR_SYNC

Far Sync의 중계 준비

- Far Sync가 받고 다시 보낼 준비가 됐는지
- SRL 구성, 전송 대상 상태
- WHEN PRIMARY IS: 전환 후를 가정한 판정
- RedoRoutes가 역할별로 다를 때 유용
- Far Sync 구성에서는 전환 전 점검 목록에

14.7

SHOW와 VALIDATE의 차이

- 의도대로인가, 넘길 수 있는가
- 저장된 결과와 새 검사
- 감시용과 점검용으로 나눠 쓴다

두 명령이 묻는 질문

기준이 다르다

- SHOW: 의도대로 동작하는가
- VALIDATE: 지정한 작업을 지금 할 수 있는가
- SHOW: 저장된 점검 결과, 가벼움
- VALIDATE: 명령 시점에 접속해 새로 검사
- SHOW는 짧은 주기 감시, VALIDATE는 정기 점검

전송을 꺼 둔 채 잊은 Standby

SHOW는 조용했다

- 점검 중 LogShipping=OFF, 원복 누락
- SHOW CONFIGURATION: 계속 SUCCESS
- 지연 지표도 한동안 0에 가깝게 보임
- VALIDATE: Ready for Switchover No와 이유
- ON으로 되돌리자 갭 해소, 손실 없음

14.8

구성 내보내기와 가져오기

- 구성 메타데이터를 XML로
- 위치는 trace 디렉터리 고정
- 가져오기는 구성이 없을 때만

EXPORT CONFIGURATION

읽을 수 있는 구성 기록

- 이진 구성 파일 내용을 XML로 내보냄
- 위치는 trace 디렉터리, 경로 지정 시 ORA-16540
- TO 생략 시 SID_dmon_번호_brkmeta_n.trc
- Default="True"로 손댄 속성 구별
- 정기 내보내기와 diff로 변경 이력 관리

IMPORT CONFIGURATION

덮어쓰지 않는 가져오기

- trace 디렉터리의 파일을 읽어 구성 파일에 기록
- 구성이 있으면 ORA-16504로 거부
- 대표 용도: 다운그레이드 후 구성 복원
- 구성 파일 손상 시 REMOVE 후 IMPORT
- REMOVE부터 IMPORT까지 감시 공백

14.9

Broker 로그와 경고 로그

- drc 로그는 그동안의 기록
- 경고 로그와 나란히 읽기
- 스크립트와 감시 주기

drc<SID>.log

Broker의 활동 기록

- 경고 로그와 같은 ADR trace 디렉터리
- 구성원마다 각자의 파일
- 명령과 결과, 속성 변경, 상태 요약
- 화면의 낱말 대신 ORA-16607 등 코드로 기록
- 경고가 생기고 사라진 시각은 로그에만

경고 로그와 TraceLevel

무엇을 하려 했고, 무엇이 일어났나

- 경고 로그: Broker가 실행한 ALTER SYSTEM 등
- Redo Apply 기동·중지 기록
- drc 로그와 같은 시각을 나란히 비교
- TraceLevel: 기본 USER, 분석 때 SUPPORT
- 분석이 끝나면 USER로 되돌림

감시 스크립트와 주기

명령마다 맞는 주기가 있다

- dgmgrl -silent로 배너 없이 결과만
- SET ECHO ON, SHOW ALL은 클라이언트 설정
- 짧은 주기: SHOW CONFIGURATION 상태와 코드
- 하루: 로그의 ORA-166 계열 건수 집계
- 주·월, 전환 직전: VALIDATE 계열, EXPORT

14.10

운영 시 주의점

- SUCCESS만으로 전환을 판단하지 않는다
- 원복은 체크리스트로
- 결과 파일과 로그를 남긴다

운영 시 주의점

조용한 위험을 줄이는 습관

- 계획된 전환 전에는 VALIDATE DATABASE
- updated 시간이 늘면 통신부터 확인
- Broker 관리 파라미터는 속성으로 변경
- 유지 보수 때 바꾼 속성은 원복 체크리스트에
- EXPORT 결과는 trace 밖으로, DATAFILE은 결과 확인

요약

- DMON이 상태를 모아 SUCCESS·WARNING·ERROR로 집계, 기준은 Intended State
- ORA-166xx는 구성, 구성원, 원인 순으로 읽는다
- SHOW CONFIGURATION → DATABASE → 모니터링 속성
- 두 큐로 전송 구간과 적용 구간을 가른다
- 임계값 3종 19c 기본 30초, 업무 기준으로 조정
- VALIDATE는 전환 가능성, EXPORT는 기록, drc 로그는 이력

퀴즈 (1/2)

- Q1. ORA-16809가 보이면 먼저? → StatusReport 조회
- Q2. APPLY-OFF에서 적용 중지 상태는? → SUCCESS
- Q3. 값을 설정할 수 없는 속성은? → RecvQEntries (ORA-16568)

퀴즈 (2/2)

- Q4. 보낼 큐 비고 받은 큐가 쌓이면? → 적용 구간 병목
- Q5. SHOW는 SUCCESS, 전환 불가인 원인은? → LogShipping OFF
- Q6. 구성이 있을 때 IMPORT는? → ORA-16504로 거부

실습 개요

- 01 VALIDATE 읽기 / 02 SHOW와 VALIDATE의 차이 (14.6, 14.7)
- 03 모니터링 전용 속성 / 04 지연과 임계값 (14.4, 14.5)
- 05 V$ 뷰로 교차 확인 (15장 연결)
- 06 구성 내보내기 / 07 Fast-Start Failover 표시 (14.8, 14.3)
- 08 VALIDATE의 나머지 형태 (14.6)
- 09 Broker 로그 판독 (14.9)

Oracle Database 19c Active Data Guard 과정

V$ 뷰 기반 모니터링과 문제 해결

요약 너머의 숫자와 ORA- 번호를 읽는다

제15장

학습 목표

- SQL로 Data Guard를 감시하는 이유를 설명한다
- 역할별로 볼 V$ 뷰를 구분한다
- 전송 경로와 적용 경로의 상태를 판정한다
- 전송·적용·갭 문제를 증상에서 원인으로 좁힌다
- ORA- 오류 위치와 LOG_ARCHIVE_TRACE를 활용한다
- 상시 점검 스크립트와 Logical Standby 감시를 설계한다

Agenda

- 15.2 SQL로 Data Guard를 감시하는 이유
- 15.3 역할별 뷰 지도
- 15.4 전송 경로를 읽는 뷰 / 15.5 적용 경로를 읽는 뷰
- 15.6 증상에서 원인으로 좁히는 진단 순서
- 15.7 경고 로그와 LOG_ARCHIVE_TRACE / 15.8 상시 점검 스크립트 설계
- 15.9 Logical Standby 모니터링 / 15.10 운영 시 주의점

15.2

SQL로 Data Guard를 감시하는 이유

- Broker는 요약, 원인은 V$ 뷰에
- 정보는 역할마다 나뉘어 있다
- 증상, 위치, 원인, 과정의 네 층

Broker 요약과 V$ 뷰

경고 코드는 증상, ORA- 번호는 원인

- Broker 판정은 V$ 뷰와 경고 로그를 묶은 결과
- 끊겼다는 사실은 알려도 왜 끊겼는지는 모름
- 원인은 V$ARCHIVE_DEST.ERROR, 경고 로그에
- Broker 없는 구성은 V$ 뷰가 유일한 수단
- Broker가 멈춰도 V$ 뷰는 현재 값을 낸다

관찰 지점은 역할마다 다르다

보낸 쪽은 보낸 것만, 받은 쪽은 받은 것만 안다

- Primary: 보냈는가, 실패했는가
- Standby: 받았는가, 적용했는가, 얼마나 늦었는가
- 같은 V$ARCHIVED_LOG도 역할에 따라 내용이 다름
- 진단 쿼리는 양쪽에서 함께 돌린다

증상, 위치, 원인, 과정

위층에서 끝나면 아래로 내려가지 않는다

- 증상: SHOW CONFIGURATION, lag
- 위치: V$ARCHIVE_DEST, V$DATAGUARD_PROCESS
- 원인: ERROR 열, V$DATAGUARD_STATUS, 경고 로그
- 과정: LOG_ARCHIVE_TRACE 트레이스
- 대부분의 장애는 원인 층에서 끝남

15.3

역할별 뷰 지도

- 출발점은 V$DATABASE
- Primary와 Standby의 뷰 묶음
- 빈 결과를 읽는 법

V$DATABASE에서 시작한다

역할을 모르면 다른 뷰를 잘못 읽는다

- DATABASE_ROLE, OPEN_MODE로 역할과 상태 확인
- PROTECTION_MODE는 설정, PROTECTION_LEVEL은 실제
- 동기화가 깨지면 LEVEL이 RESYNCHRONIZATION
- SWITCHOVER_STATUS의 GAP 값도 함께
- FORCE_LOGGING이 YES인지 확인

Primary에서 보는 뷰

보낸 결과와 상대편의 상태

- V$ARCHIVE_DEST: 설정, 상태, 마지막 오류
- V$ARCHIVE_DEST_STATUS: 목적지 너머 Standby 상태
- V$ARCHIVED_LOG: 목적지별 아카이브 기록
- V$REDO_DEST_RESP_HISTOGRAM: SYNC 응답 시간
- V$DATAGUARD_STATUS: 최근 전송 사건

Standby에서 보는 뷰

받았는가, 적용했는가, 얼마나 늦었는가

- V$DATAGUARD_PROCESS: 수신·적용 역할과 ACTION
- V$STANDBY_LOG: SRL이 리두를 받는가
- V$DATAGUARD_STATS, V$ARCHIVE_GAP
- V$RECOVERY_PROGRESS: 적용 속도
- V$STANDBY_EVENT_HISTOGRAM: 지연의 분포

빈 결과를 읽는 법

정상이라서 빈 것인가, 잘못 물어서 빈 것인가

- Standby의 V$ARCHIVE_GAP이 비면 갭 없음
- Primary의 V$ARCHIVE_GAP은 원래 비어 있음
- Primary의 V$DATAGUARD_STATS는 지표가 비어 있음
- 마운트 상태 Standby는 DBA_ 뷰를 못 씀
- 점검 스크립트는 V$ 뷰만으로 구성

15.4

전송 경로를 읽는 뷰

- V$ARCHIVE_DEST의 STATUS와 ERROR
- VALID_NOW 판정
- V$ARCHIVE_DEST_STATUS, SRL, 응답 시간 분포

V$ARCHIVE_DEST의 STATUS

한 단어로 요약된 목적지 상태

- VALID, INACTIVE, DEFERRED, ERROR
- DISABLED, BAD PARAM, ALTERNATE, FULL
- 19c는 31개 목적지가 모두 행으로 나옴
- TARGET='STANDBY'로 원격 목적지만 거름

ERROR 열과 재시도

원인은 ORA- 번호로 남는다

- ERROR: 마지막 오류의 번호와 메시지
- FAIL_DATE, FAIL_SEQUENCE: 언제, 몇 번 로그
- REOPEN_SECS: 재시도까지 대기(기본 300초)
- MAX_FAILURE: 재시도 한도

VALID_TYPE, VALID_ROLE, VALID_NOW

설정이 틀린 것과 지금 안 쓰이는 것을 구분

- YES: 현재 역할에 맞게 쓰이는 중
- WRONG VALID_TYPE: 해당 로그 유형이 없음
- WRONG VALID_ROLE: 현재 역할에서 쓰이지 않음
- UNKNOWN: 목적지 미정의
- Primary와 Standby의 목적지 구성은 비대칭

V$ARCHIVE_DEST_STATUS

Primary에서 목적지 너머를 본다

- DATABASE_MODE: 상대편이 마운트인지 읽기 전용인지
- RECOVERY_MODE: IDLE이면 적용 중지
- GAP_STATUS: NO GAP, RESOLVABLE, UNRESOLVABLE
- SRL, STANDBY_LOGFILE_COUNT
- APPLIED_SEQ#는 현재 순번보다 한두 개 뒤

V$STANDBY_LOG로 SRL 읽기

리두가 SRL을 거치고 있는가

- STATUS: UNASSIGNED와 ACTIVE
- ARCHIVED 값은 STATUS와 짝지어 해석
- 쓰레드마다 ACTIVE 하나, SEQUENCE#가 현재 순번
- 전부 UNASSIGNED면 SRL을 거치지 않는 중
- V$LOGFILE의 TYPE=STANDBY로 멤버 확인

V$REDO_DEST_RESP_HISTOGRAM

SYNC 응답 시간은 평균보다 분포로

- SYNC 목적지별 응답 시간 구간과 빈도
- 정상: 0~1초 구간에 몰리고 긴 구간은 드묾
- 긴 구간 빈도 증가 = 네트워크·Standby I/O 흔들림
- NET_TIMEOUT을 정하는 근거 자료
- 인스턴스 기동 이후 누적, 기간은 스냅샷 차이로

15.5

적용 경로를 읽는 뷰

- 프로세스와 APPLIED 열
- 적용 속도와 지연의 분포
- 사건 기록과 지표의 조합

V$DATAGUARD_PROCESS

적용은 살아 있는가, 무엇을 하는가

- managed recovery 행이 없으면 적용 미기동
- ACTION: APPLYING_LOG, WAIT_FOR_LOG, WAIT_FOR_GAP
- logmerger와 RFS async의 SEQUENCE# 비교
- RFS 행이 없으면 Primary와 연결 없음

V$ARCHIVED_LOG의 APPLIED

가장 많이 쓰고 가장 많이 오해하는 열

- 값은 YES, NO, IN-MEMORY
- 실시간 적용 중인 리두는 아직 이 뷰에 없음
- 최대 APPLIED 순번은 실제보다 한두 개 뒤
- 값이 오래 안 변해도 곧 중단은 아님
- Primary 쪽 값은 Standby보다 늦게 갱신

V$RECOVERY_PROGRESS

적용 속도를 숫자로 잰다

- Active Apply Rate: 실제 적용 시간 기준 속도
- Average Apply Rate: 대기 포함 평균
- Redo Applied, Last Applied Redo
- 기준은 Primary의 리두 생성 속도
- 속도 정상인데 시각이 멀어지면 공급 문제

V$STANDBY_EVENT_HISTOGRAM

지금 값이 아니라 지연이 쌓인 모양

- apply lag를 표본으로 잴 때마다 구간 카운트 증가
- 건강한 모양: 0초 구간이 압도적
- 긴 구간의 LAST_TIME_UPDATED로 사건 시각 추적
- STANDBY_MAX_DATA_DELAY를 정하는 근거

V$DATAGUARD_STATUS

경고 로그를 SQL로 읽는 창

- FACILITY: Transport, Apply, RFS, FAL 등 출처
- SEVERITY: Error, Fatal만 골라 상시 점검
- ERROR_CODE가 0이 아닌 행이 오류
- ORA-16037처럼 의도된 동작도 코드로 남음
- 메모리 기반, 재기동하면 사라짐

V$DATAGUARD_STATS의 진단 활용

값 하나보다 조합과 신선도

- transport lag 큼: 전송 구간
- transport lag 작고 apply lag 큼: 적용 구간
- DATUM_TIME이 오래되면 지표를 믿지 않음
- 두 서버의 시계를 같은 NTP 소스에 맞춤

15.6

증상에서 원인으로 좁히는 진단 순서

- 첫 관찰로 갈래를 정한다
- 전송, 적용, 갭 문제
- 자주 만나는 ORA- 오류

첫 관찰로 갈래 정하기

lag 두 개와 프로세스 목록

- transport lag 증가, RFS 없음 → 전송 문제
- apply lag만 증가 → 적용 문제
- MRP ACTION이 WAIT_FOR_GAP → 갭 문제
- PROTECTION_LEVEL 저하 → SYNC 전송 문제

전송 문제의 점검 목록

ERROR 열이 비어 있을 때

- tnsnames.ora 서비스 이름과 SERVICE 값 일치
- LOG_ARCHIVE_DEST_STATE_n이 ENABLE인가
- Standby 리스너와 인스턴스가 떠 있는가
- 추가한 목적지를 아직 적용하지 않았는가
- 리두 전송 인증(비밀번호 파일)이 맞는가

적용 문제

프로세스가 있는가, 무엇을 기다리는가

- managed recovery 없음: 미기동 또는 오류로 정지
- 정지 원인은 V$DATAGUARD_STATUS와 경고 로그
- WAIT_FOR_LOG 지속: 실시간 적용 꺼짐, DELAY
- APPLYING_LOG인데 lag 증가: 처리량 부족
- 처리량은 Active Apply Rate로 확인

갭 문제

적용은 순서대로만 진행된다

- Standby V$ARCHIVE_GAP: 빠진 순번 범위
- Primary GAP_STATUS: RESOLVABLE인가
- 자동 해소: ARCn ping과 FAL
- Primary에서 지워졌으면 수동 개입
- 19c: RECOVER STANDBY DATABASE FROM SERVICE

자주 만나는 ORA- 오류 ①

연결과 인증

- ORA-12541: Standby 리스너 없음
- ORA-12514: 리스너가 서비스를 모름
- ORA-01017, ORA-16191: 전송 인증 실패
- ORA-16047: DB_UNIQUE_NAME 불일치
- ORA-16012: 데이터베이스 식별자 불일치

자주 만나는 ORA- 오류 ②

전송 시간, SRL, FAL, 적용

- ORA-16198: SYNC 전송 중 NET_TIMEOUT 초과
- ORA-16086: Standby에 쓸 수 있는 SRL 없음
- ORA-16055: FAL 요청 거절
- ORA-16401: 이미 받은 로그 재수신 거절, 대개 무해
- ORA-16037: 사용자 적용 취소, 의도했다면 정상

15.7

경고 로그와 LOG_ARCHIVE_TRACE

- 오류가 남는 다섯 곳
- 추적 수준의 구조
- 켜고 끄는 절차

ORA- 오류가 남는 위치

같은 사건, 다른 양의 정보

- V$ARCHIVE_DEST.ERROR: 목적지별 마지막 오류
- V$DATAGUARD_STATUS: 최근 메시지, 메모리
- 경고 로그: 앞뒤 사건 전체
- 프로세스 트레이스, Broker 로그 drcSID.log
- 위치는 V$DIAG_INFO의 Diag Trace

LOG_ARCHIVE_TRACE의 수준

2의 거듭제곱을 더해 켠다

- 기본값 0, 추적 꺼짐
- 각 수준은 1, 2, 4, 8 … 32768
- 6 = 2(목적지 상태) + 4(작업 단계)
- 512 네트워크, 1024 RFS, 128 FAL, 2048 ping
- 4096 실시간 적용, 8192 Redo Apply

켜고 끄는 절차

다음 아카이브 작업부터 반영

- ALTER SYSTEM으로 열림·마운트 상태에서 변경
- 정수가 아니면 ORA-02017
- SCOPE=MEMORY로 켜면 재기동 시 자동 해제
- 트레이스 파일 이름에 프로세스 이름이 들어감
- Broker 구성은 Broker 쪽에서 변경

추적은 좁게, 짧게

진단 도구가 장애를 만들지 않도록

- 의심 구간에 맞는 수준만 선택
- 켠 시각과 끈 시각을 작업 기록에
- 끄는 것을 잊으면 ADR 디렉터리가 찬다
- 지원 요청에는 양쪽 경고 로그를 함께

15.8

상시 점검 스크립트 설계

- 설계 원칙
- Primary와 Standby 점검 항목
- 임계값과 판정

설계 원칙

사람이 읽지 않아도 판정이 나오게

- 역할을 먼저 판정하고 분기
- 결과는 숫자나 정해진 단어로
- 정상 기대값을 미리 정의
- lag는 DATUM_TIME과 묶어서
- 마운트 상태에서 동작, V$ 뷰만 사용

Primary 점검 항목

보냈는가, 보호 수준은 유지되는가

- STATUS<>'VALID'인 원격 목적지 수 = 0
- PROTECTION_MODE = PROTECTION_LEVEL
- 상대편 RECOVERY_MODE가 IDLE이 아님
- GAP_STATUS = NO GAP
- 최근 점검 이후 Error·Fatal 메시지 = 0

Standby 점검 항목

적용이 살아 있고, 늦지 않았는가

- managed recovery 행 수 = 1
- RFS 계열 행 1 이상
- V$ARCHIVE_GAP 행 수 = 0
- apply lag를 초로 환산해 기준과 비교
- SYSDATE와 DATUM_TIME 차이가 수십 초 이내

임계값과 운영 연계

기준은 업무에서 거꾸로 정한다

- transport lag ↔ RPO, apply lag·finish time ↔ RTO
- 주의와 경보 두 단계로 판정
- 양쪽을 같은 주기, 같은 시각에 실행
- Broker 임계값 속성과 함께 운영

15.9

Logical Standby 모니터링

- 물리 Standby용 뷰가 조용해지는 이유
- 상태와 진행, 트랜잭션
- 이벤트와 멈춘 적용

물리 Standby용 뷰가 조용해진다

적용 쪽 뷰 계열이 통째로 바뀐다

- MRP 대신 LSP와 SQL Apply 프로세스들
- 받은 로그는 DBA_LOGSTDBY_LOG에 등록
- V$ARCHIVE_GAP, V$RECOVERY_PROGRESS는 비어 있음
- 물리용 점검 스크립트는 정상으로 오판
- 전송 쪽 뷰는 물리 Standby와 같다

상태와 진행

STATE와 SCN으로 본다

- V$LOGSTDBY_STATE: APPLYING, IDLE, WAITING ON GAP 등
- REALTIME_APPLY: Y이면 실시간 적용
- V$LOGSTDBY_PROGRESS: APPLIED_SCN, LATEST_SCN
- LATEST 멈춤은 수신, APPLIED만 멈춤은 적용 문제
- RESTART_SCN 이전 로그는 삭제 가능

V$LOGSTDBY_TRANSACTION

지금 처리 중인 Primary 트랜잭션

- PRIMARY_XID, TYPE, MINING_STATUS, APPLY_STATUS
- ID는 Primary에서 부여된 트랜잭션 ID
- Standby의 실제 트랜잭션은 V$TRANSACTION
- 오래 머무는 트랜잭션이 지연의 후보

DBA_LOGSTDBY_EVENTS와 멈춘 적용

실패한 문장과 오류가 남는다

- EVENT_TIME, XID, EVENT, STATUS_CODE, STATUS
- 보관 건수는 MAX_EVENTS_RECORDED로 조정
- 원인을 고친 뒤 다시 시작
- SKIP FAILED TRANSACTION은 정합성을 사람이 책임
- 반복되는 유형은 SKIP 규칙 검토

15.10

운영 시 주의점

- 역할 확인 후 해석
- 과거 오류와 현재 오류 구분
- 메모리 뷰와 로그의 보존 차이

운영 시 주의점

조회 결과를 읽는 순간의 실수들

- 역할을 먼저 확인하고 값을 해석
- ERROR와 이력 오류는 시각과 함께
- 메모리 뷰는 재기동 시 사라짐, 로그로 보존
- Logical Standby 갈래를 점검에 따로
- 두 서버 시계 동기화

요약

- Broker는 요약, 원인 숫자와 ORA- 번호는 V$ 뷰와 경고 로그
- Primary는 전송, Standby는 수신·적용·지연을 안다
- V$ARCHIVE_DEST는 내 전송, _STATUS는 목적지 너머
- 적용은 managed recovery·ACTION·Apply Rate로 판정
- 진단: 갈래 정하기 → ERROR 열 → 경고 로그 → 트레이스
- 점검은 역할 분기·숫자 판정·신선도, Logical은 별도 뷰

퀴즈 (1/2)

- Q1. Broker 끊김 경고의 원인을 먼저 볼 곳은? → Primary ERROR 열
- Q2. 대비용 목적지의 WRONG VALID_ROLE은? → 정상
- Q3. APPLIED 최대 순번이 현재보다 1 작으면? → 실시간 적용 정상

퀴즈 (2/2)

- Q4. 리스너 복구 후에도 ERROR가 남으면? → REOPEN_SECS 대기
- Q5. LOG_ARCHIVE_TRACE=6이면? → 수준 2와 4
- Q6. Logical에서 물리용 점검이 무경보면? → 뷰가 비어 오판

실습 개요

- 01 로그 파일과 SRL / 02 아카이브 목적지 (15.3, 15.4)
- 03 전송 중단 재현과 진단 (15.6)
- 04 적용의 진행과 APPLIED (15.5)
- 05 LOG_ARCHIVE_TRACE (15.7)
- 06 상시 점검 스크립트 (15.8)
- 07 Logical Standby 모니터링 (15.9)

Oracle Database 19c Active Data Guard 과정

역할 전환과 이후 운영

Switchover, Failover, Reinstate, Fast-Start Failover, 백업 전략

제16장

학습 목표

- Switchover와 Failover를 구분하고 상황에 맞게 고른다
- VALIDATE와 VERIFY로 전환 가능 여부를 판정한다
- Broker와 SQL의 Switchover 절차와 내부 단계를 설명한다
- Failover의 종류와 데이터 손실을 판단하고 옛 Primary를 되살린다
- Fast-Start Failover의 Observer, 임계값, LagLimit를 설명한다
- 역할 기반 서비스, 전환 후 점검, Standby 백업 전략을 세운다

Agenda

- 16.2 역할 전환의 종류와 선택 / 16.3 Switchover 사전 점검
- 16.4 Switchover 실행 / 16.5 Failover
- 16.6 Failover 이후 옛 Primary 복구
- 16.7 Fast-Start Failover
- 16.8 역할 기반 서비스와 클라이언트 재접속
- 16.9 역할 전환 후 운영 점검 / 16.10 Data Guard 환경의 백업 전략

16.2

역할 전환의 종류와 선택

- Primary 역할과 Standby 역할
- 계획된 Switchover, 비상용 Failover
- 전환과 세션, 버퍼 캐시

두 역할과 역할 관리 서비스

구성 안의 Primary는 언제나 하나

- Primary와 Standby 역할은 서로 배타적
- Standby 하위 유형: 물리, 논리, 스냅샷
- 스냅샷 Standby는 물리로 되돌린 뒤 전환 대상
- 역할은 컨트롤 파일(V$DATABASE)과 Broker 구성 파일에 기록

Switchover와 Failover

계획 여부가 모든 차이를 만든다

- Switchover: 계획 전환, 손실 없음, 로그 리셋 없음
- Failover: 비상 전환, 손실 가능, RESETLOGS
- Switchover는 Primary에서, Failover는 Standby에서 시작
- 옛 Primary: 곧바로 Standby 대 구성에서 배제

어느 쪽을 고르는가

장애라고 해서 무조건 Failover는 아니다

- Primary가 열리고 통신되면 Switchover
- 재기동, 파일 복구로 끝나는 장애가 많음
- 판단 기준: RTO와 RPO
- 기준과 결정권자를 절차서에 미리 적어 둔다

전환과 세션, 버퍼 캐시

ADG Standby가 Primary가 될 때

- Data Guard는 세션을 새 Primary로 옮기지 않음
- 18c부터 ADG Standby의 버퍼 캐시 유지
- STANDBY_DB_PRESERVE_STATES: NONE, SESSION, BUFFER, ALL
- 실시간 조회가 전제, ADG 라이선스 필요

16.3

Switchover 사전 점검

- 전환을 막는 조건
- SWITCHOVER_STATUS와 VALIDATE DATABASE
- SWITCHOVER TO ... VERIFY

전환을 막는 조건

명령 전에 판정한다

- Standby에 아카이브 로그 갭이 있음
- 과거 시점으로의 전환이 필요함
- Primary가 열려 있지 않고 열 수도 없음
- 지연 적용(DELAY)이 걸린 대상은 미리 해제

SWITCHOVER_STATUS 읽기

역할마다 정상값이 다르다

- Primary: TO STANDBY
- 물리 Standby: TO PRIMARY
- SESSIONS ACTIVE: 세션 정리와 함께 전환 가능
- RECOVERY NEEDED, NOT ALLOWED: 원인부터 확인

Broker의 VALIDATE DATABASE

가장 넓은 범위를 한 번에 점검

- Ready for Switchover, Ready for Failover 두 줄이 판정
- Flashback Database 상태로 Reinstate 가능성 예측
- 전환 뒤 로그 구성: 옛 Primary의 SRL 확인
- Primary와 대상 Standby 양쪽에 실행, VERBOSE로 상세

SQL의 SWITCHOVER ... VERIFY

Broker 없는 구성의 사전 검사

- Primary에서 ALTER DATABASE SWITCHOVER TO boston VERIFY
- 전환은 하지 않고 검사만 수행 (12.1부터)
- ORA-16475: 가능하지만 경고 있음, alert log 확인
- ORA-16470: 대상의 Redo Apply가 돌지 않음

역할을 바꿔도 동작하는 설정

전환 30분 전 점검 순서

- Primary에도 Standby와 같은 SRL
- LOG_ARCHIVE_DEST_n의 VALID_FOR, 양쪽 FAL_SERVER
- Standby에 Primary와 같은 임시 파일
- 구성 상태, lag와 갭, VALIDATE, 정적 접속 경로 순으로 확인

16.4

Switchover 실행

- Broker SWITCHOVER의 내부 단계
- SQL Switchover 절차
- Broker 구성에서 SQL로 전환하면

Broker SWITCHOVER의 내부 단계

명령 하나가 여섯 단계를 처리

- 상태 확인, 인스턴스 정리, 역할 교환
- 구성 파일에 새 역할 기록
- 옛 Primary 재기동과 Redo Apply 시작
- 새 Primary 열기와 리두 전송 시작
- 재기동은 정적 등록 또는 StaticConnectIdentifier로 접속

SQL Switchover 절차

12.1부터 Primary에서 한 번

- Primary: SWITCHOVER TO boston VERIFY
- Primary: SWITCHOVER TO boston (옛 Primary 종료)
- 새 Primary: ALTER DATABASE OPEN
- 새 Standby: STARTUP MOUNT 후 적용 기동
- 역할 중립 설정이 없으면 파라미터도 손으로

Broker 구성에서 SQL로 전환하면

막히지 않고 실행된다

- SQL 전환은 Broker가 거부하지 못함
- 구성 파일은 옛 역할, 컨트롤 파일은 새 역할
- 한동안 SUCCESS, 이후 ORA-16816
- 복구: 실제 역할 확인, 구성 재작성, ENABLE
- 규칙: Broker 구성의 전환은 DGMGRL로만

16.5

Failover

- 완전 Failover와 즉시 Failover
- 데이터 손실 판단과 대상 선택
- Broker와 SQL의 절차

Failover가 하는 일

한 방향 조작

- 옛 Primary를 구성에서 배제, Standby가 역할 인수
- 남은 리두 적용 후 RESETLOGS로 열림
- 갈림길 SCN: STANDBY_BECAME_PRIMARY_SCN
- 앞서 적용한 다른 Standby는 되감기 또는 재생성

완전 Failover와 즉시 Failover

기본은 완전, 즉시는 마지막 수단

- 완전: 받은 리두를 모두 적용, 손실 최소
- 즉시: 추가 적용 없음, 가장 빠름, 대개 손실
- 즉시 후에는 다른 Standby도 모두 정리 대상
- FAILOVER TO boston [IMMEDIATE]

데이터 손실 판단과 대상 선택

손실은 전송이 결정한다

- MaxProtection, MaxAvailability(동기 상태): 손실 없음
- MaxPerformance: transport lag만큼 손실 가능
- 옛 Primary가 마운트되면 FLUSH REDO TO boston
- 대상: 물리 Standby, transport lag가 가장 작은 쪽

Broker와 SQL의 Failover 절차

대상 Standby에서 실행

- DGMGRL: 대상에 접속해 FAILOVER TO boston
- SQL: 적용 CANCEL, FAILOVER TO boston, OPEN
- 갭이 해소되지 않으면 ACTIVATE PHYSICAL STANDBY
- 이후: 보호 모드 재설정, 옛 Primary 정리

Failover 전에 옛 Primary를 멈춘다

split brain 방지

- 네트워크만 끊긴 옛 Primary는 살아 있을 수 있음
- 양쪽이 동시에 쓰기를 받는 상황
- 옛 Primary 쪽 트랜잭션은 Reinstate 때 사라짐
- 수동 Failover 절차에 옛 Primary 종료·차단 포함

16.6

Failover 이후 옛 Primary 복구

- 그대로 붙일 수 없는 이유
- Flashback Database와 REINSTATE
- 다시 만들어야 하는 경우

옛 Primary를 그대로 붙일 수 없는 이유

갈림길 SCN 이후의 서로 다른 역사

- 옛 Primary에만 있는 리두가 남아 있을 수 있음
- 새 Primary는 갈림길 SCN에서 새 incarnation 시작
- 갈림길로 되감으면 공통 상태로 돌아감
- RMAN 복원보다 Flashback Database가 훨씬 빠름

Reinstate의 전제

플래시백은 Failover 전에 준비

- 장애 전부터 Flashback Database ON
- 갈림길 SCN까지의 플래시백 로그가 FRA에 남아 있음
- DB_FLASHBACK_RETENTION_TARGET, FRA 여유 공간
- 옛 Primary를 마운트할 수 있어야 함

Broker로 Reinstate

ORA-16661이면 되살릴 수 있다

- ORA-16661: Reinstate 필요, ORA-16795: 재생성 필요
- 옛 Primary는 STARTUP MOUNT까지만
- 새 Primary에 접속해 REINSTATE DATABASE chicago
- 원래 배치로는 Switchover로 복귀

SQL로 되살리기와 재생성

Broker가 하던 일을 순서대로

- 새 Primary에서 STANDBY_BECAME_PRIMARY_SCN 조회
- 옛 Primary: MOUNT, FLASHBACK DATABASE TO SCN
- CONVERT TO PHYSICAL STANDBY 후 적용 기동
- 불가능하면 DUPLICATE FOR STANDBY 후 ENABLE DATABASE

16.7

Fast-Start Failover

- Observer와 FSFO 대상
- 발동 조건, 임계값, LagLimit
- 19c의 다중 Observer와 observe-only

FSFO의 구성 요소

판단까지 자동으로

- Broker 구성에서만, DGMGRL이나 EM으로 설정
- Observer: DGMGRL에 포함된 OCI 클라이언트
- Observer는 Primary와 Standby가 아닌 제3의 서버에
- FastStartFailoverTarget으로 대상 지정
- Data Guard 기본 기능, ADG 옵션 불필요

언제 발동하는가

연결 상실과 상태 점검 조건

- Primary가 Observer와 대상 모두와 끊김 > Threshold(기본 30초)
- 인스턴스 장애, SHUTDOWN ABORT도 발동
- 기본 켜짐: Datafile Write Errors, Corrupted Dictionary·Controlfile
- 기본 꺼짐: Inaccessible Logfile, Stuck Archiver
- DBMS_DG.INITIATE_FS_FAILOVER로 애플리케이션이 요청

전제 조건과 데이터 손실 한도

보호 모드별 LogXptMode

- MaxProtection: SYNC (12.2부터, Standby 2개 이상)
- MaxAvailability: SYNC 또는 FASTSYNC
- MaxPerformance: ASYNC, FastStartFailoverLagLimit 이내
- 양쪽 SRL과 Flashback Database, Observer용 tnsnames
- PmyShutdown, AutoReinstate 기본 TRUE

구성 순서

대상, 모드, 임계값, 활성화, Observer

- 양쪽에 FastStartFailoverTarget을 서로 지정
- 보호 모드와 FastStartFailoverThreshold 설정
- ENABLE FAST_START FAILOVER
- START OBSERVER ... IN BACKGROUND
- SHOW FAST_START FAILOVER로 확인

19c의 Observer 운영

다중 Observer와 observe-only

- 12.2부터 구성당 Observer 최대 3개 (교재의 1개는 12.1 기준)
- 판단은 마스터 Observer 하나, 나머지는 대기
- SET MASTEROBSERVER TO, SHOW OBSERVER
- 19c: ENABLE FAST_START FAILOVER OBSERVE ONLY

Observer 위치와 FSFO 운영

오판 전환을 줄이려면

- Observer는 클라이언트가 가장 많은 쪽이나 제3 장소에
- 계획 작업은 정상 종료, 습관적 ABORT 금지
- 긴 작업 전 DISABLE, 작업 뒤 ENABLE
- Observer 프로세스도 감시 대상

16.8

역할 기반 서비스와 클라이언트 재접속

- 역할 기반 서비스
- 접속 기술자와 재시도
- TAF와 FAN

역할 기반 서비스

클라이언트는 서비스 이름 하나만 안다

- 서비스가 Primary 역할에서만 기동되게
- srvctl add service ... -role PRIMARY
- 조회용 PHYSICAL_STANDBY 서비스는 ADG 필요
- Restart 없으면 DBMS_SERVICE와 기동 트리거

접속 기술자와 재시도

두 주소, 하나의 서비스

- ADDRESS_LIST에 두 호스트를 모두 기재
- 서비스가 떠 있는 쪽에만 접속 성공
- CONNECT_TIMEOUT, RETRY_COUNT
- TRANSPORT_CONNECT_TIMEOUT으로 대기 단축

TAF와 FAN

재접속을 돕는 기술

- TAF: 끊기면 OCI가 자동 재접속, SELECT 이어 읽기
- 진행 중 트랜잭션은 롤백
- FAN: 상태 변화를 이벤트로 즉시 통지
- FCF: 연결 풀의 죽은 연결 정리와 재충전

16.9

역할 전환 후 운영 점검

- 성공 메시지 다음에 확인할 것
- 전환 훈련

역할 전환 후 운영 점검

위에서부터 차례로

- 역할과 열린 모드, Broker 구성 상태
- 리두 전송, Redo Apply, 동기화 지표
- 보호 모드, FSFO 상태
- 서비스와 애플리케이션 접속, 백업·배치·모니터링
- Failover 뒤에는 새 Primary의 전체 백업

전환 훈련을 정기 일정으로

Switchover는 운영 중에도 연습할 수 있다

- 분기나 반기마다 실제로 역할을 바꿔 운영
- 드러나는 문제 대부분은 Data Guard 바깥
- 하드코딩 호스트, 한쪽에만 있는 배치와 백업
- 소요 시간을 기록해 RTO 근거로

16.10

Data Guard 환경의 백업 전략

- Standby로 백업 오프로드
- 복구 카탈로그와 DB_UNIQUE_NAME
- 아카이브 로그 삭제 정책과 호환성

Standby에서 백업 받기

Primary의 부하를 덜어 준다

- 물리 Standby 백업은 Primary에서 그대로 사용
- DBID와 파일 번호가 같기 때문
- RMAN은 Standby에 TARGET으로 접속
- Standby의 BCT는 ADG 옵션 필요
- MAA 권장: 양쪽 모두 백업, 매일 증분

복구 카탈로그와 DB_UNIQUE_NAME

백업 기록을 양쪽이 공유하려면

- Standby 백업 기록은 Standby 컨트롤 파일에만
- 교재: 물리 Standby 구성에서 카탈로그 필수
- Standby는 별도 REGISTER 불필요
- CONFIGURE DB_UNIQUE_NAME ... CONNECT IDENTIFIER
- 카탈로그 없으면 CATALOG 명령으로 수동 등록

연관과 접근성

누가 만들었나, 누가 쓸 수 있나

- 모든 백업은 만든 DB의 DB_UNIQUE_NAME과 연관
- 디스크 백업: 연관된 DB만 접근
- 테이프(SBT) 백업: 모든 DB가 접근
- 연관 변경: CHANGE … RESET DB_UNIQUE_NAME

아카이브 로그 삭제 정책

백업했다고 지우면 갭이 생긴다

- Primary: APPLIED ON ALL STANDBY
- 적용 지연 구성: SHIPPED TO ALL STANDBY 고려
- 백업 받는 쪽: BACKED UP n TIMES TO DEVICE TYPE
- ALL을 빼면 필수 목적지만 따짐
- 역할 기준으로 정의해 전환 뒤에도 유지

백업 호환성

무엇을 바꿔 쓸 수 있나

- 데이터 파일, 아카이브 로그: 호환
- 컨트롤 파일: Standby용과 일반용 상호 복원
- SPFILE: 호환 안 됨, 사이트별 백업
- Logical Standby 백업은 Primary에 사용 불가

요약

- Switchover는 계획·무손실, Failover는 비상·RESETLOGS
- 전환 전 SWITCHOVER_STATUS, VALIDATE, VERIFY로 판정
- Broker 구성의 전환은 DGMGRL로만, SQL은 불일치 유발
- 완전 Failover가 기본, 손실은 전송 상태가 결정
- 옛 Primary는 Flashback과 REINSTATE, 불가하면 재생성
- FSFO는 Observer·Threshold·LagLimit, 19c 다중 Observer
- Standby 백업, 카탈로그, APPLIED ON ALL STANDBY

퀴즈 (1/2)

- Q1. Switchover와 Failover 뒤 온라인 로그는? → 유지 / RESETLOGS
- Q2. Primary를 열 수 없을 때 Switchover는? → 불가
- Q3. Broker 구성에서 SQL로 전환하면? → 실행되나 ORA-16816

퀴즈 (2/2)

- Q4. Failover 뒤 ORA-16661이면? → MOUNT 후 REINSTATE DATABASE
- Q5. LagLimit 30, 대상 지연 90초에서 Primary 소실? → 자동 전환 없음
- Q6. 카탈로그 없이 받은 Standby 백업이 Primary에 안 보이면? → 카탈로그 또는 CATALOG

실습 개요

- 01 전환 사전 점검 / 02 Switchover 실행 (16.3, 16.4)
- 03 전환 후 확인과 되돌리기 / 04 전환 거부 재현 (16.4, 16.9)
- 05 SQL 전환 사고와 복구 (16.4.3)
- 06 플래시백 준비 / 07 Failover와 Reinstate (16.5, 16.6)
- 08 관측자와 FSFO 실제 발동 (16.7)
- 09 Standby 백업 활용 (16.10)